

Reto 1 · Almi-Chain

Documentación del proyecto

ERP Odoo y TPV · Base de datos con RAID · Web en AWS · Recreativa con MAME · Arcade stick · Red

Alberto · Oier · Julen · Gontzal · Richar

1.º ASIR · Almi · Curso 2026-2027

Bilbao, octubre-noviembre de 2026

Índice

Introducción....6
Objetivo y alcance....6
Estado del proyecto (27 de octubre de 2026)....6
Origen de las evidencias....7
Uso de inteligencia artificial....7
Qué hay en cada carpeta....7
Máquinas del reto....8
Máquinas virtuales exportadas....9
Reglas del repositorio....9
0. Gestión del proyecto....10
Contenido....10
Estado de la gestión (27 de octubre de 2026)....10
Equipo y plazo....10
Decisiones del equipo e indicaciones del profesor....11
Nombres comunes....12
Red NAT ALMICHAIN....12
Contraseñas y secretos....13
Dónde se cumple cada punto del enunciado....13
1. Windows: equipo del Gerente y TPV de barra....15
Objetivo....15
Requisitos....15
Arquitectura....16
Instalación y configuración paso a paso....16
Seguridad aplicada....19
Pruebas....19
Problemas previstos y solución....20
Fuentes....21
2. Recreativa con MAME (1 núcleo, 1 GB, sin swap)....22
Objetivo....22
Requisitos....22

Arquitectura....22
Instalación y configuración paso a paso....22
Seguridad aplicada....28
Pruebas....28
Problemas encontrados y solución....29
Fuentes....30
3. Arcade stick con Arduino....31
Objetivo....31
Requisitos....31
Arquitectura y cableado....31
Programas....32
Instalación paso a paso....32
Pruebas....33
Seguridad aplicada....33
Problemas previstos y solución....34
Fuentes....34
4. Red: WAN, plano, Packet Tracer y repetidor WDS....35
Objetivo....35
Necesidades de comunicación (R01)....35
Línea WAN (R02)....35
Plan de direccionamiento (R03)....35
Plano y esquema lógico (R04)....36
Packet Tracer (R05-R06)....37
Routers reales (R07-R08)....38
Seguridad aplicada....38
Pruebas (R09-R10)....39
Problemas encontrados y previstos....40
Fuentes....40
5. ERP Odoo + TPV....41
Objetivo....41
Requisitos....41
Arquitectura....42
Instalación y configuración paso a paso....42

Seguridad aplicada....66
Pruebas....67
Problemas encontrados y solución....68
Fuentes....69
6. Web en AWS con Terraform, DuckDNS y HTTPS....70
Objetivo....70
Requisitos....70
Arquitectura....71
Archivos....71
Despliegue paso a paso....72
Seguridad aplicada....72
Pruebas....72
Problemas previstos y solución....73
Fuentes....73
7. Base de datos, RAID y copias de seguridad....74
Objetivo....74
Requisitos....74
Arquitectura....74
Modelo de datos....74
Instalación y configuración paso a paso....78
Pruebas de restauración (B10)....86
La VM exportada y la prueba de fallo de disco (B11)....87
Seguridad aplicada....92
Pruebas....92
Problemas encontrados y solución....95
Fuentes....95
Presupuesto global (G11)....96
Equipos del restaurante (P02)....96
Red (R02, R04)....97
Arcade stick (S01)....98
Software (P03)....98
Total....98
Resumen de pruebas, conclusiones y trabajo pendiente....99

Resumen de las pruebas....99
Conclusiones....99
Trabajo pendiente antes de la entrega (13 de noviembre)....100
Bibliografía....100
Índice de anexos....101
Anexos....102
Anexo A. Máquinas virtuales exportadas....102
Anexo B. Routers reales y repetidor WDS....105
Anexo C. Web: guía de despliegue....107
Anexo D. Web: explicación del código....111
Anexo E. Herramientas: cómo se hicieron las VM, las pruebas y las capturas....115

Introducción

Proyecto del **Reto 1 de 1.º de ASIR en Almi**: la infraestructura informática de **Almi Bites**, el restaurante de la cadena ficticia **Almi-Chain**. Equipo: **Alberto, Oier, Julen, Gontzal y Richar** · 16 de octubre – 13 de noviembre de 2026.

El diseño es para máquinas virtuales de VirtualBox en los PC de clase (Zorin OS); dónde se ha probado de verdad cada cosa está en «Origen de las evidencias». Lo único físico son los 2 routers, la impresora de tickets y el arcade stick con Arduino. La web está en AWS Academy.

Objetivo y alcance

Diseñar, montar, asegurar y probar la informática de un restaurante: puestos Windows para el gerente y los TPV, una recreativa Linux optimizada con un mando hecho con Arduino, la red del local (con Wi-Fi segura y un repetidor WDS), un ERP con TPV que factura solo, una web pública desplegada con Terraform y una base de datos propia con disco redundante y copias. Todo con **mínimo privilegio** y **seguridad por defecto**, documentado y con pruebas.

Fuera del alcance: comprar el material del presupuesto, la conexión real del local y los datos reales del restaurante (todos los datos son de ejemplo y ficticios).

Estado del proyecto (27 de octubre de 2026)

Lo que está hecho, lo que está preparado pero sin probar y lo que falta. “Probado” quiere decir que hay pruebas ejecutadas con su captura o transcripción; el origen de cada prueba está en el apartado siguiente.

Bloque	Estado	Pruebas	Qué falta
00 Gestión	En marcha	—	Reparto de tareas entre los 5, Anexo I firmado y Anexo II
01 Windows gerente y TPV	**Preparado, sin probar**	0 de 11	Montar las dos VM en clase, ejecutar los scripts y hacer las pruebas y capturas
02 Recreativa	**Hecho y probado** (en QEMU)	7 de 9	Benchmark dentro de la VM en VirtualBox y prueba con el stick
03 Arcade stick	**Preparado, sin probar** (los programas compilan)	0 de 6	Montaje físico, cableado y pruebas con la placa real
04 Red	**Preparado, sin probar**	0 de 14	Archivo `.pkt`, plano del local y configuración de los routers físicos
05 ERP Odoo	**Hecho y probado** (servidor de pruebas y VM en QEMU)	8 de 11 completas, 3 a medias	Envío real del correo (contraseña de aplicación de Gmail) e impresión en la impresora física
06 Web en AWS	**Preparado, sin desplegar**	0 de 10 (Lighthouse solo en local)	Despliegue en AWS Academy, capturas, SSL Labs y Lighthouse
07 Base de datos	**Hecho y probado**	15 de 15	Repetir en VirtualBox las

Bloque	Estado	Pruebas	Qué falta
	(servidor de pruebas y VM en QEMU)		pruebas del RAID (hechas en QEMU)
Presupuesto	**Incompleto**	—	Filas *por completar*

Origen de las evidencias

Para que se pueda valorar cada captura sabiendo de dónde sale:

Capturas	Dónde se hicieron
`05-erp-odoo/capturas/01–35` y `07-bd-raid-copias/capturas/01–12`	En un **servidor de pruebas Linux en la nube** (Ubuntu 24.04, sin VirtualBox) al que se le puso el nombre `almi-odoo`/`almi-bd` y las IP 192.168.10.30 y .20 en la interfaz local. Los comandos y las salidas son reales, pero **no son las VM de VirtualBox del aula**, y las "copias a la otra máquina" de esas capturas iban al mismo servidor. Las de terminal están dibujadas a partir de la salida real; las del navegador son de Chromium sin interfaz, con la barra de dirección dibujada
`07-bd-raid-copias/capturas/18` en adelante, `05-erp-odoo/capturas/36` en adelante y `02-linux-recreativa/capturas/*`	Arrancando los discos de las **VM exportadas (.ova) en QEMU** sin aceleración, por consola serie (la de MAME, por la pantalla de la VM arrancando con GRUB). No es VirtualBox. Las capturas de terminal hechas así llevan al lado su transcripción `.txt` con la salida real
Benchmark de MAME (bloque 02)	En el ordenador anfitrión limitado a 1 núcleo, **no dentro de la VM**
Bloques 01, 03, 04 y 06	**Sin capturas todavía**

Cómo se construyeron y probaron las VM, y con qué herramientas se hicieron las capturas: `herramientas/` (`herramientas/README.md`).

Uso de inteligencia artificial

Parte del contenido de este repositorio (scripts, configuraciones, documentación, capturas y la auditoría) se ha preparado con la ayuda de un asistente de IA (Claude, de Anthropic), a partir del enunciado, de la guía paso a paso del equipo y de las decisiones del equipo. Todo lo que se presenta como probado se ejecutó de verdad en los entornos que indica el apartado anterior; lo que no se ha podido ejecutar se marca como pendiente.

Qué hay en cada carpeta

Carpeta	Bloque	Qué contiene
`00-gestion`	Gestión del proyecto	Tablero de tareas, guías paso a paso, anexos
`01-windows-gerente-tpv`	Equipo del gerente y TPV	Scripts de PowerShell: usuarios, seguridad, quiosco, certificado de Odoo
`02-linux-recreativa`	Recreativa con MAME	Configuración de arranque directo

Carpeta	Bloque	Qué contiene
		a MAME, optimización a 1 núcleo y 1 GB sin swap, benchmark
`03-arcade-stick`	Arcade stick con Arduino	Cableado y programas del mando (mando USB o teclado USB)
`04-red`	Red	Plan de direccionamiento, configuración de Packet Tracer y de los routers con WDS
`05-erp-odoo`	ERP Odoo + TPV	Docker, HTTPS, configuración de Odoo, los 2 TPV, factura automática, copias
`06-web-aws`	Web en AWS	La web y su despliegue con Terraform, DuckDNS y HTTPS
`07-bd-raid-copias`	Base de datos	Esquema MariaDB, usuarios, consultas, RAID 1, copias y restauración
`diagramas`	—	Modelo entidad/relación del equipo (los diagramas de arquitectura están en cada README)
`docs`	—	Memoria del reto (Word y PDF), anexo de las máquinas virtuales y conclusiones
`herramientas`	—	Cómo se construyeron y probaron las VM exportadas y cómo se genera la memoria
`presupuesto`	—	Presupuesto global

Máquinas del reto

Máquina	Sistema	IP	Para qué
ALMI-ODOO	Ubuntu Server 24.04 + Docker	192.168.10.30	ERP Odoo 18 y los TPV, por HTTPS
ALMI-BD	Ubuntu Server 24.04	192.168.10.20	MariaDB con RAID 1 y copias cruzadas con ALMI-ODOO
ALMI-ARCADE	Ubuntu 24.04 mínimo + MAME	DHCP	Recreativa: 1 núcleo, 1 GB, sin swap
ALMI-GERENTE	Windows 10 Pro	DHCP	Equipo del despacho (usuario Gerente)
ALMI-TPV	Windows 10 Pro	DHCP	Los dos TPV de barra (usuarios TPV1 y TPV2) en modo quiosco
almichain-web	Ubuntu 24.04 en AWS	IP elástica	Web https://almichain2.duckdns.org

Todas las VM van en la **Red NAT `ALMICHAIN`** de VirtualBox (192.168.10.0/24, puerta de enlace 192.168.10.1). Cómo crearla: 00-gestion (`00-gestion/README.md`).

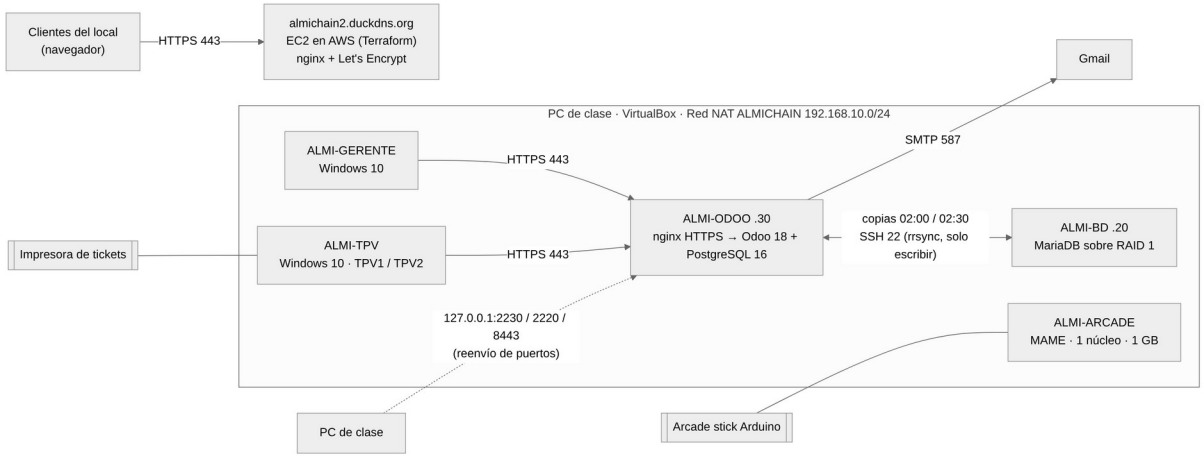


Figura 1. Diagrama

En el restaurante real cada grupo de equipos va en su VLAN (gestión, TPV, clientes y administración) con las reglas del bloque de red (`04-red/README.md`); en el laboratorio todas las VM comparten la Red NAT.

Máquinas virtuales exportadas

Las VM Linux (ALMI-ODOO, ALMI-BD y ALMI-ARCADE) están exportadas en formato **.ova**. No caben en el repositorio, así que se entregan aparte en trozos con un script para unirlos: ver `docs/maquinas-virtuales.md`. Para usarlas: VirtualBox → **Archivo** → **Importar servicio virtualizado** → elegir la .ova → **Importar**. Después, en **Configuración** → **Red**, poner el adaptador en **Red NAT** → **ALMICHAIN**. Las contraseñas no están en el repositorio. Cada VM genera sus propios secretos la primera vez que arranca y `almiadmin` tiene que cambiar su contraseña inicial al entrar (ver `docs/maquinas-virtuales.md`).

Reglas del repositorio

- Ninguna contraseña, token ni clave privada se sube al repositorio (ver `.gitignore`). Todas están en KeePassXC.
- Cada uno trabaja en la carpeta de su bloque. Commits con el formato `Bloque: qué se ha hecho`.
- Las capturas de cada bloque van en su carpeta `capturas/` y se citan en su README.

0. Gestión del proyecto

Contenido

Archivo	Qué es
`00-gestion/Guias-paso-a-paso-Reto1-Almi-Chain.docx`	Guías paso a paso de las 8 partes del reto, con las tareas G, P, A, S, R, E, W y B
`00-gestion/tablero`	Tablero tipo Trello con las 91 tareas (abrir `tablero/index.html`, instrucciones en `00-gestion/tablero/LEEME.md`). Se exporta a JSON en `tablero/` al final de cada sesión

> **La guía paso a paso es un documento de planificación** hecho antes de empezar. Las decisiones posteriores (sin doble arranque, Ubuntu en la recreativa en vez de Debian, dominio `almichain2`, modelo de datos de 13 tablas…) están en "Decisiones del equipo" y **mandan sobre la guía**. La guía numera las partes del 1 al 8 y las carpetas van del 00 al 07:

Parte de la guía	1	2	3	4	5	6	7	8
Carpeta	00-gestion	01-windows-gerente-tpv	02-linux-recreativa	03-arcade-stick	04-red	05-erp-odoo	06-web-aws	07-bd-raid-copias

Estado de la gestión (27 de octubre de 2026)

Entregable	Estado
Anexo I (contrato de trabajo en equipo)	**Pendiente**: hay un borrador en el Drive del equipo sin terminar ni firmar. No se sube al repositorio hasta quitarle los datos personales
Anexo II	**Pendiente**
Reparto de tareas en el tablero	**Pendiente**: las tareas del archivo no tienen responsable y no hay ningún `.json` exportado todavía
Presentación final (G15)	Pendiente (semana 4)
Commits de los 5	Pendiente: hasta ahora el historial solo tiene commits de un miembro. A partir de aquí, cada uno hace sus commits con su usuario de Git y con el formato `Bloque: qué se ha hecho`

Equipo y plazo

- **Equipo**: Alberto, Oier, Julen, Gontzal y Richar. El reparto **se hará** en el tablero (pendiente: ver «Estado de la gestión»).
- **Plazo**: 4 semanas, del 16 de octubre al 13 de noviembre de 2026.

Semana	Fechas	Objetivo
S1	16 – 23 oct	Análisis y diseño: anexos, repo, contraseñas, modelo E/R, plano de

Semana	Fechas	Objetivo
		red, boceto web, ERP elegido, material del stick
S2	26 – 30 oct	Instalaciones base: VM, Odoo, servidor con RAID, web maquetada, Packet Tracer
S3	2 – 6 nov	Integración y seguridad: TPV + impresora + Odoo, factura automática, web con HTTPS, WDS, stick, copias
S4	9 – 13 nov	Pruebas y entrega: restauración, fallo de disco, benchmarks, revisión cruzada, documentación, presentación

Decisiones del equipo e indicaciones del profesor

Se distingue lo que **indicó el profesor** (de palabra, en clase) de lo que **decidió el equipo** después. Las decisiones del equipo que cambian algo del enunciado necesitan la **conformidad del profesor por escrito** (un correo o el acta en el Anexo II): está **pendiente** de pedirla.

Tema	Decisión	Quién
Qué cuenta	La lista de tareas del enunciado (los RA no se tienen en cuenta)	Profesor
Equipos físicos	Solo 2 routers, 1 impresora de tickets y el arcade stick con Arduino	Profesor
Resto	Máquinas virtuales de VirtualBox en los PC de clase (Zorin OS, 16 GB de RAM)	Profesor
Red	Packet Tracer y 2 routers reales: uno da señal al otro como repetidor (WDS)	Profesor
ERP	Odoo	Profesor
Web	Geolocalización, HTTPS y dominio de DuckDNS, desplegada con Terraform en AWS Academy	Profesor
Base de datos	Con RAID y con pruebas	Profesor
Recreativa	1 núcleo y 1 GB, sin swap, MAME	Profesor
Seguridad	Mínimo privilegio: cada usuario solo accede a lo suyo	Profesor
Licencias	Windows 10 Pro/Education con licencia; todo lo demás gratis	Profesor
Gerente	**Sin doble arranque**: Windows 10 Pro en la VM ALMI-GERENTE y la recreativa en una VM aparte (ALMI-ARCADE). **El enunciado pide doble arranque: falta**	Equipo

Tema	Decisión	Quién
	escribir aquí el motivo y conseguir la conformidad del profesor	
TPV	Una sola VM, ALMI-TPV, con los usuarios TPV1 y TPV2, que simulan los dos equipos de barra. **Falta el motivo y la conformidad del profesor**	Equipo
Impresora	La comparten los dos TPV en el laboratorio (solo hay una física)	Equipo
ERP	Que haga algo funcional de verdad (factura automática al cobrar), sin usar todas las aplicaciones	Equipo
Recreativa	Ubuntu 24.04 mínimo en lugar de Debian (justificado en el bloque 02)	Equipo

Nombres comunes

Elemento	Nombre	Dónde
Equipo del gerente	ALMI-GERENTE	VirtualBox · Windows 10 Pro
TPV de barra	ALMI-TPV	VirtualBox · Windows 10 Pro · TPV1 y TPV2
Recreativa	ALMI-ARCADE	VirtualBox · Ubuntu 24.04 mínimo + MAME
Servidor de base de datos	ALMI-BD	VirtualBox · Ubuntu Server 24.04 · 192.168.10.20
Servidor del ERP	ALMI-ODOO	VirtualBox · Ubuntu Server 24.04 · 192.168.10.30
Servidor web	almichain-web	AWS Academy · EC2 creada con Terraform
Dominio	almichain2.duckdns.org	DuckDNS
Red interna de VirtualBox	ALMICHAIN	Red NAT 192.168.10.0/24
Administrador en todas las máquinas	almiadmin	Contraseña en el gestor de contraseñas

Red NAT ALMICHAIN

Una vez en cada PC que tenga máquinas del reto:

1. VirtualBox → **Archivo** → **Herramientas** → **Administrador de red** → pestaña **Redes NAT** → **Crear**.

2. Nombre `ALMICHAIN`, prefijo `192.168.10.0/24`, **Habilitar DHCP** → **Aplicar**. Después, acotar el DHCP para que no reparta las IP fijas de los servidores (.20 y .30). En una terminal del PC:

```
VBoxManage dhcpserver modify --network=ALMICHAIN --lower-ip=192.168.10.100 --upper-ip=192.168.10.199
VBoxManage list dhcpservers        # comprobar el rango
# Si dice que el servidor DHCP no existe, en algunas versiones se llama
```

```
NatNetworkALMICHAIN:
# VBoxManage dhcpserver modify --network=NatNetworkALMICHAIN --lower-ip=...
(pendiente de comprobar en clase)
```

3. En cada VM: **Configuración → Red → Adaptador 1 → Red NAT → ALMICHAIN**.
4. Para administrar desde el PC, en **Reenvío de puertos** de la red ALMICHAIN:

Nombre	Protocolo	IP anfitrión	Puerto anfitrión	IP invitado	Puerto invitado
odoo-ssh	TCP	**127.0.0.1**	2230	192.168.10.30	22
odoo-https	TCP	**127.0.0.1**	8443	192.168.10.30	443
bd-ssh	TCP	**127.0.0.1**	2220	192.168.10.20	22

La **IP anfitrión 127.0.0.1** es importante: si se deja vacía, VirtualBox abre esos puertos en todas las interfaces del PC y cualquier ordenador del aula podría intentar entrar por SSH en nuestros servidores. Con 127.0.0.1 solo se llega desde el propio PC.

Desde el PC: `ssh -p 2230 almiadmin@localhost` y Odoo en `https://localhost:8443` (el certificado incluye `localhost`).

Lo mismo por comandos: `VBoxManage natnetwork modify --netname ALMICHAIN --port-forward-4 "odoo-ssh:tcp:[127.0.0.1]:2230:[192.168.10.30]:22"` (y igual con las otras dos).

Contraseñas y secretos

Ninguna contraseña, clave de AWS ni token se escribe en el repositorio ni en la documentación. Están en la base **almichain.kdbx** de **KeePassXC**, que se comparte por USB, nunca por Git. En la documentación se escribe “contraseña guardada en el gestor”.

Dónde se cumple cada punto del enunciado

Punto del enunciado	Dónde
Necesidades, configuración y presupuesto de equipos e impresoras	01-windows-gerente-tpv (`01-windows-gerente-tpv/README.md`), `presupuesto`
Hardware y software necesario	01 (`01-windows-gerente-tpv/README.md`), 03 (`03-arcade-stick/README.md`), 04 (`04-red/README.md`)
Recreativa: instalación, optimización y monitorización	02-linux-recreativa (`02-linux-recreativa/README.md`)
Arcade stick con Arduino	03-arcade-stick (`03-arcade-stick/README.md`)
Necesidades de comunicación, WAN, plano, Wi-Fi segura y WDS	04-red (`04-red/README.md`)
Sistemas operativos	01 (`01-windows-gerente-tpv/README.md`), 02 (`02-linux-recreativa/README.md`), 05 (`05-erp-odoo/README.md`), 07 (`07-bd-raid-copias/README.md`)
Mínimo privilegio y seguridad por defecto	Apartado “Seguridad aplicada” de cada bloque
Web: diseño, HTTPS, AWS, dominio, despliegue reproducible, geolocalización, formulario y slideshow	06-web-aws (`06-web-aws/README.md`)
ERP + TPV con un caso real automatizado	05-erp-odoo (`05-erp-odoo/README.md`)

Punto del enunciado	Dónde
Impresora	01 (`01-windows-gerente-tpv/README.md`) y 05 (`05-erp-odoo/README.md`)
Modelo E/R, tablas, base de datos y consultas	07-bd-raid-copias (`07-bd-raid-copias/README.md`)
Disco redundante, plan de copias y restauración	07-bd-raid-copias (`07-bd-raid-copias/README.md`), 05 (`05-erp-odoo/README.md`)
Pruebas	Tabla “Pruebas” de cada bloque
Documentación con diagramas	Los README y `docs/` (`docs`)
Seguimiento y control de versiones	Este tablero y el historial de Git
Anexo I y Anexo II	Esta carpeta (**pendientes**, ver “Estado de la gestión”)

1. Windows: equipo del Gerente y TPV de barra

Objetivo

Preparar los puestos de Windows del restaurante con **mínimo privilegio**: el equipo del despacho para el **Gerente** (ALMI-GERENTE) y una VM con los dos **TPV de barra** (ALMI-TPV, usuarios TPV1 y TPV2) que solo abren su TPV de Odoo en **modo quiosco** e imprimen el ticket en la **impresora de tickets física**. Incluye las necesidades, el hardware y el presupuesto de los equipos reales.

Tareas del tablero: P01 a P12.

Requisitos

Dato	ALMI-GERENTE	ALMI-TPV
Sistema	Windows 10 Pro (o Education) con licencia	Windows 10 Pro (o Education) con licencia
Memoria / CPU	3072 MB / 2	3072 MB / 2
Disco	40 GB VDI dinámico	40 GB VDI dinámico
Pantalla	128 MB, VBoxSVGA	128 MB, VBoxSVGA
Red	Red NAT ALMICHAIN (DHCP)	Red NAT ALMICHAIN (DHCP)
USB	Habilitado	Habilitado (impresora)
Usuarios	almiadmin (administrador), Gerente	almiadmin (administrador), TPV1, TPV2

> Decisión del equipo: al final no se hace doble arranque. La recreativa es una VM aparte, ALMI-ARCADE (bloque 02), y el equipo del gerente solo lleva Windows.

Necesidades y equipos reales (P01-P03)

Puesto	Quién	Para qué	Restricciones	En el laboratorio
Equipo del despacho	Gerente	Ofimática, Odoo (facturas, personal, inventario), correo	Usuario estándar, sin instalar programas	ALMI-GERENTE, usuario Gerente
TPV de barra 1	Camareros	Solo cobrar en el TPV de Odoo e imprimir tickets	Quiosco: solo el TPV, sin escritorio ni USB	ALMI-TPV, usuario TPV1
TPV de barra 2	Camareros	Igual que el 1	Igual que el 1	ALMI-TPV, usuario TPV2
Impresora de tickets (×2 en el real)	TPV	Imprimir el ticket de cada venta	Solo la usa el TPV	1 impresora física compartida

Requisitos mínimos para la compra: equipo del gerente con 4 núcleos, 16 GB de RAM y SSD NVMe de 500 GB; TPV táctil todo en uno de 15” con 8 GB de RAM y SSD de 128 GB; impresora térmica de 80 mm ESC/POS con corte automático, USB + Ethernet y salida para cajón. El presupuesto con modelo, precio, enlace y fecha está en `presupuesto`.

Software	Dónde	Licencia y coste
Windows 10 Pro	Gerente y TPV	De pago, por equipo
Microsoft Edge	Gerente y TPV	Incluido en Windows
Odoo Community (TPV y ERP)	Servidor ALMI-ODOO, por navegador	LGPL, gratis
Controlador de la impresora	TPV	Gratis, del fabricante
Microsoft Defender	Todos	Incluido en Windows
KeePassXC	Equipo del gerente	Libre, gratis

Arquitectura

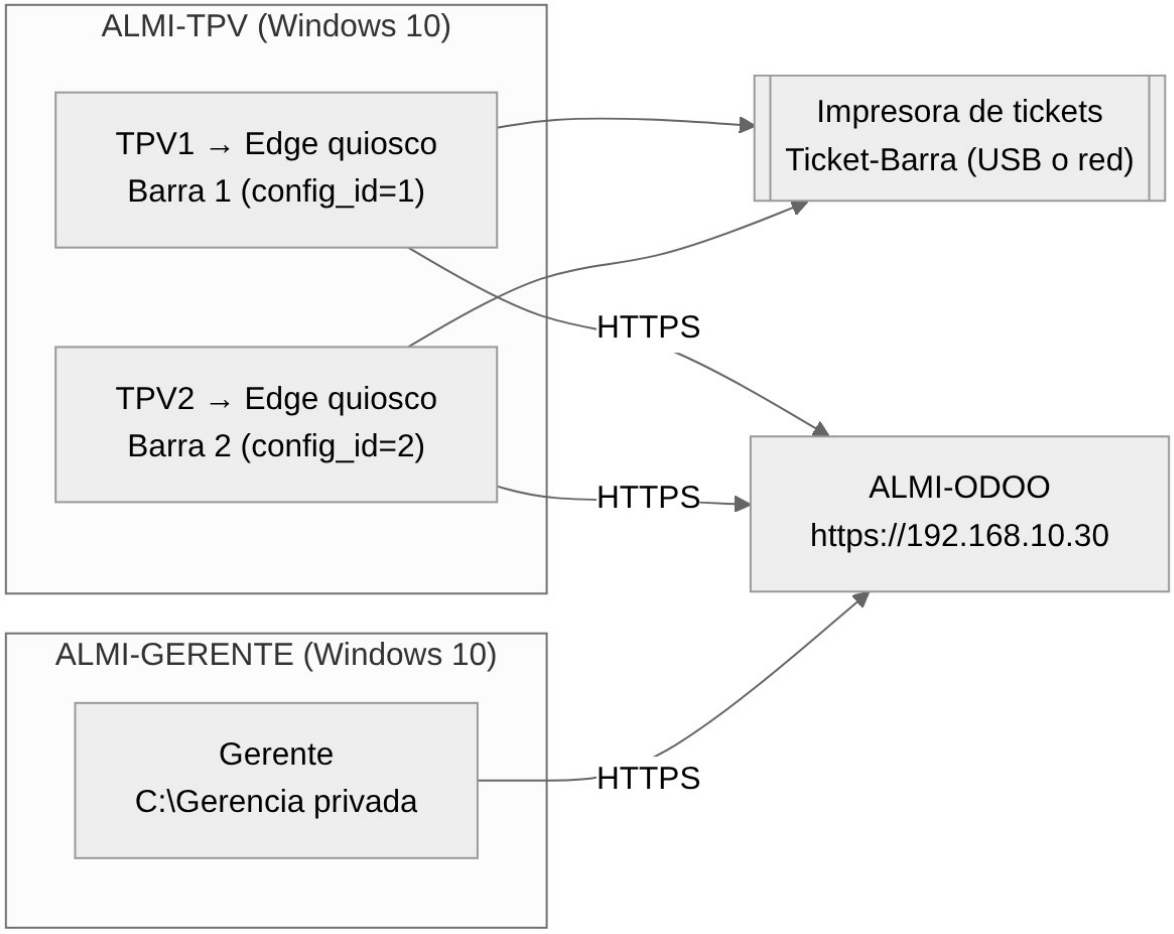


Figura 2. Diagrama

En el restaurante real habría un PC del gerente y dos TPV físicos con una impresora cada uno; en el laboratorio las dos barras comparten la impresora.

Instalación y configuración paso a paso

1. Crear las VM e instalar Windows (P04-P06)

1. VirtualBox → **Nueva** → nombre ALMI-GERENTE (y luego ALMI-TPV), ISO de Windows 10, **Omitir instalación desatendida**, datos de la tabla de requisitos, sin EFI.

2. Instalación personalizada en todo el disco, **cuenta sin conexión** `almiadmin` con contraseña de 12 caracteres o más (KeePassXC), privacidad en "No", diagnóstico "Requerido".
3. **Guest Additions** (Dispositivos → Insertar imagen de CD de las Guest Additions), activar Windows y **Windows Update** hasta que no haya más.
4. Nombre del equipo en PowerShell como administrador: `Rename-Computer -NewName "ALMI-GERENTE" -Restart` (o `ALMI-TPV`).
5. **Copiar los scripts** de la carpeta `01-windows-gerente-tpv/scripts` a la VM, por ejemplo a `C:\Almi\scripts`: con una carpeta compartida de VirtualBox (Dispositivos → Carpetas compartidas, de solo lectura) o descargando el repositorio en ZIP desde GitHub con `almiadmin`.

> **Riesgo: Windows 10 ya no tiene soporte** (fin de soporte el 14 de octubre de 2025): no recibe parches de seguridad salvo con el programa de pago ESU. Se usa porque es el Windows de clase con licencia. En el restaurante real, los equipos nuevos irían con **Windows 11 Pro** (ver el presupuesto).

2. Seguridad general (P08)

`01-windows-gerente-tpv/scripts/1-seguridad.ps1` en las dos VM, en PowerShell como administrador (`Set-ExecutionPolicy -Scope Process Bypass` antes). Va **antes que los usuarios**, porque Windows comprueba la directiva de contraseñas cuando se crea o se cambia una contraseña, no después:

- Invitado desactivado.
- Contraseñas de 12 caracteres con complejidad, caducidad de 90 días y **bloqueo de 15 minutos tras 5 fallos**.
- UAC en "Notificarme siempre": un usuario estándar no puede instalar nada sin la contraseña de `almiadmin`.
- Defender y cortafuegos activos en los tres perfiles.
- **Sin SMBv1** y **sin Escritorio remoto**.
- Sin reproducción automática.
- En ALMI-GERENTE, bloqueo de pantalla a los 5 minutos.

Los scripts buscan los grupos por su SID (`S-1-5-32-544` Administradores, `S-1-5-32-545` Usuarios), así que funcionan igual con Windows en español o en inglés, y se paran si algo falla (`$ErrorActionPreference = "Stop"` y comprobación de `secedit`, `net accounts` e `icacls`).

3. Usuarios con mínimo privilegio (P07)

Script	Dónde	Qué hace
`01-windows-gerente-tpv/scripts/2-usuarios-gerente.ps1`	ALMI-GERENTE	Usuario estándar **Gerente** y carpeta **`C:\Gerencia`** solo para Administradores, SYSTEM y Gerente
`01-windows-gerente-tpv/scripts/2-usuarios-tpv.ps1`	ALMI-TPV	Usuarios estándar **TPV1** y **TPV2**

Las contraseñas se piden al ejecutar el script (no están escritas en él), tienen que tener 12 caracteres o más y cumplir la complejidad (si no, el script se para), y se guardan en KeePassXC.

TPV1 y TPV2 son cuentas de quiosco: **no pueden cambiar su contraseña y no les caduca**, a diferencia del resto (90 días). Si caducara, la barra se quedaría bloqueada en mitad del servicio; la cambia `almiadmin` cuando toca.

Prueba de que la directiva funciona: `New-LocalUser -Name prueba -Password (ConvertTo-SecureString "1234" -AsPlainText -Force)` tiene que dar error (*InvalidPasswordException*).

4. Impresora de tickets física (P09)

Se instala **antes** del quiosco, porque después TPV1 y TPV2 ya no pueden tocar nada.

1. Apuntar marca, modelo y si tiene red (RJ45).
2. **Por USB**: VM apagada → Configuración → USB → filtro con la impresora → arrancar → instalar el controlador del fabricante. **Por red**: impresora en la misma red que el PC → *Agregar impresora por dirección TCP/IP* con su IP (en el restaurante real, 192.168.20.50 en la VLAN de TPV).
3. Nombre de la impresora: **Ticket-Barra**. Papel de 80 mm, corte al final y apertura de cajón.
4. Página de prueba (foto del ticket).
5. Desmarcar *Permitir que Windows administre mi impresora predeterminada* y poner **Ticket-Barra como predeterminada en TPV1 y en TPV2** (es por usuario).

5. Quiosco de TPV1 y TPV2 (P10)

En ALMI-TPV con `almiadmin`: `mmc` → Editor de objetos de directiva de grupo para **TPV1** y para **TPV2** → guardar como `Directivas-TPV.msc`. En **Configuración de usuario** → **Plantillas administrativas** de cada uno, en **Habilitada**:

Ruta	Directiva
Panel de control	Prohibir el acceso a Configuración de PC y a Panel de control
Sistema	Impedir el acceso al símbolo del sistema (también scripts)
Sistema	Impedir el acceso a herramientas de edición del Registro
Sistema → Opciones de Ctrl+Alt+Supr	Quitar Administrador de tareas
Sistema → Acceso de almacenamiento extraíble	Todas las clases de almacenamiento extraíble: denegar acceso a todo
Menú Inicio y barra de tareas	Quitar el menú Ejecutar del menú Inicio
Componentes de Windows → Explorador de archivos	Ocultar estas unidades especificadas en Mi PC (todas)
Componentes de Windows → Explorador de archivos	Impedir el acceso a las unidades desde Mi PC (todas)
Sistema	**Interfaz de usuario personalizada**: Edge en quiosco con su TPV

Línea de la interfaz personalizada (`01-windows-gerente-tpv/scripts/quiosco-TPV1.txt`); en TPV2 cambia `config_id=2`:

```
"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe" --kiosk
https://192.168.10.30/pos/ui?config_id=1 --edge-kiosk-type=fullscreen --kiosk-printing --no-
first-run
```

`--kiosk-printing` imprime el ticket directamente en la impresora predeterminada del usuario, sin cuadro de diálogo. Los `config_id` se han comprobado en Odoo: **Barra 1 = 1** y **Barra 2 = 2**.

`gpupdate /force`, cerrar sesión y entrar como TPV1: se abre Edge a pantalla completa con Barra 1. Antes hay que haber instalado el certificado de Odoo (apartado 6); si no, Edge enseña un aviso de seguridad que en modo quiosco no se puede saltar.

Primer inicio en cada TPV: la primera vez Odoo pide usuario y contraseña. Se entra con `barra1@almichain.local` en TPV1 y `barra2@almichain.local` en TPV2 (contraseñas en KeePassXC)

y Edge guarda la sesión en el perfil del usuario de Windows. Si un día la sesión de Odoo caduca, vuelve a salir esa pantalla y el encargado la repite.

6. Certificado de Odoo

`01-windows-gerente-tpv/scripts/3-certificado-odoo.ps1` en las dos VM, como `almiadmin`:

1. En ALMI-ODOO se apunta la huella: `sudo openssl x509 -in /etc/ssl/almichain/odoo.crt -noout -fingerprint -sha256`.
2. El script descarga la parte pública del certificado de `http://192.168.10.30/almichain-odoo.crt` (no usa SSH ni la contraseña del servidor), enseña su huella y **pide escribir la del servidor**. Si no coinciden, o si el certificado fuera de autoridad (`CA:TRUE`), no instala nada.
3. Si coinciden, lo instala en *Entidades de certificación raíz de confianza* del equipo. Edge muestra el candado sin avisos.

Comprobar la huella es lo que evita que alguien en la red sirva un certificado falso en ese momento. El certificado es de servidor (`CA:FALSE`), así que aunque esté en el almacén de raíces solo vale para Odoo.

Seguridad aplicada

Resumen: solo `almiadmin` es administrador; Gerente, TPV1 y TPV2 son usuarios estándar; contraseñas largas con bloqueo; UAC al máximo; Defender y cortafuegos activos; sin SMBv1, sin Escritorio remoto ni reproducción automática; TPV en quiosco sin panel de control, consola, registro, administrador de tareas, USB ni unidades; carpeta del gerente privada; Odoo solo por HTTPS con certificado de confianza.

Pruebas

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
1	Solo almiadmin es administrador	`Get-LocalGroupMember -SID S-1-5-32-544` en las dos VM (lo hace `01-windows-gerente-tpv/scripts/4-comprobar-tpv.ps1`, que además comprueba la impresora y que el 443 de Odoo responde y el 8069 no)	Solo almiadmin (y el Administrador integrado, desactivado)	Pendiente — en clase	
2	El gerente no puede instalar programas	Como Gerente, ejecutar un instalador	Pide la contraseña de administrador	Pendiente — en clase	
3	Contraseña corta rechazada	Como Gerente, cambiar la contraseña por 1234; como almiadmin, `New-`	Error de requisitos en los dos casos	Pendiente — en clase	

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
		`LocalUser -Name prueba -Password (ConvertTo-SecureString "1234" -AsPlainText -Force)`			
4	Bloqueo de cuenta	Fallar 5 veces la contraseña de Gerente	Cuenta bloqueada 15 minutos	Pendiente — en clase	
5	Carpeta privada	`icacls C:\Gerencia`	Solo Administradores, SYSTEM y Gerente	Pendiente — en clase	
6	TPV1 y TPV2 en quiosco	Iniciar sesión con cada uno	Solo Edge a pantalla completa con su barra	Pendiente — en clase	
7	TPV sin herramientas	Como TPV1, Ctrl + Alt + Supr	Sin Administrador de tareas	Pendiente — en clase	
8	TPV sin USB	Pendrive en la sesión de TPV2	Acceso denegado	Pendiente — en clase	
9	Impresión física	Cobrar en Barra 1 y en Barra 2	Dos tickets por la impresora sin preguntar	Pendiente — en clase	
10	Candado de Odoo	Abrir https://192.168.10.30 en Edge	Candado sin aviso	Pendiente — en clase	
11	Huella del certificado	Ejecutar `3-certificado-odoo.ps1` y escribir una huella equivocada	No instala nada	Pendiente — en clase	

Estado: este bloque no se ha podido probar todavía: necesita las VM de Windows 10 de clase (ISO y licencia), que no estaban disponibles donde se preparó el repositorio. Los scripts se han revisado a mano, pero **no se han ejecutado** (no había PowerShell). Las pruebas y las capturas se hacen en clase y se guardan en `01-windows-gerente-tpv/capturas`.

Problemas previstos y solución

Este bloque todavía no se ha montado (ver el estado en las pruebas), así que estos son problemas **previstos** (de la guía y de la experiencia de otros montajes), no encontrados. Cuando se monte en clase se apuntan aquí los que aparezcan de verdad.

Problema	Solución
Windows 10 obliga a usar cuenta Microsoft en la instalación	Desconectar el cable de red de la VM (Configuración → Red → desmarcar *Cable conectado*) para que deje crear cuenta local
La impresora predeterminada es por usuario	Ponerla a mano en TPV1 y TPV2 antes de aplicar el quiosco
Los scripts de PowerShell con tildes se veían mal	Guardados en UTF-8 con BOM

Fuentes

- Microsoft Learn: New-LocalUser, net accounts, Directivas de grupo y Microsoft Edge en modo quiosco.
- Guías paso a paso del equipo, parte 2.

2. Recreativa con MAME (1 núcleo, 1 GB, sin swap)

Objetivo

Una máquina recreativa con **MAME** para la sala del restaurante, en un equipo muy limitado: **1 núcleo y 1 GB de RAM, sin usar el disco como memoria (swap)**. Al encenderla entra sola en MAME, sin pedir contraseña, con un usuario sin privilegios. Se mide el rendimiento antes y después de optimizar y se monitoriza mientras se juega.

Tareas del tablero: A01 a A10.

Requisitos

Elemento	Valor
VM	**ALMI-ARCADE** · 1 CPU · 1024 MB · disco de 8 GB · **sin swap** · gráficos VMSVGA · Red NAT ALMICHAIN (DHCP) · USB activado (arcade stick)
Sistema	Ubuntu 24.04 LTS mínimo (sin escritorio)
Software	Servidor X mínimo (`xserver-xorg-core`, `xinit`), MAME 0.264, `alsa-utils`
Usuarios	`almiadmin` (administrador) y `arcade` (sin sudo, entra solo y abre MAME)

> **Decisiones del equipo.** No se hace doble arranque con el equipo del gerente: la recreativa es una VM aparte con exactamente 1 núcleo y 1 GB. Se usa **Ubuntu 24.04 mínimo** en lugar de Debian: el resultado es el mismo (sistema sin escritorio, solo X y MAME) y comparte paquetes y comandos con los servidores del reto. Como la VM tiene ya 1 núcleo y 1 GB, en GRUB se deja además `mem=1024M nr_cpus=1` para que el límite no dependa de la configuración de VirtualBox.

Arquitectura

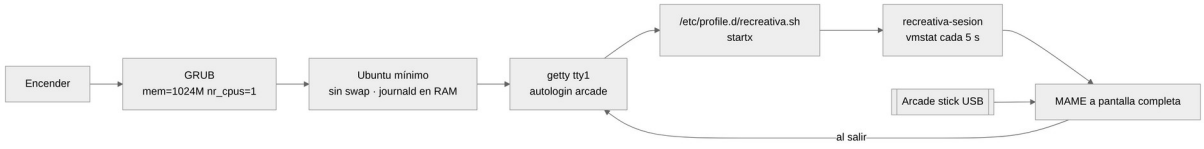


Figura 3. Diagrama

Instalación y configuración paso a paso

Los archivos están en `02-linux-recreativa/config` y `02-linux-recreativa/scripts`.

1. VM e instalación (A01-A02)

1. VirtualBox → **Nueva** → ALMI-ARCADE, Ubuntu (64-bit), **1024 MB y 1 CPU**, disco de 8 GB. Pantalla: VMSVGA. Red NAT ALMICHAIN. USB activado.

2. Instalar **Ubuntu Server 24.04** en modo **minimized** (**Omitir instalación desatendida** en VirtualBox), disco entero, nombre `almi-arcade`, usuario `almiadmin`, **sin** OpenSSH. (En la VM exportada el paquete `openssh-server` está instalado pero **desactivado**: `systemctl is-enabled ssh → disabled`.)

3. **Quitar el swap**. Ubuntu 24.04 no crea una partición de intercambio sino un **archivo** `/swap.img`, y el instalador no deja quitarlo. Después de instalar:

```
sudo swapoff -a
sudo sed -i '/swap.img/d' /etc/fstab
sudo rm -f /swap.img
swapon --show        # sin salida
free -m              # Swap: 0
```

2. 1 núcleo y 1 GB (A03)

`/etc/default/grub`:

```
GRUB_TIMEOUT=3
GRUB_CMDLINE_LINUX_DEFAULT="quiet mem=1024M nr_cpus=1"
```

`sudo update-grub` y reiniciar.



Figura 4. 1 núcleo, 961 MB de memoria total, swap 0, sin /swap.img y los parámetros del núcleo (console=tty0 console=ttyS0 los añade la prueba en QEMU para manejarla por la consola serie; la VM entregada no los lleva)

3. Usuarios y seguridad (A04)

```
sudo useradd -m -s /bin/bash -c "Recreativa" -G audio,video,input arcade
sudo passwd arcade
sudo chmod 700 /home/arcade /home/almiadmin
sudo apt install -y libpam-pwquality cracklib-runtime   # cracklib-runtime: el diccionario de contraseñas débiles
sudo sed -i 's/^# *minlen.*/minlen = 12/' /etc/security/pwquality.conf
sudo ufw default deny incoming && sudo ufw default allow outgoing && sudo ufw enable
```

`arcade` no está en el grupo `sudo`: no puede administrar nada.

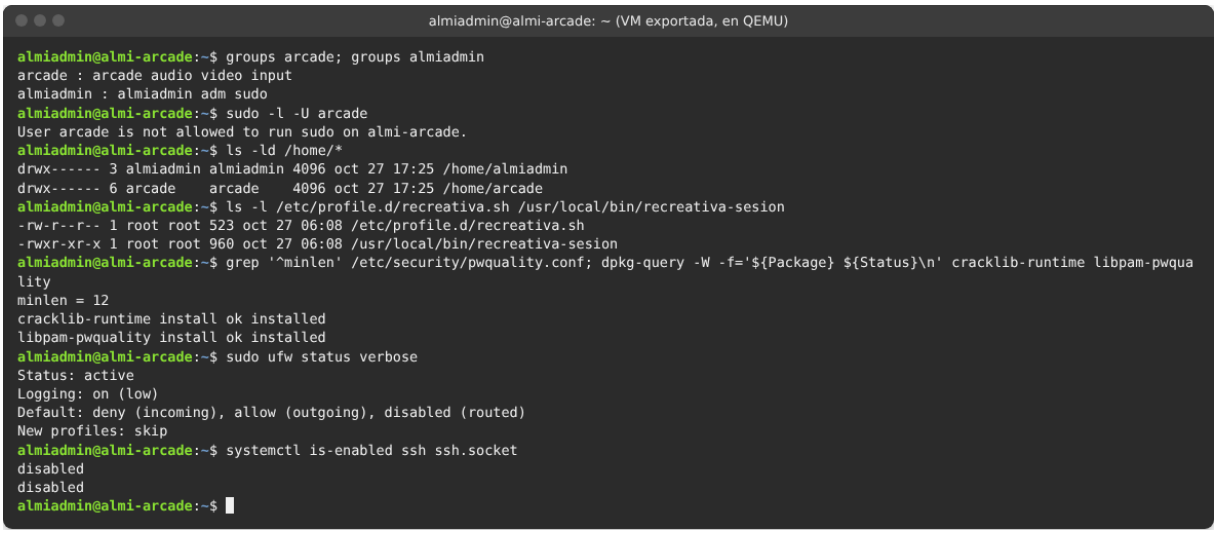


Figura 5. arcade sin sudo, carpetas privadas, arranque en archivos de root fuera de /home/arcade, contraseñas de 12 caracteres con diccionario, cortafuegos y sin SSH

4. Servidor gráfico mínimo y MAME (A05)

```
sudo apt install -y --no-install-recommends xserver-xorg-core xserver-xorg-input-libinput \
  xserver-xorg-video-fbdev xserver-xorg-video-vesa xinit x11-xserver-utils alsa-utils mame
mame -version
sudo mkdir -p /usr/local/share/games/mame/roms
```

Juegos: *Pong* y *Breakout* funcionan **sin ROMs** (MAME los emula a partir del esquema del circuito original). Para más juegos se descargan ROMs legales de mamedev.org (robby, gridlee, supertnk) a `/usr/local/share/games/mame/roms/` y se comprueban con `mame -verifyroms` (instrucciones en `02-linux-recreativa/config/LEEME-roms.txt`, que también se deja en esa carpeta de la VM). Las ROMs no se suben al repositorio.

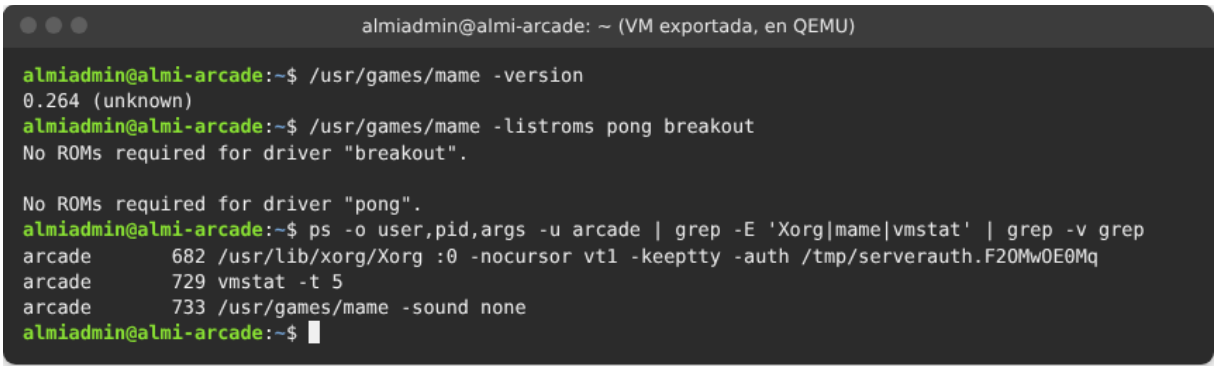


Figura 6. MAME 0.264, Pong y Breakout sin ROMs, y X y MAME corriendo como arcade

5. Arranque directo a MAME (A06)

Archivo	Dónde	Qué hace
`02-linux-recreativa/config/autologin.conf`	`/etc/systemd/system/getty@tty1.service.d/override.conf`	`arcade` entra solo en la consola 1
`02-linux-recreativa/config/recreativa-profile.sh`	`/etc/profile.d/recreativa.sh` (de root, 644)	Cuando `arcade` entra en tty1, arranca X sin cursor con la sesión de la recreativa
`02-linux-recreativa/config/recreativa-sesion`	`/usr/local/bin/recreativa-sesion` (de root, 755)	Desactiva el salvapantallas, guarda `vmstat` cada 5 s y abre MAME (sin

Archivo	Dónde	Qué hace
		sonido si la máquina no tiene tarjeta de sonido)
`02-linux-recreativa/config/ui.ini`	`/home/arcade/.mame/ui.ini`	El menú de MAME se abre con Pong seleccionado
`02-linux-recreativa/config/mame.ini`	`/home/arcade/.mame/mame.ini`	Configuración optimizada (apartado 6). Tiene que estar en ASCII y sin BOM

Desde la carpeta del repositorio:

```
sudo mkdir -p /etc/systemd/system/getty@tty1.service.d
sudo install -m 644 config/autologin.conf
/etc/systemd/system/getty@tty1.service.d/override.conf
sudo install -o root -g root -m 644 config/recreativa-profile.sh
/etc/profile.d/recreativa.sh
sudo install -o root -g root -m 755 config/recreativa-sesion /usr/local/bin/recreativa-
sesion
sudo install -d -o arcade -g arcade /home/arcade/.mame /home/arcade/rendimiento
sudo install -o arcade -g arcade -m 644 config/mame.ini config/ui.ini /home/arcade/.mame/
sudo systemctl daemon-reload && sudo reboot
```

Para administrar con MAME abierto: **Ctrl + Alt + F2** y entrar como `almiadmin`.

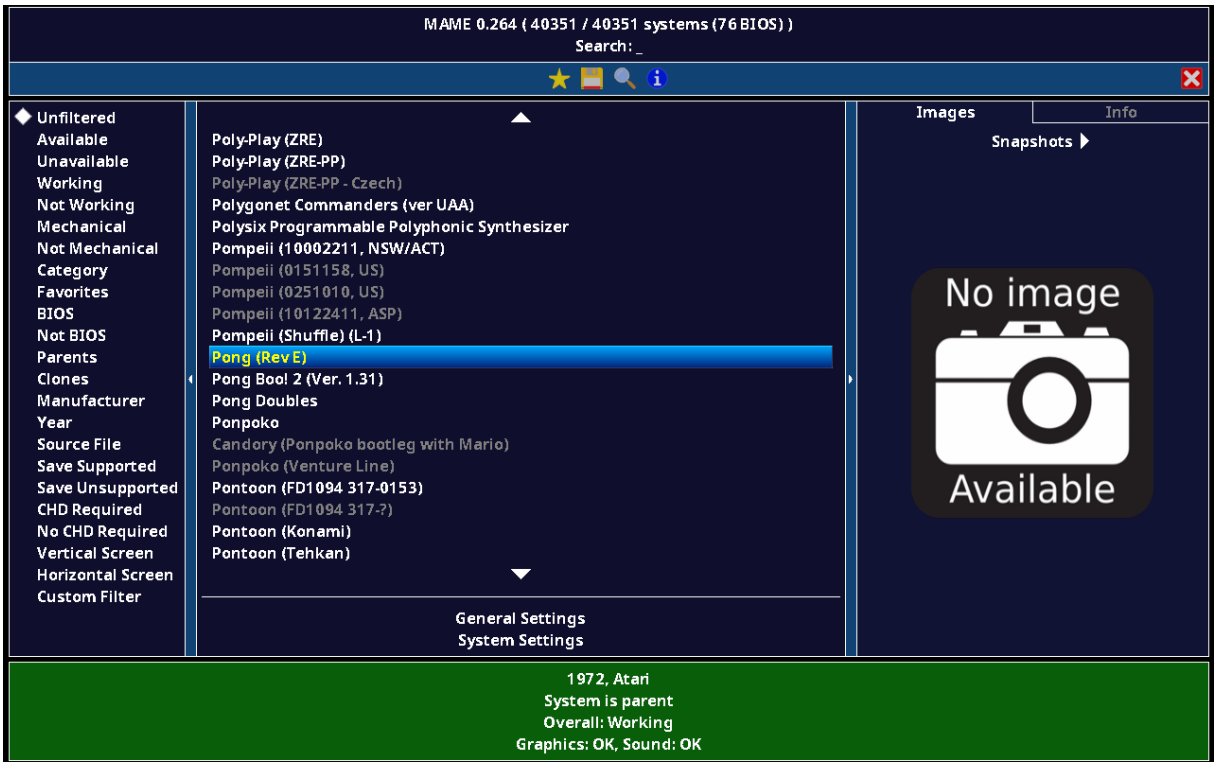


Figura 7. Al encender (arrancando con GRUB, como en VirtualBox) arcade entra solo y se abre MAME a pantalla completa con Pong seleccionado

6. Optimización (A08)

Cambio	Para qué
Sin swap y `mem=1024M nr_cpus=1`	Lo pide el reto: no usar el disco como memoria
`journald` en memoria, máximo 20 MB	No escribir en disco mientras se juega
Solo 2 consolas de texto (`NAutoVTs=2`)	Menos procesos

Cambio	Para qué
Desactivados `apt-daily`, `unattended-upgrades`, `man-db`, `e2scrub`, `fstrim`, `motd-news` y SSH	Nada trabajando en segundo plano durante la partida. Las actualizaciones las hace `almiadmin` a mano una vez por semana
Sin `systemd-networkd-wait-online`	Arranque más rápido
`02-linux-recreativa/config/mame.ini` del usuario arcade	`video soft`, `waitvsync 0`, `autoframeskip 1`, `samplerate 22050`, `skip_gameinfo 1`, `joystick 1`

Comandos:

```
sudo mkdir -p /etc/systemd/journald.conf.d
printf '[Journal]\nStorage=volatile\nRuntimeMaxUse=20M\n' | sudo tee
/etc/systemd/journald.conf.d/arcade.conf
sudo sed -i 's/^#NAutoVTs=.*/NAutoVTs=2/' /etc/systemd/logind.conf
for u in apt-daily.timer apt-daily-upgrade.timer man-db.timer e2scrub_all.timer fstrim.timer
motd-news.timer unattended-upgrades.service; do
  sudo systemctl disable --now "$u"
done
sudo systemctl disable --now ssh 2>/dev/null   # si se instaló
sudo systemctl mask systemd-networkd-wait-online.service
```

Resultado en la VM exportada, probada en QEMU: 11 servicios en marcha en la captura, de los que 3 no estarían en uso normal: `serial-getty@ttyS0` (la consola serie que se usó para la prueba; la VM exportada ya no la activa) y `user@1000`/`user@1001` (las sesiones abiertas para la captura). 2 temporizadores, el registro en memoria y SSH y las actualizaciones automáticas desactivados.

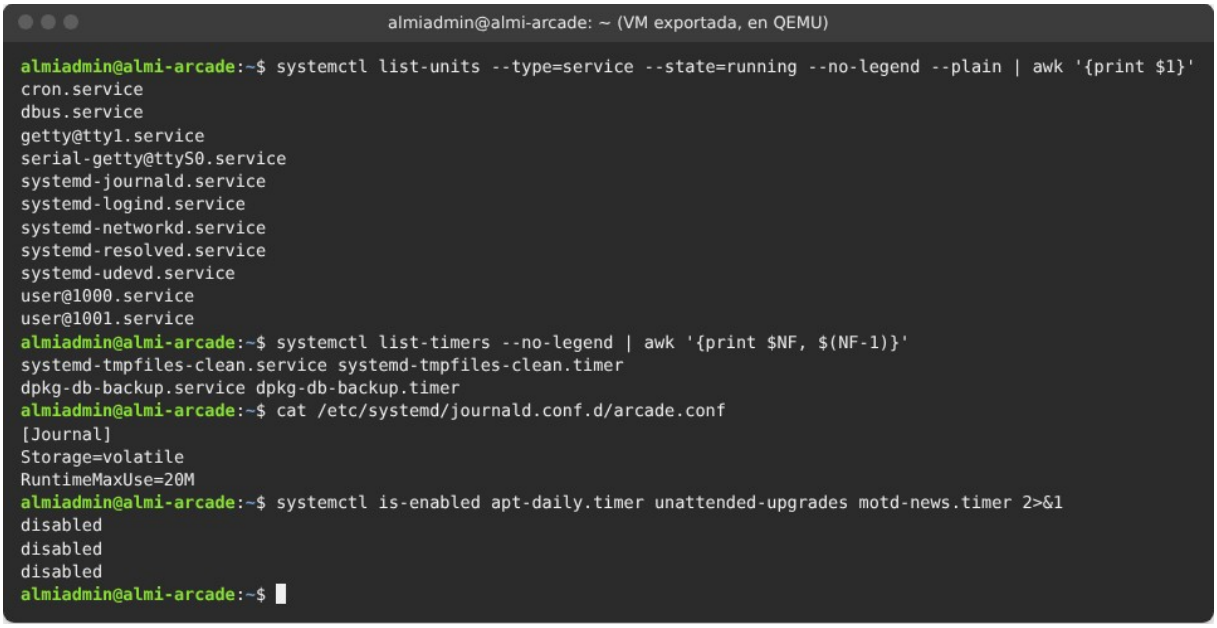


Figura 8. Servicios y temporizadores tras optimizar

7. Benchmark y monitorización (A07, A09)

`02-linux-recreativa/scripts/benchmark.sh` (instalado como `benchmark-recreativa`; se lanza con `sudo benchmark-recreativa`) ejecuta `mame -bench 60` tres veces por juego como el usuario `arcade` y da la media. 100 % es la velocidad real de la máquina original; más de 100 % es que sobra potencia.

> **Dónde se midió.** Las cifras de esta tabla se midieron con MAME 0.264 **en el ordenador donde se preparó el proyecto, no dentro de la VM**, limitando MAME a 1 núcleo con `taskset -c 0`. Sirven para comparar antes y después del `mame.ini`, pero no son la velocidad de la recreativa en VirtualBox. La medida dentro de ALMI-ARCADE en el PC de

clase está **pendiente** (prueba 8). Dentro de QEMU sin aceleración no tiene sentido medir: todo el procesador está emulado.

Resultados con MAME 0.264 limitado a **1 núcleo** (en el anfitrión), con la configuración por defecto (antes) y con el `mame.ini` optimizado (después):

Medida	Antes de optimizar	Después de optimizar
Pong (`-bench 60`, media de 3)	297,3 % (307,95 · 298,59 · 285,23)	315,0 % (311,16 · 317,15 · 316,58)
Breakout (`-bench 60`, media de 3)	97,6 % (93,54 · 88,63 · 110,77)	114,3 % (114,05 · 114,59 · 109,14)
Memoria total visible	961 MB	961 MB
Swap	0	0

Breakout es el juego más exigente (simula el circuito discreto componente a componente) y es el que más nota la optimización: pasa de ir justo (por debajo de 100 % en alguna pasada) a ir sobrado en todas.

El script se probó también **dentro de la VM exportada, en QEMU sin aceleración**, para comprobar que funciona donde se instala (antes fallaba: `sudo` no encontraba `mame`, y con el sistema en español la media salía mal). Las cifras de esa prueba (en torno al 20 %) **no miden la recreativa**: en QEMU sin aceleración todo el procesador se emula por software.

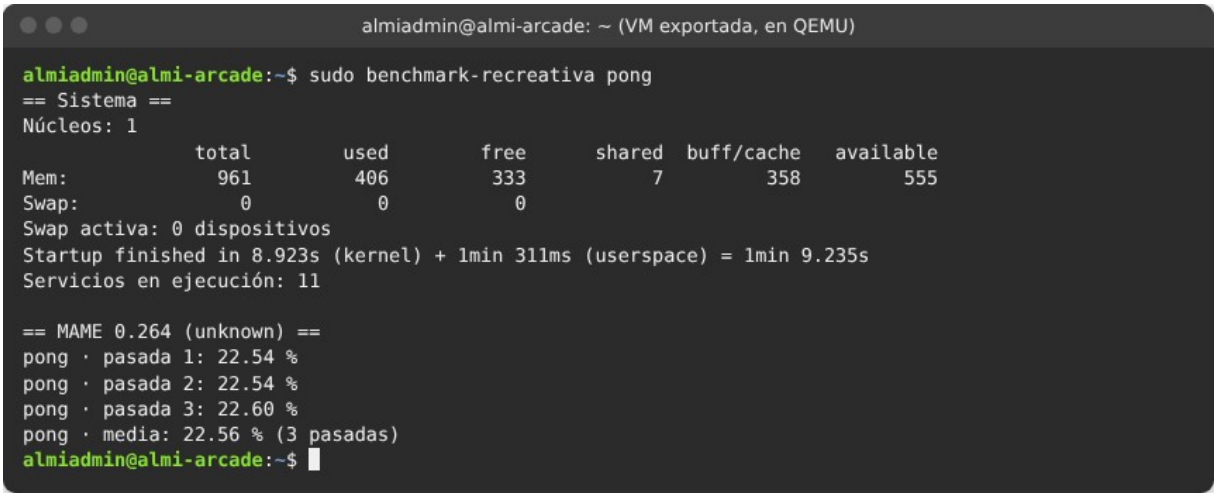


Figura 9. benchmark-recreativa ejecutado dentro de la VM (en QEMU): funciona y calcula la media; las cifras no sirven como medida

La sesión (`recreativa-sesion`) guarda un registro de `vmstat` en `/home/arcade/rendimiento/` mientras se juega. Las columnas `si` y `so` (memoria que entra y sale del swap) tienen que estar siempre a 0.

En la prueba, con MAME abierto unos 5 minutos (62 muestras): **`swpd`, `si` y `so` siempre a 0**. La CPU sale al 100 % porque esta prueba se hizo arrancando el disco de la VM en **QEMU sin aceleración por hardware** (todo el procesador emulado), no en VirtualBox; con VT-x el uso tiene que ser mucho menor (pendiente de medir en clase). Las cifras de velocidad buenas son las del benchmark de arriba.

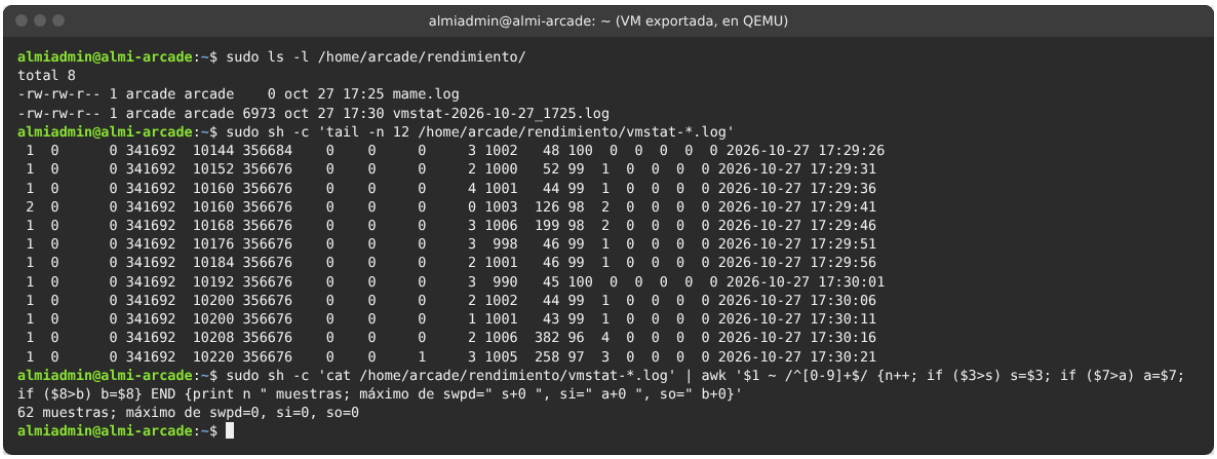


Figura 10. Registro de vmstat con MAME abierto: swap siempre a 0

Seguridad aplicada

- `arcade` sin sudo, contraseña de 12 caracteres, carpeta privada. Lo que se arranca al entrar está **fuera de su carpeta** y es de root (`/etc/profile.d/recreativa.sh` y `/usr/local/bin/recreativa-sesion`): dentro de `/home/arcade` no serviría, porque el dueño de una carpeta puede borrar o renombrar cualquier archivo que haya en ella, aunque sea de root.
- Cortafuegos con toda la entrada bloqueada y **sin servidor SSH**.
- Las ROMs, solo lectura para `arcade` en una carpeta del sistema.

Pruebas

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
1	1 núcleo	`nproc`	1	1	02 (`02-linux-recreativa/capturas/02-nucleo-memoria-swap.png`)
2	1 GB de RAM	`free -m`	Total menor de 1024 MB	961 MB	02 (`02-linux-recreativa/capturas/02-nucleo-memoria-swap.png`)
3	Sin swap	`swapon --show`	Sin salida	Sin salida; Swap 0	02 (`02-linux-recreativa/capturas/02-nucleo-memoria-swap.png`)
4	No se usa swap al jugar	Registro de vmstat	`si` y `so` siempre 0	`swpd`, `si` y `so` a 0 en las 62 muestras	06 (`02-linux-recreativa/capturas/06-vmstat.png`)
5	Arranque directo a MAME	Encender	MAME sin pedir contraseña	Autologin de arcade y MAME abierto	01 (`02-linux-recreativa/capturas/01-arranque-`

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
					`mame.png`) 04 (`02-linux-recreativa/capturas/04-mame-version-juegos.png`)
6	arcade sin privilegios	`groups arcade`; `sudo -l` como arcade	Sin sudo	No está en sudo	03 (`02-linux-recreativa/capturas/03-usuarios-seguridad.png`)
7	Juegos disponibles	`mame -listroms pong breakout`	No necesitan ROMs	“No ROMs required”	04 (`02-linux-recreativa/capturas/04-mame-version-juegos.png`)
8	Mejora de rendimiento	Tabla del benchmark	Igual o mayor velocidad después	En el anfitrión con 1 núcleo: Pong +6 %, Breakout +17 %. **Dentro de la VM en VirtualBox: pendiente — en clase**	—
9	Stick en MAME	Ver bloque 03	Se juega con el stick	Pendiente — en clase (hace falta el Arduino)	

Las pruebas 1 a 7 se hicieron arrancando el disco de `ALMI-ARCADE.ova` (el mismo que se entrega) en **QEMU** con `herramientas/probar-vm/pruebas-arcade.sh` (`herramientas/probar-vm/pruebas-arcade.sh`) (no en VirtualBox, que no estaba disponible donde se preparó el repositorio).

Problemas encontrados y solución

Problema	Causa	Solución
MAME se quedaba colgado al arrancar, con la CPU al 100 % y sin abrir ventana	El `mame.ini` tenía comentarios con tildes delante de la marca BOM que pone `mame -createconfig`, y el lector de MAME entra en un bucle	Comentarios del `mame.ini` solo en ASCII y sin BOM
MAME se cerraba al momento con *Segmentation fault* y la sesión se reiniciaba en bucle	MAME 0.264 falla al abrir el sonido si la VM no tiene tarjeta de sonido	La sesión comprueba `/proc/asound/cards` y, si no hay tarjeta, arranca MAME con `-sound none`. En VirtualBox conviene activar el audio (Intel HD Audio)
X arrancaba solo la primera vez (“drmSetMaster failed: Permission denied”)	Faltaba `libpam-systemd`: la sesión de arcade no se registraba en logind y X sin root no podía usar la pantalla	Instalar `libpam-systemd` (viene en una instalación normal de Ubuntu)

Problema	Causa	Solución
X no arranca con “Failed to load module fbdev/vesa”	Sin aceleración gráfica disponible, X no tenía controlador de respaldo	Instalar `xserver-xorg-video-fbdev` y `xserver-xorg-video-vesa`; en VirtualBox usar VMSVGA
No había ROMs legales descargables desde clase en el primer momento		Pong y Breakout no necesitan ROMs; el resto se añade desde mamedev.org

Fuentes

- Documentación de MAME: opciones de línea de órdenes y `-bench` — https://docs.mamedev.org/
- ROMs gratuitas para uso no comercial — https://www.mamedev.org/roms/
- `man systemd-journald.conf`, `man agetty`, `man vmstat`
- Guías paso a paso del equipo, parte 3.

3. Arcade stick con Arduino

Objetivo

Construir el mando de la recreativa: un **joystick de 8 direcciones y 9 botones** (6 de juego, Moneda, Start y Menú) conectados a una placa **Arduino** que el ordenador reconoce como **mando USB** o como **teclado USB**, y configurarlo en MAME dentro de la VM ALMI-ARCADE.

Tareas del tablero: S01 a S09.

Requisitos

Elemento	Detalle
Placa	La que da el profesor (no es un Uno). Ver la tabla de caminos
Joystick	Arcade de 8 direcciones con 4 microinterruptores
Botones	6 de 30 mm (juego) + 3 de 24 mm (Moneda, Start, Menú), con microinterruptor
Cables	Faston de 2,8 mm y cable USB de la placa
Software	Arduino IDE 2 (AppImage) en Zorin; librería ArduinoJoystickLibrary (camino A) o Keyboard (camino B)

Qué camino seguir según la placa

Placa	Chip	¿Mando o teclado USB?	Camino
Arduino Leonardo, Micro, Pro Micro y clones	ATmega32U4	Sí, de serie	**A** (mando) o **B** (teclado)
Arduino Uno R4 Minima / WiFi	Renesas RA4M1	Sí, como teclado	**B** (teclado)
Uno R3, Mega, Nano	ATmega328P / 2560	No de serie	**C** (plan B)

Para saber cuál es: nombre impreso en la placa y `lsusb` al conectarla.

Arquitectura y cableado

Cada microinterruptor tiene **COM** (común), **NO** (normalmente abierto) y **NC** (no se usa). Todos los COM en cadena a **GND**; cada NO a su pin. No hacen falta resistencias: se usan las internas de la placa (`INPUT_PULLUP`, pulsado = LOW).

Función	Pin	Camino A (mando)	Camino B (tecla)	En MAME
Arriba	2	Eje Y = -1	Flecha arriba	Mover arriba
Abajo	3	Eje Y = +1	Flecha abajo	Mover abajo
Izquierda	4	Eje X = -1	Flecha izquierda	Mover izquierda
Derecha	5	Eje X = +1	Flecha derecha	Mover derecha

Función	Pin	Camino A (mando)	Camino B (tecla)	En MAME
Botón 1	6	Botón 1	Ctrl izquierdo	Botón 1
Botón 2	7	Botón 2	Alt izquierdo	Botón 2
Botón 3	8	Botón 3	Espacio	Botón 3
Botón 4	9	Botón 4	Shift izquierdo	Botón 4
Botón 5	10	Botón 5	Z	Botón 5
Botón 6	A0	Botón 6	X	Botón 6
Moneda	A1	Botón 7	5	Meter moneda
Start	A2	Botón 8	1	Empezar (jugador 1)
Menú	A3	Botón 9	Tab	Menú de MAME

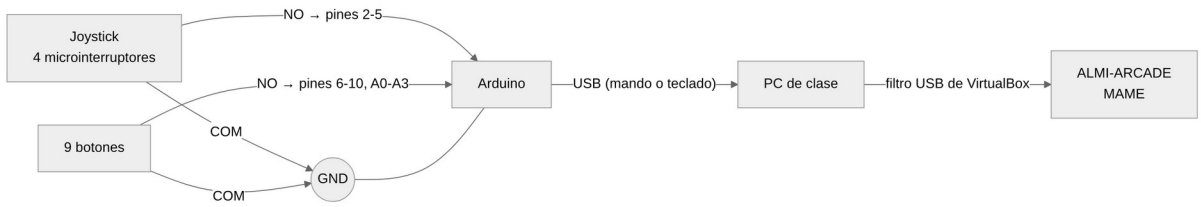


Figura 11. Diagrama

El joystick empuja el microinterruptor del lado contrario: si al mover arriba sale “Abajo”, se intercambian esos dos cables.

Programas

Programa	Para qué
`03-arcade-stick/sketches/prueba_cableado/prueba_cableado.ino`	Escribe en el Monitor Serie (9600) qué se pulsa. Sirve para comprobar los 13 cables antes de nada
`03-arcade-stick/sketches/arcade_stick_mando/arcade_stick_mando.ino`	**Camino A**: mando USB con ejes X/Y y 9 botones. Solo envía cuando algo cambia
`03-arcade-stick/sketches/arcade_stick_teclado/arcade_stick_teclado.ino`	**Camino B**: teclado USB con las teclas por defecto de MAME. **Modo seguro**: si se conecta con Menú pulsado no envía teclas, para poder reprogramar la placa

Los tres programas compilan sin errores para Arduino Leonardo (ATmega32U4) con la librería ArduinoJoystickLibrary (camino A) y la librería Keyboard (camino B).

Instalación paso a paso

1. **Arduino IDE 2** en Zorin: AppImage de arduino.cc, `sudo apt install -y libfuse2`, `chmod +x`, `sudo usermod -aG dialout $USER` y volver a iniciar sesión.
2. **Tools → Board**: Leonardo, Micro (o Leonardo para un Pro Micro de 5 V) o Uno R4. **Tools → Port**: `/dev/ttyACM0`. Probar con *Blink*.
3. Cablear según la tabla con la placa desconectada.
4. Subir `prueba_cableado` y comprobar las 13 entradas en el Monitor Serie.

5. Camino A: **Sketch → Include Library → Add .ZIP Library** con ArduinoJoystickLibrary (github.com/MHeironimus/ArduinoJoystickLibrary) y subir `arcade_stick_mando`. Camino B: subir `arcade_stick_teclado`.
6. Probar en Zorin: `sudo apt install -y joystick evtest`; camino A con `jstest /dev/input/js0`, camino B con `sudo evtest`. En Windows, `joy.cpl`.
7. **Pasar el stick a la recreativa**: VM ALMI-ARCADE apagada → **Configuración → USB → +** → elegir el Arduino. Al arrancar, la placa pasa a la VM.
8. **Controles en MAME**: con el juego abierto, **Tab** → *Input (this Machine)* o *Input (general)* → elegir cada acción y pulsar el botón del stick. MAME lo guarda en `~/.mame/cfg/`. En el camino B no hace falta: ya usa las teclas de MAME.

Camino C (plan B) si la placa es un Uno R3, Mega o Nano

Esas placas no pueden ser mando USB de serie. Opciones: (1) pedir un Pro Micro (unos 6-8 €) o (2) mantener el programa de prueba que escribe por el puerto serie y un pequeño script en la VM que convierta lo que llega en pulsaciones de teclado (`python3-evdev` con `uinput`). Se documenta cuál se usa.

Pruebas

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
1	Cableado	Monitor Serie con `prueba_cableado`	Sale el nombre correcto de las 13 entradas	Pendiente — en clase	
2	Reconocido como mando/teclado	`lsusb` y `jstest` / `evtest`	Aparece el dispositivo y responde	Pendiente — en clase	
3	Las 4 direcciones	`jstest` (ejes) o editor de texto (flechas)	Cada dirección mueve su eje o flecha	Pendiente — en clase	
4	Los 9 botones	`jstest` o `evtest`	Cada botón su número o tecla	Pendiente — en clase	
5	Funciona en la VM	Filtro USB y `lsusb` dentro de ALMI-ARCADE	Aparece en la VM	Pendiente — en clase	
6	Se juega en MAME	Pong o Breakout con el stick	Moneda, Start y movimiento funcionan	Pendiente — en clase	

Estado: los tres programas compilan, pero el stick **no se ha montado ni probado** todavía (falta el material y la placa). Las capturas y fotos del montaje irán en `03-arcade-stick/capturas`.

Seguridad aplicada

- El stick solo llega a la recreativa: en VirtualBox se pasa con un **filtro USB** a ALMI-ARCADE y no a las demás VM.
- **Modo seguro** del programa de teclado (camino B): si se conecta con *Menú* pulsado no envía ninguna tecla, para poder volver a programar la placa si un botón se queda pegado.
- El programa de teclado solo envía las 13 teclas de la tabla (no puede escribir texto). Ojo: Botón 1 y Botón 2 son Ctrl y Alt, así que pulsados a la vez con otra tecla forman combinaciones (es la

distribución por defecto de MAME). En la recreativa no hay escritorio, así que no tienen efecto fuera de MAME.

Problemas previstos y solución

El stick todavía no se ha montado, así que estos problemas son **previstos**, no encontrados:

Problema	Solución
Al mover arriba sale “Abajo” (o izquierda/derecha cambiadas)	El joystick empuja el microinterruptor del lado contrario: se intercambian esos dos cables
La placa es un Uno R3, Mega o Nano	No puede ser mando USB de serie: camino C
Un botón pegado o un programa que no deja de enviar teclas impide reprogramar la placa	Conectarla con *Menú* pulsado (modo seguro) o pulsar *reset* justo cuando el IDE empieza a subir el programa (en un Pro Micro, dos veces seguidas)

Fuentes

- ArduinoJoystickLibrary — https://github.com/MHeironimus/ArduinoJoystickLibrary
- Arduino Keyboard library — https://docs.arduino.cc/language-reference/en/functions/usb/Keyboard/
- MAME, configuración de controles — https://docs.mamedev.org/usingmame/defaultkeys.html
- Guías paso a paso del equipo, parte 4.

4. Red: WAN, plano, Packet Tracer y repetidor WDS

Objetivo

Diseñar la red del restaurante: elegir la línea **WAN**, hacer el **plan de direccionamiento con VLAN**, el **plano del local**, la red completa en **Packet Tracer** (switch, router, operador, servidores, TPV, Wi-Fi) con seguridad, y configurar los **dos routers físicos**: el principal seguro y el secundario como **repetidor WDS**.

Tareas del tablero: R01 a R10.

Necesidades de comunicación (R01)

Equipo	Cantidad	Conexión	VLAN	Necesita
Equipo del gerente	1	Cable	10 Gestión	Odoo, servidores, Internet
Servidor BD (ALMI-BD)	1	Cable	10 Gestión	Red interna
Servidor ERP (ALMI-ODOO)	1	Cable	10 Gestión	Red interna, Internet (correo)
TPV de barra	2	Cable	20 TPV	Solo Odoo e Internet
Impresora de tickets	2	USB al TPV (o cable)	20 TPV	Solo su TPV
Recreativa	1	Cable	30 Clientes	Nada (opcional Internet)
Wi-Fi de clientes	2 puntos (principal + repetidor)	Inalámbrico	30 Clientes	Solo Internet
Router, switch y puntos de acceso	—	—	99 Administración	Solo los administra almiadmin

Línea WAN (R02)

Criterios: fibra simétrica de 300 Mb/s o más, **IP fija**, soporte 24 h y respaldo móvil 4G/5G para que el TPV no se quede sin línea. La comparativa de 4 operadores (precio, enlace y fecha) irá en el presupuesto (`presupuesto/README.md`): **pendiente** (la fila está «por completar»).

Plan de direccionamiento (R03)

VLAN	Nombre	Red	Puerta de enlace	DHCP	Direcciones fijas
10	GESTION	192.168.10.0/24	192.168.10.1	.100 – .199	BD .20, Odoo .30
20	TPV	192.168.20.0/24	192.168.20.1	.100 – .199	Impresoras .50 y .51

VLAN	Nombre	Red	Puerta de enlace	DHCP	Direcciones fijas
30	CLIENTES	192.168.30.0/24	192.168.30.1	.100 – .250	—
99	ADMIN	192.168.99.0/24	192.168.99.1	Sin DHCP	Switch .2
998	SIN-USO	—	—	—	Puertos libres apagados
999	NATIVA	—	—	—	Nativa del troncal, sin equipos
WAN	Operador	203.0.113.0/30	203.0.113.1	—	Router del local .2

La VLAN 10 coincide con la Red NAT ALMICHAIN de VirtualBox (192.168.10.0/24): los servidores tienen la misma IP en el laboratorio y en el diseño.

Plano y esquema lógico (R04)

Plano del local: pendiente. Se hará en draw.io (`diagramas/red-plano.drawio` y `.png`) con la vista del local, el rack, las tomas T01…, los cables Cat 6 U/UTP T568B, el router Wi-Fi en la sala y el repetidor WDS en la terraza con su cobertura. Todavía **no está** en el repositorio.

Esquema lógico (VLAN, direcciones y reglas entre redes):

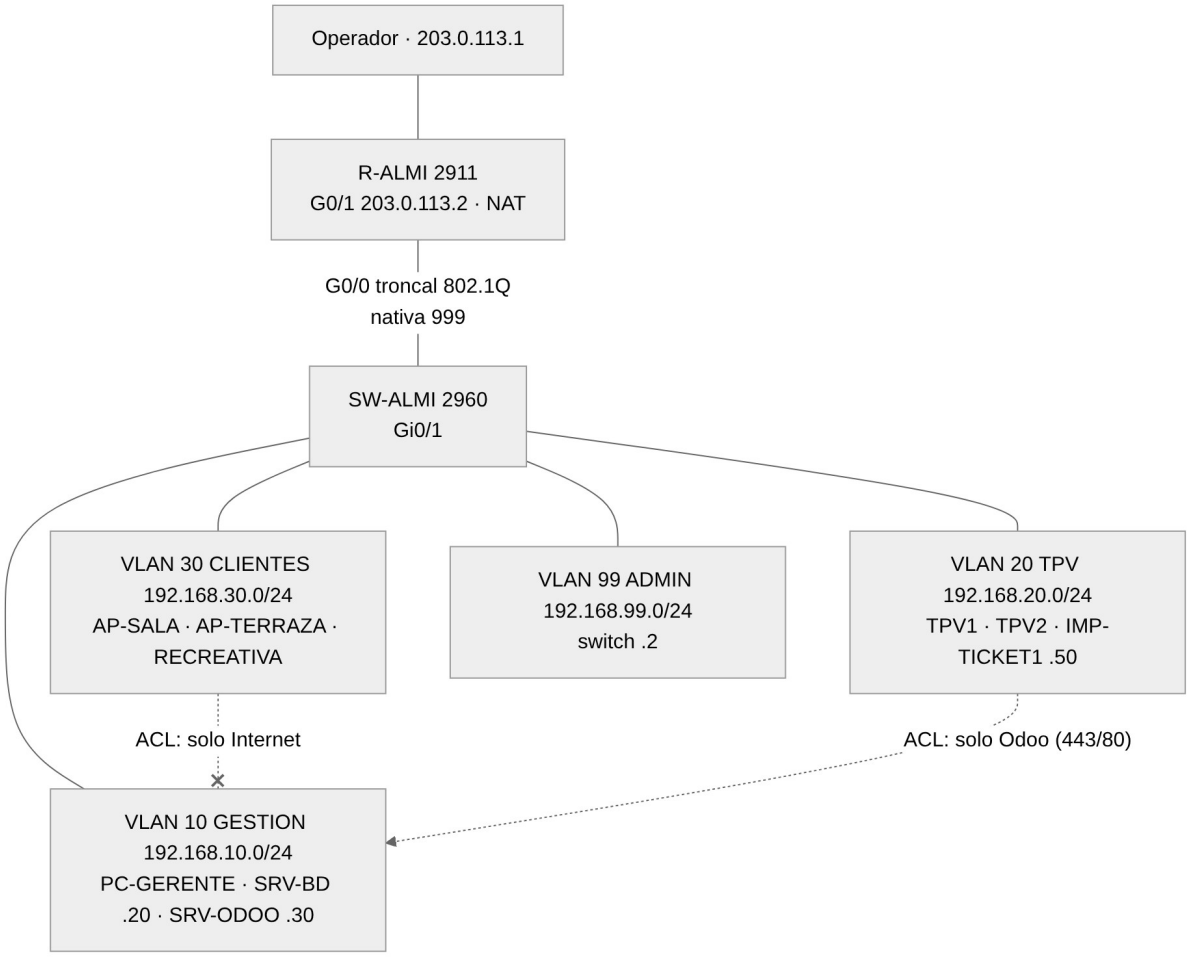


Figura 12. Diagrama

Packet Tracer (R05-R06)

Equipos: R-ALMI y ISP (Router 2911), SW-ALMI (2960-24TT), INTERNET (Server-PT), PC-GERENTE, SRV-BD, SRV-ODOO, TPV1, TPV2, IMP-TICKET1, AP-SALA, AP-TERRAZA, RECREATIVA y dos portátiles con WPC300N.

Puerto del switch	Equipo	VLAN	Seguridad
Fa0/1-3	Gerente, BD, Odoo	10	port-security, 1 MAC sticky, shutdown
Fa0/5-7	TPV1, TPV2, impresora	20	port-security, 1 MAC sticky, shutdown
Fa0/10-12	AP-SALA, AP-TERRAZA, recreativa	30	portfast
Gi0/1	R-ALMI	troncal 10,20,30,99	nativa 999
Resto	—	998	apagados

Configuraciones por línea de comandos, listas para pegar en la pestaña **CLI** (lo que se hace con el ratón está justo después):

Archivo	Equipo	Qué configura
`04-red/packet-tracer/SW-ALMI.txt`	Switch	VLAN, puertos de acceso con seguridad, troncal, puertos sin uso apagados, IP de gestión en VLAN 99, SSH v2 solo desde las VLAN 10 y 99
`04-red/packet-tracer/R-ALMI.txt`	Router	Subinterfaces 802.1Q, DHCP por VLAN (solo en el rango del plan), NAT con sobrecarga, ACL de clientes y de TPV, SSH v2 solo desde las VLAN 10 y 99 (`access-class`), bloqueo tras 3 intentos fallidos
`04-red/packet-tracer/ISP.txt`	Operador	Enlace WAN y red de “Internet” 198.51.100.0/24

Listas de acceso del router:

- **CLIENTES-SOLO-INTERNET** (entrada en G0/0.30): los clientes del Wi-Fi no llegan a Gestión, TPV ni Administración.
- **TPV-SOLO-ODOO** (entrada en G0/0.20): los TPV solo llegan a Odoo (192.168.10.30) por **443** (y 80, que redirige a HTTPS); el resto de Gestión y Administración, bloqueado.

Lo que se configura con el ratón (no tiene CLI en Packet Tracer)

Servidor INTERNET (simula una web de fuera): **Desktop** → **IP Configuration** → **Static**: IP 198.51.100.10, máscara 255.255.255.0, puerta 198.51.100.1, DNS 198.51.100.10. **Services** → **DNS**: activado, nombre `www.ejemplo.com` → 198.51.100.10. **Services** → **HTTP**: activado.

Equipos con IP fija (Desktop → IP Configuration → Static):

Equipo	IP	Máscara	Puerta de enlace	DNS
SRV-BD	192.168.10.20	255.255.255.0	192.168.10.1	198.51.100.10

Equipo	IP	Máscara	Puerta de enlace	DNS
SRV-ODOO	192.168.10.30	255.255.255.0	192.168.10.1	198.51.100.10
IMP-TICKET1	192.168.20.50	255.255.255.0	192.168.20.1	—

En SRV-ODOO se dejan activados **Services → HTTP** y **HTTPS** (simulan la web de Odoo). La ACL de los TPV deja pasar los dos puertos, así que la prueba 6 se puede hacer con `http://` y con `https://`.

Equipos con DHCP: PC-GERENTE, TPV1, TPV2 y RECREATIVA → **Desktop → IP Configuration → DHCP**. Tiene que salir una IP de su VLAN (por ejemplo 192.168.20.100 en TPV1).

Wi-Fi de clientes:

1. AP-SALA y AP-TERRAZA → **Config → Port 1**: SSID `AlmiChain-Clientes`, **WPA2-PSK**, clave de 12 caracteres o más, cifrado **AES** (los dos iguales).
2. CLIENTE1 y CLIENTE2 → **Physical**: apagar, quitar el módulo de cable, poner **WPC300N**, encender.
3. **Config → Wireless0**: mismo SSID, WPA2-PSK, la misma clave, IP por **DHCP**. Tiene que salir una IP 192.168.30.x.

El archivo de Packet Tracer se guardará como `packet-tracer/almichain.pkt` (indicando la versión de Packet Tracer). **Todavía no está en el repositorio**: la red no se ha montado aún.

Routers reales (R07-R08)

R1, principal: reset de fábrica, contraseña de administración nueva, firmware actualizado, SSID `AlmiChain-Clientes`, **canal fijo** (1, 6 u 11, el menos usado: `nmcli -f SSID,CHAN,SIGNAL dev wifi list`), ancho de 20 MHz, **WPA2-PSK AES**, **WPS desactivado**, **administración remota y UPnP desactivados**, aislamiento de clientes si lo tiene.

R2, repetidor WDS: reset, contraseña nueva, **IP de LAN en la red de R1 fuera de su rango DHCP** (p. ej. 192.168.0.2), **DHCP desactivado**, **mismo canal** que R1, **Habilitar puente WDS** → buscar `AlmiChain-Clientes` → WPA2-PSK con la clave de R1. En Estado tiene que poner WDS **Run/Conectado**. Si el modelo no tiene WDS, se usa el modo repetidor/extensor y se documenta.

Guía detallada: `04-red/routers/configuracion-routers.md`.

Seguridad aplicada

- **Segmentación por VLAN**: gestión, TPV, clientes y administración separadas; la VLAN nativa del troncal (999) y la de los puertos sin uso (998) no llevan tráfico.
- **ACL en el router**: los clientes del Wi-Fi solo salen a Internet; los TPV solo llegan a Odoo por 443 y 80.
- **Puertos del switch**: *port-security* con 1 MAC aprendida (*sticky*) y apagado si cambia; puertos sin uso apagados.
- **Administración**: solo por SSH v2 con usuario local, solo desde las VLAN 10 y 99 (`access-class`), bloqueo tras 3 intentos fallidos, cierre por inactividad y contraseñas cifradas (`enable secret`, `service password-encryption`).
- **Wi-Fi**: WPA2-PSK con AES y clave de 12 caracteres o más; WPS, administración remota y UPnP desactivados; aislamiento de clientes si el router lo permite.
- Contraseñas de los equipos de red en KeePassXC, nunca en los archivos del repositorio (`<contraseña>`).

Pruebas (R09-R10)

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
1	VLAN	`show vlan brief` en SW-ALMI	Puertos en 10, 20, 30 y 998	Pendiente — en clase	
2	Troncal	`show interfaces trunk`	Gi0/1, nativa 999, VLAN 10,20,30,99	Pendiente — en clase	
3	DHCP	`show ip dhcp binding` en R-ALMI	IPs a PCs y portátiles	Pendiente — en clase	
4	Cliente a Internet	CLIENTE1 → http://www.ejemplo.com	Abre la web	Pendiente — en clase	
5	Cliente aislado	CLIENTE1 `ping 192.168.10.30` y `192.168.20.100`	Fallan	Pendiente — en clase	
6	TPV solo a Odoo	TPV1 → http://192.168.10.30 y `ping 192.168.10.20`	Abre Odoo; el ping falla	Pendiente — en clase	
7	Gerente	`ping 192.168.10.20` y `198.51.100.10`	Responden	Pendiente — en clase	
8	NAT y ACL	`show ip nat translations` y `show access-lists`	Traducciones y coincidencias en los deny	Pendiente — en clase	
9	SSH	PC-GERENTE `ssh -l almiadmin 192.168.10.1`	Entra en R-ALMI	Pendiente — en clase	
9b	SSH bloqueado desde fuera de gestión	CLIENTE1 `ssh -l almiadmin 192.168.30.1` y TPV1 `ssh -l almiadmin 192.168.20.1`	Conexión rechazada	Pendiente — en clase	
10	Routers seguros	Contraseña de fábrica y WPS	No entra con la de la pegatina; WPS desactivado	Pendiente — en clase	
11	Cliente en R2 recibe IP de R1	`ip -4 addr` y `ip route`	Red de R1, puerta = R1	Pendiente — en clase	
12	Repetidor sin pérdidas	`ping -c 20` a R1 y a 8.8.8.8	0 % de pérdidas	Pendiente — en clase	

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
13	Velocidad y cobertura	`iperf3 -c` en 4 puntos	Por R2 ~la mitad (normal en WDS); en el punto más lejano R2 da más señal	Pendiente — en clase	

Estado: la red todavía no se ha montado en Packet Tracer ni con los routers físicos: las configuraciones se han revisado a mano, pero ninguna prueba está hecha. El `.pkt`, las capturas y los resultados se hacen en clase.

Problemas encontrados y previstos

La red todavía no se ha montado en Packet Tracer ni con los routers físicos. El primer problema se encontró al revisar la configuración de la guía; los otros dos son **previstos** (documentación de los fabricantes y de la guía), no observados.

Problema	Solución
(Encontrado al revisar) La ACL de la guía dejaba a los TPV solo el puerto 80, pero Odoo va por HTTPS	Se añade `permit tcp 192.168.20.0 0.0.0.255 host 192.168.10.30 eq 443` antes del `deny`
WDS no funciona con WPA3 en la mayoría de routers domésticos	WPA2-PSK con AES en los dos
Con canal automático el WDS se cae	Canal fijo e igual en R1 y R2

Fuentes

- Cisco Packet Tracer: configuración de VLAN, router-on-a-stick, DHCP, NAT y ACL extendidas.
- Manual del fabricante de los routers (modo WDS).
- Guías paso a paso del equipo, parte 5.

5. ERP Odoo + TPV

Objetivo

El restaurante necesita un ERP con facturación, clientes (CRM), personal, marketing, inventario y TPV. Se ha elegido **Odoo 18 Community**, instalado con **Docker** en la VM **ALMI-ODOO** y publicado solo por **HTTPS** con nginx. Los dos TPV de la barra son los puntos de venta **Barra 1** y **Barra 2** de Odoo. Cuando se cobra una venta a un cliente, Odoo **crea y publica la factura**, la **deja preparada para enviarla por correo** con el PDF y **descuenta el stock**, sin que nadie toque nada. El envío real por Gmail está pendiente: falta poner la contraseña de aplicación de la cuenta del equipo (apartado 6, prueba 5).

Tareas del tablero: E01 a E13.

Requisitos

Elemento	Valor
VM	ALMI-ODOO · 3 GB de RAM · 2 núcleos · disco de 25 GB · Red NAT ALMICHAIN
Sistema	Ubuntu Server 24.04 LTS
IP	192.168.10.30/24 · puerta de enlace 192.168.10.1 · DNS 1.1.1.1 y 9.9.9.9
Software	Docker y Docker Compose (paquetes de Ubuntu), imágenes `odoo:18.0` y `postgres:16`, nginx 1.24
Usuario de administración	`almiadmin` (contraseña en KeePassXC)
Licencias	Odoo Community LGPL, Ubuntu, Docker, nginx y PostgreSQL: todo libre y gratis

Por qué Odoo (E01)

ERP	Licencia	TPV integrado	Contabilidad española	Instalación con Docker
Odoo Community	LGPL, gratis	Sí (Punto de venta, modo bar/restaurante)	Sí (plan contable español)	Imagen oficial
Dolibarr	GPL, gratis	Sí (TakePOS)	Sí	Imágenes de la comunidad
ERPNext	GPL, gratis	Sí (POS)	Parcial	Oficial, pero más pesada

Odoo tiene en un solo programa todo lo que pide el enunciado (facturas, CRM, personal, marketing, inventario y TPV), permite automatizar sin programar un módulo (reglas de automatización) y su imagen de Docker hace la instalación repetible. Además es el ERP que indicó el profesor.

Arquitectura

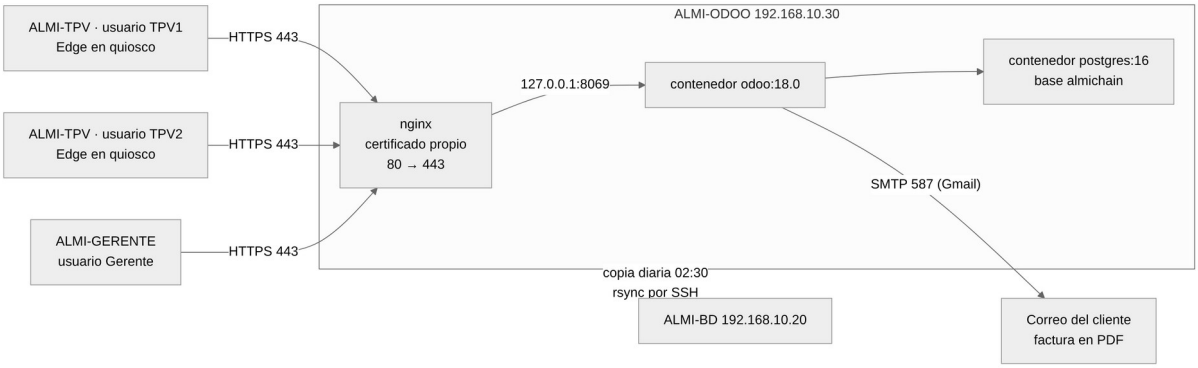


Figura 13. Diagrama

- Odoo solo escucha en `127.0.0.1:8069`: desde la red **solo se entra por nginx con HTTPS**.
- La contraseña de PostgreSQL está en `/opt/odoo/.env` (solo la lee root) y no sale del servidor.
- Los datos están en volúmenes de Docker (`odoo-db` y `odoo-data`), así que se puede actualizar la imagen sin perder nada.

Instalación y configuración paso a paso

Los archivos que se usan están en `05-erp-odoo/config` y `05-erp-odoo/scripts`. Las contraseñas de los ejemplos (<…>) se sustituyen por las del gestor de contraseñas.

1. Ubuntu Server (E02)

1. VirtualBox → **Nueva** → ALMI-ODOO, ISO de Ubuntu Server 24.04, Linux / Ubuntu (64-bit), **Omitir instalación desatendida** (si no, VirtualBox crea el usuario `vboxuser`), 3072 MB, 2 CPU, 25 GB. Red: **Red NAT ALMICHAIN**.
2. Instalador: teclado español, red manual (192.168.10.30/24, puerta 192.168.10.1, DNS 1.1.1.1,9.9.9.9), disco entero, nombre `almi-odoo`, usuario `almiadmin`, **Install OpenSSH server**.
3. Al terminar: `sudo apt update && sudo apt full-upgrade -y`. El instalador usa LVM y solo da a la raíz la mitad del disco; para usarlo entero: `sudo lvextend -l +100%FREE -r /dev/ubuntu-vg/ubuntu-lv` (comprobar con `df -h /`).
4. Cortafuegos con las redes del diseño (plan de direccionamiento (`04-red/README.md`)): la administración por SSH solo desde la VLAN 10 (gestión, donde están los servidores y el gerente) y la 99 (administración); la web de Odoo desde la VLAN 10 y la VLAN 20 (TPV). La VLAN 30 de clientes no llega a nada:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
for red in 192.168.10.0/24 192.168.99.0/24; do sudo ufw allow from $red to any port 22 proto
tcp; done
for red in 192.168.10.0/24 192.168.20.0/24; do sudo ufw allow from $red to any port 80,443
proto tcp; done
sudo ufw enable
sudo ufw status verbose
```

> En el laboratorio todas las VM están en la misma Red NAT 192.168.10.0/24, así que el TPV entra por la regla de la VLAN 10. La regla de la VLAN 20 es la del restaurante real.

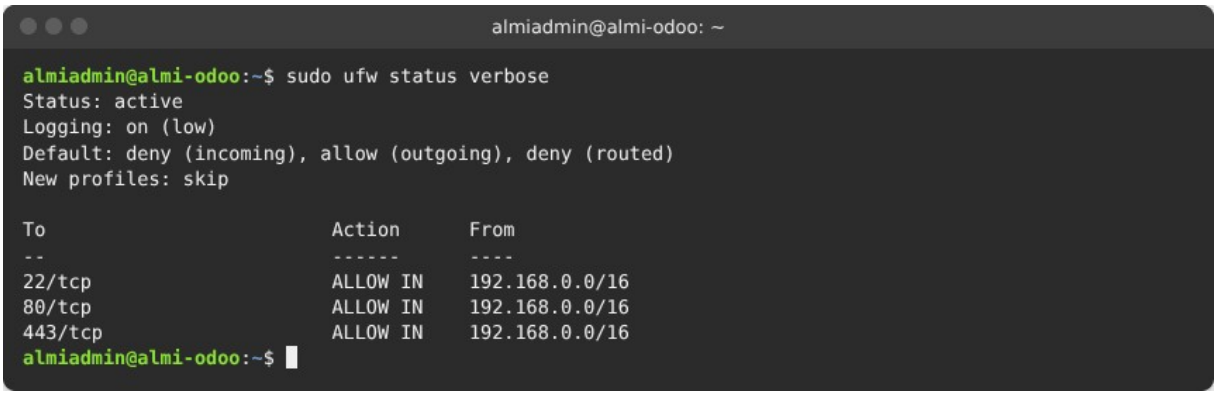


Figura 14. Cortafuegos en el servidor de pruebas con la regla anterior (192.168.0.0/16); la regla actual, por VLAN, está en la captura 38

2. Docker y Odoo (E03)

```
sudo apt install -y docker.io docker-compose-v2
sudo systemctl enable --now docker
sudo mkdir -p /opt/odoo/config
sudo chmod 700 /opt/odoo
echo "DB_PASSWORD=$(openssl rand -base64 24)" | sudo tee /opt/odoo/.env
sudo chmod 600 /opt/odoo/.env
```

El formato de ese archivo está en `05-erp-odoo/config/env.example` (sin la contraseña real).

`almiadmin` **no** se mete en el grupo `docker` (ese grupo equivale a ser root): siempre `sudo docker`.

Se copian `05-erp-odoo/config/docker-compose.yml` a `/opt/odoo/docker-compose.yml` y `05-erp-odoo/config/odoo.conf` a `/opt/odoo/config/odoo.conf`, **cambiando el texto entre `<` `>` de `admin_passwd` por la contraseña maestra** (`openssl rand -base64 24`, guardada en KeePassXC). Si se deja la plantilla tal cual, esa frase pública sería la contraseña. **La primera vez se comentan con `;` las líneas `list_db` y `dbfilter`**: si no, Odoo no enseña el formulario para crear la base (apartado 4). Y se arranca:

```
sudo chmod 644 /opt/odoo/config/odoo.conf
sudo docker compose --project-directory /opt/odoo up -d
```

> Como `/opt/odoo` tiene permisos 700, `almiadmin` no puede hacer `cd /opt/odoo`. Por eso todos los comandos usan `--project-directory /opt/odoo` con `sudo`.

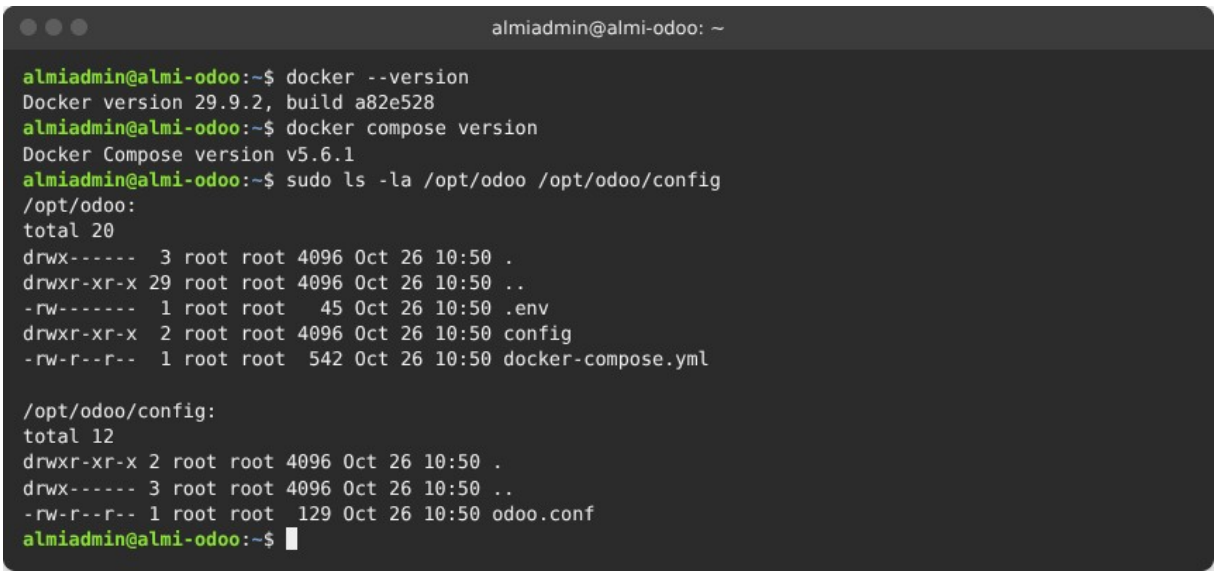


Figura 15. Versiones de Docker y permisos de /opt/odoo

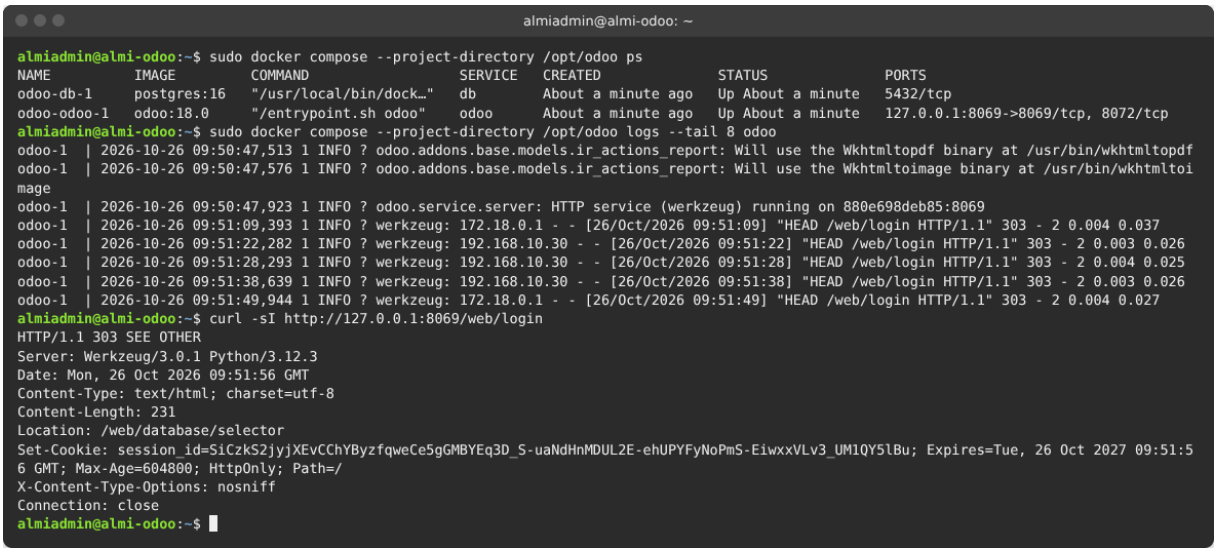


Figura 16. Los dos contenedores en marcha y Odoo respondiendo en 127.0.0.1:8069

3. HTTPS interno con nginx (E04)

Certificado propio para el servidor (válido 825 días) y nginx delante de Odoo. La configuración completa está en `05-erp-odoo/config/nginx-odoo.conf`: redirige el puerto 80 al 443, solo permite TLS 1.2 y 1.3 y pasa a Odoo las cabeceras `X-Forwarded-*` (por eso `odoo.conf` lleva `proxy_mode = True`).

El certificado es **de servidor y no de autoridad** (`CA:FALSE`, uso `serverAuth`). Es importante porque en los Windows se instala en el almacén de raíces de confianza: si fuera `CA:TRUE` (lo que hace `openssl req -x509` por defecto), quien tuviera su clave privada podría firmar certificados de **cualquier** web y los Windows del restaurante se los creerían. Con `CA:FALSE` solo sirve para este servidor.

```
sudo apt install -y nginx
sudo mkdir -p /etc/ssl/almichain
sudo openssl req -x509 -nodes -newkey rsa:2048 -days 825 \
  -keyout /etc/ssl/almichain/odoo.key -out /etc/ssl/almichain/odoo.crt \
  -subj "/C=ES/ST=Bizkaia/L=Bilbao/O=Almi-Chain/CN=192.168.10.30" \
  -addext "basicConstraints=critical,CA:FALSE" \
  -addext "keyUsage=critical,digitalSignature,keyEncipherment" \
  -addext "extendedKeyUsage=serverAuth" \
  -addext "subjectAltName=IP:192.168.10.30,DNS:almi-odoo,DNS:localhost,IP:127.0.0.1"
sudo chmod 600 /etc/ssl/almichain/odoo.key
sudo openssl x509 -in /etc/ssl/almichain/odoo.crt -noout -fingerprint -sha256   # se apunta: se comprueba en los Windows
sudo cp config/nginx-odoo.conf /etc/nginx/sites-available/odoo   # desde la carpeta 05-erp-odoo del repositorio
sudo ln -s /etc/nginx/sites-available/odoo /etc/nginx/sites-enabled/odoo
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

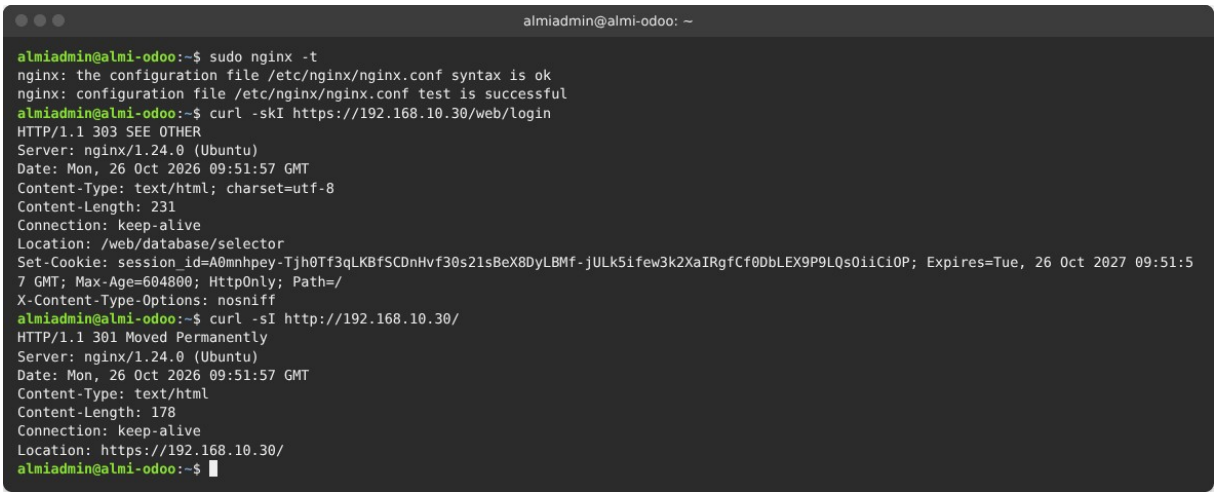


Figura 17. nginx correcto, HTTPS respondiendo y HTTP redirigiendo a HTTPS

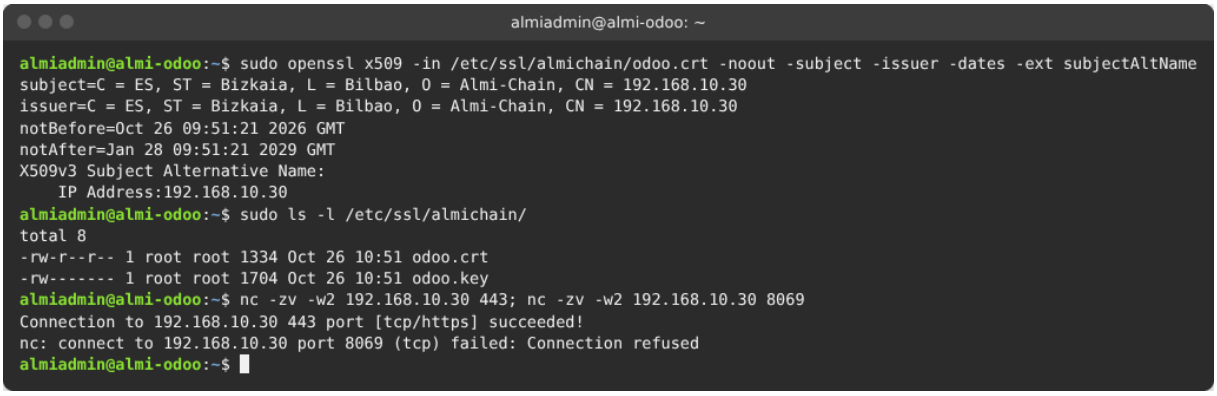


Figura 18. Certificado en el servidor de pruebas (era CA:TRUE, antes de corregirlo; el actual está en la captura 37), clave solo para root, 443 abierto y 8069 cerrado

nginx publica también la parte **pública** del certificado en `http://192.168.10.30/almichain-odoo.crt` (nunca la clave). En los dos Windows se descarga e instala como de confianza con `01-windows-gerente-tpv/scripts/3-certificado-odoo.ps1`, que **pide la huella SHA-256** apuntada en el servidor y no lo instala si no coincide. Así no hace falta entrar por SSH al servidor desde los TPV (la red no lo permite) y Edge muestra el candado sin avisos.

4. Base de datos `almichain` (E05)

En el navegador, `https://192.168.10.30` → formulario de creación: contraseña maestra, base `almichain`, correo del administrador, idioma **Spanish (ES)**, país **Spain**, sin datos de demostración.

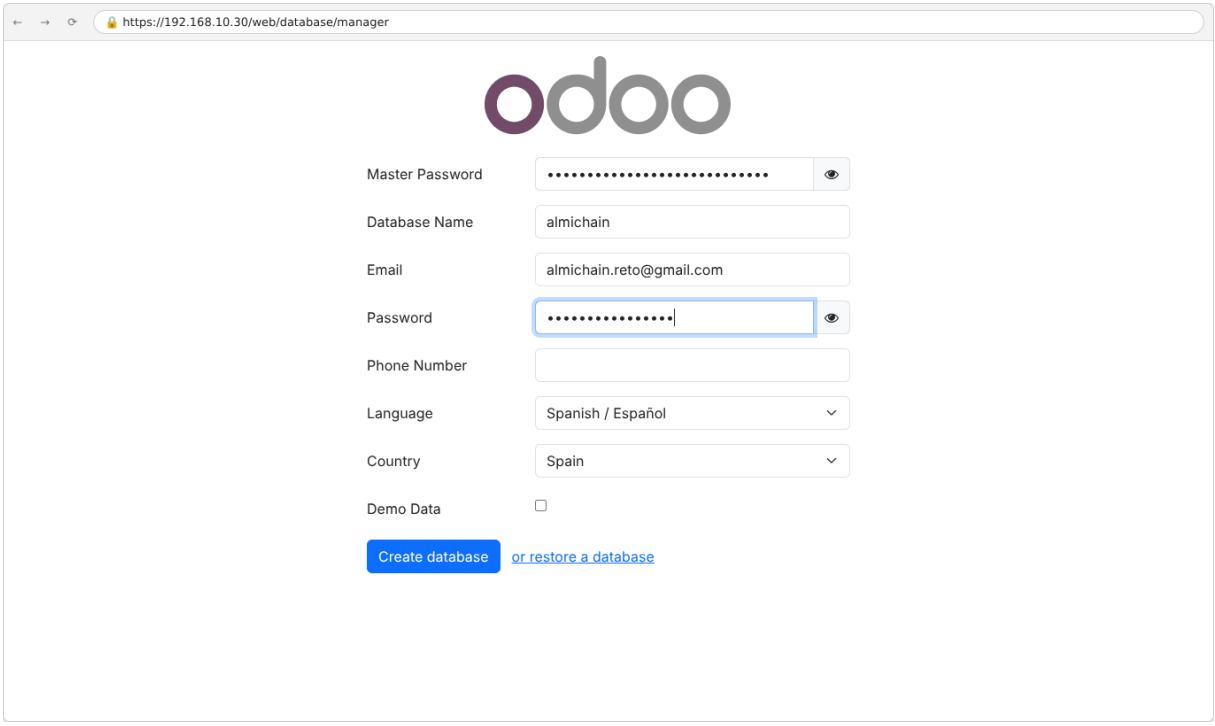


Figura 19. Formulario de creación de la base de datos almichain

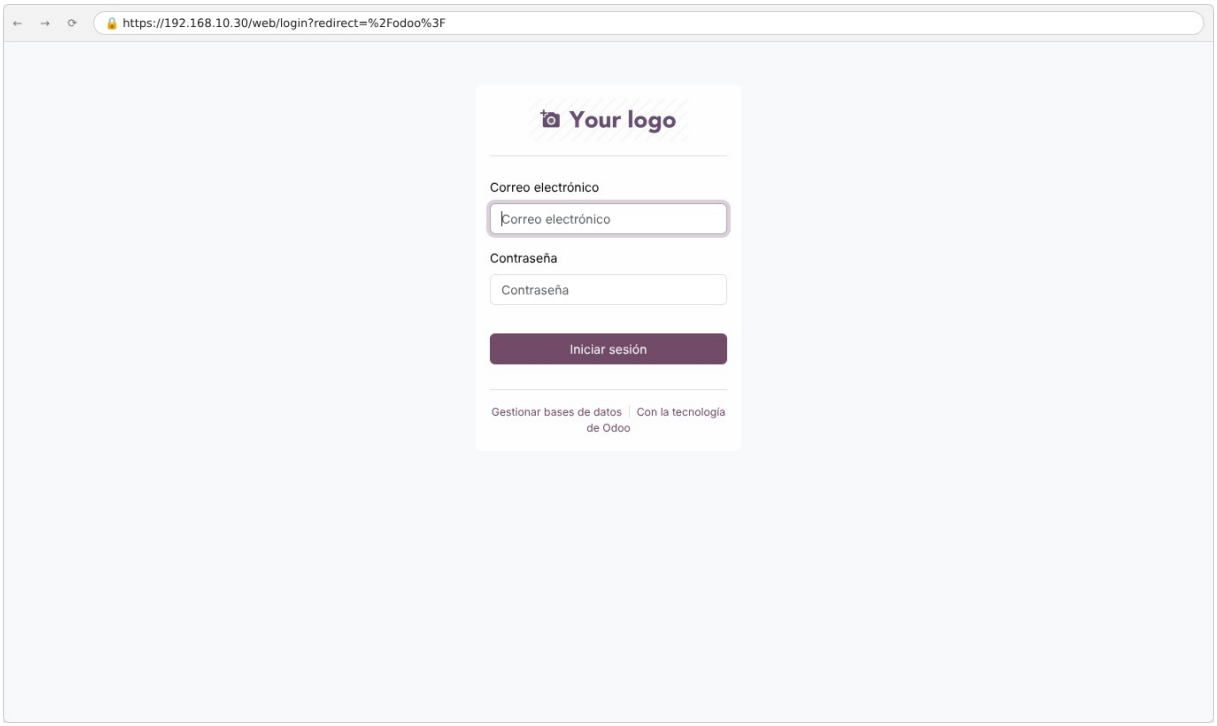


Figura 20. Base creada: pantalla de acceso de Odoo

Después se oculta el gestor de bases de datos para que nadie pueda borrarlas o copiarlas desde la web. Al final de `odoo.conf` se quitan los `;` de las dos líneas:

```
list_db = False
dbfilter = ^almichain$

sudo docker compose --project-directory /opt/odoo restart odoo
```

```
almiadmin@almi-odoo: ~
almiadmin@almi-odoo:~$ sudo grep -v admin_passwd /opt/odoo/config/odoo.conf
[options]
addons_path = /mnt/extra-addons
data_dir = /var/lib/odoo
proxy_mode = True
list_db = False
dbfilter = ^almichain$
almiadmin@almi-odoo:~$ curl -sk -o /dev/null -w '%{http_code}\n' https://192.168.10.30/web/database/manager
200
almiadmin@almi-odoo:~$ curl -sk https://192.168.10.30/web/database/manager | grep -o 'database manager has been disabled[^<]*'
database manager has been disabled by the administrator
almiadmin@almi-odoo:~$
```

Figura 21. El gestor de bases de datos queda desactivado

5. Aplicaciones, empresa, usuarios, productos, TPV y regla (E05 a E10)

Todo esto se configuró con el script `05-erp-odoo/scripts/configurar_odoo.py`, que hace por XML-RPC lo mismo que los pasos a mano de la guía (en la interfaz de Odoo) y sirve para rehacerlo en unos minutos si hay que reinstalar (las contraseñas se pasan como variables de entorno, no están en el script):

```
export ODOO_URL=https://192.168.10.30 ODOO_CA=/etc/ssl/almichain/odoo.crt
export ODOO_ADMIN_LOGIN=... ODOO_ADMIN_PASS=... ODOO_GERENTE_PASS=... ODOO_BARRA1_PASS=...
ODOO_BARRA2_PASS=...
python3 scripts/configurar_odoo.py
```

Aplicaciones: Facturación, Punto de venta, CRM, Empleados, Marketing por correo electrónico, Inventario y Reglas de automatización.

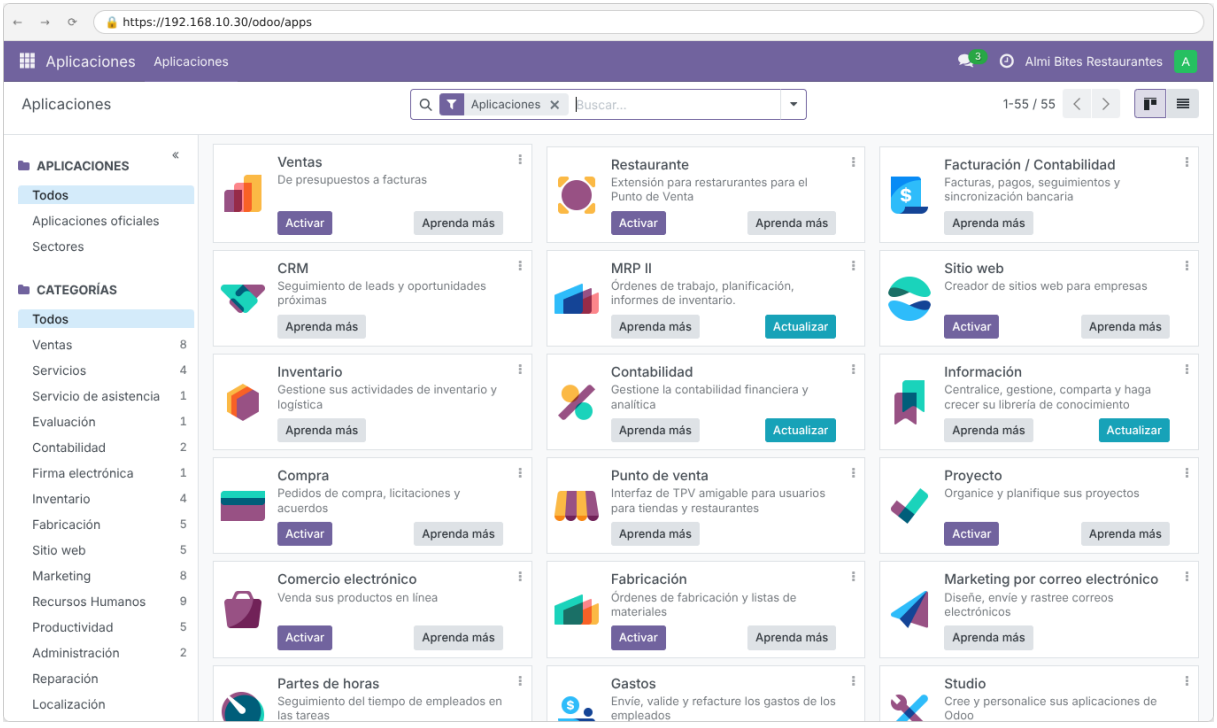


Figura 22. Aplicaciones instaladas

Empresa: Almi Bites Restaurantes, Bilbao, EUR, con el logotipo de la web.

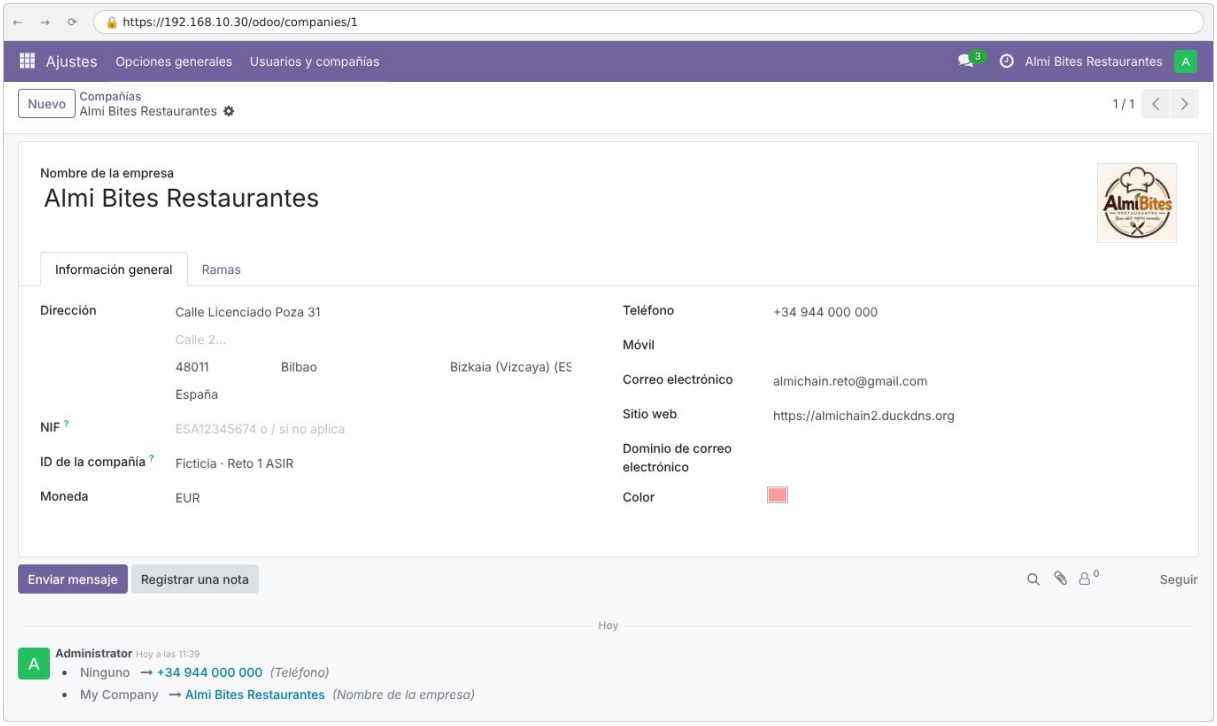


Figura 23. Datos de la empresa

Usuarios con mínimo privilegio (E06):

Usuario	Inicio de sesión	Permisos
Gerente	gerente@almichain.local	Facturación: Facturación · TPV: Administrador · CRM: Administrador · Empleados: Administrador · Inventario: Administrador · Marketing: Usuario · **sin Ajustes**
Camarero Barra 1	barra1@almichain.local	Solo Punto de venta: Usuario
Camarero Barra 2	barra2@almichain.local	Solo Punto de venta: Usuario

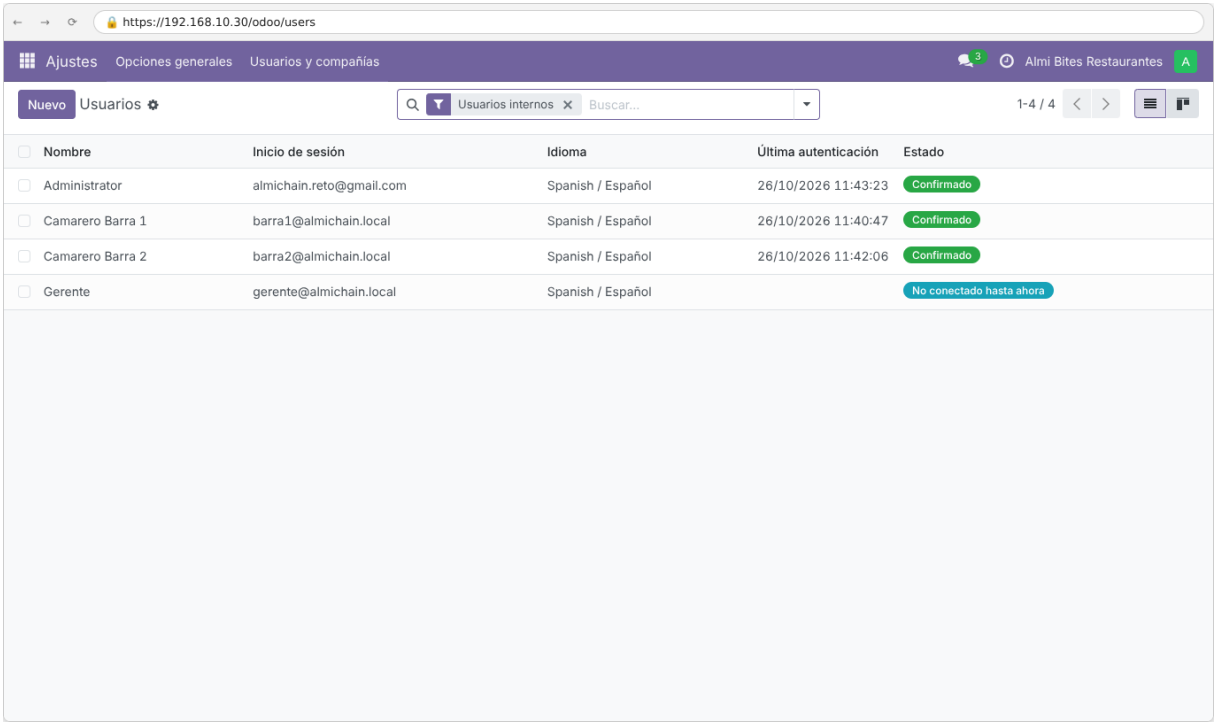


Figura 24. Usuarios de Odoo

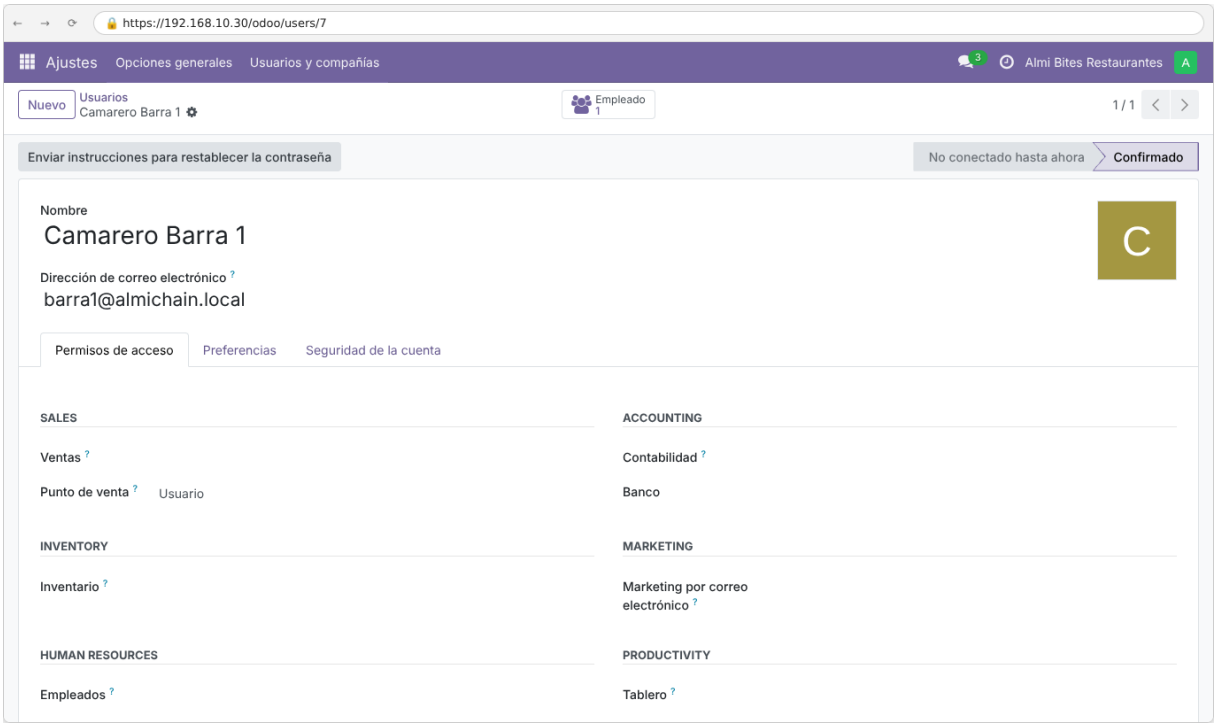


Figura 25. Camarero Barra 1: solo Punto de venta como Usuario

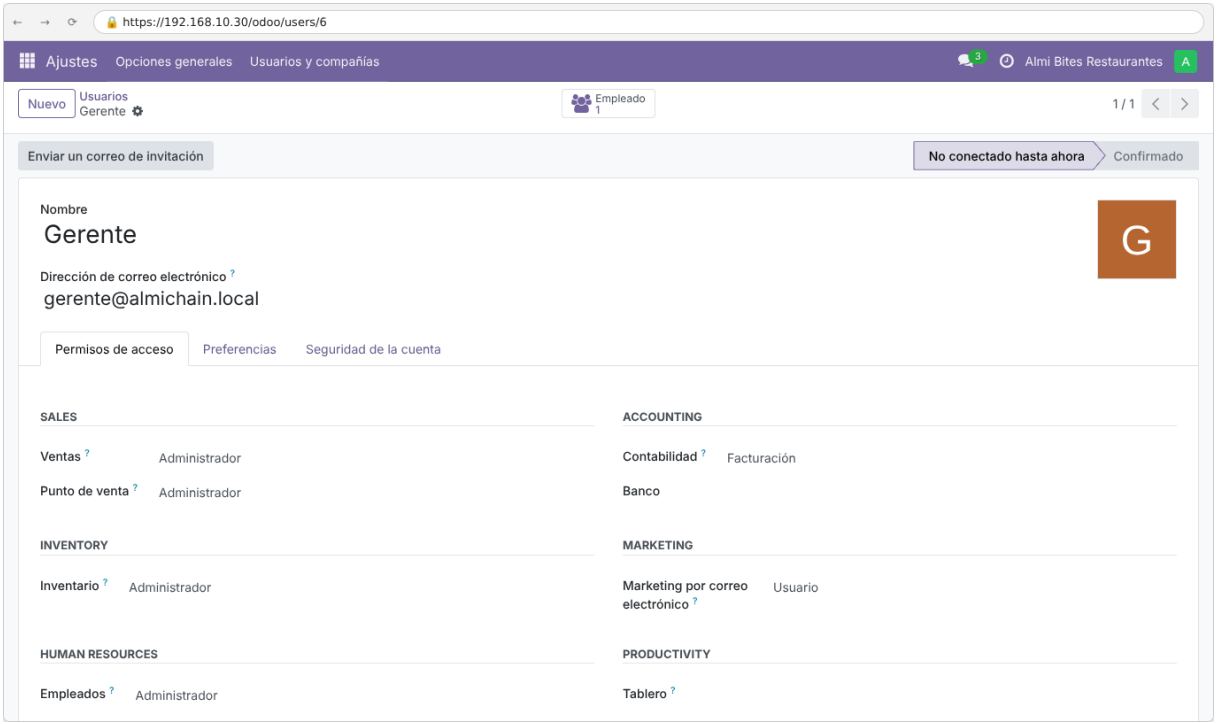


Figura 26. Permisos del Gerente

Empleados: Gorka Larrea (Gerente, Dirección, usuario Gerente), Nerea Olabarria (Camarera, Sala, usuario Camarero Barra 1), Asier Landa (Camarero, Sala, usuario Camarero Barra 2) y Miren Agirre (Cocinera, Cocina).

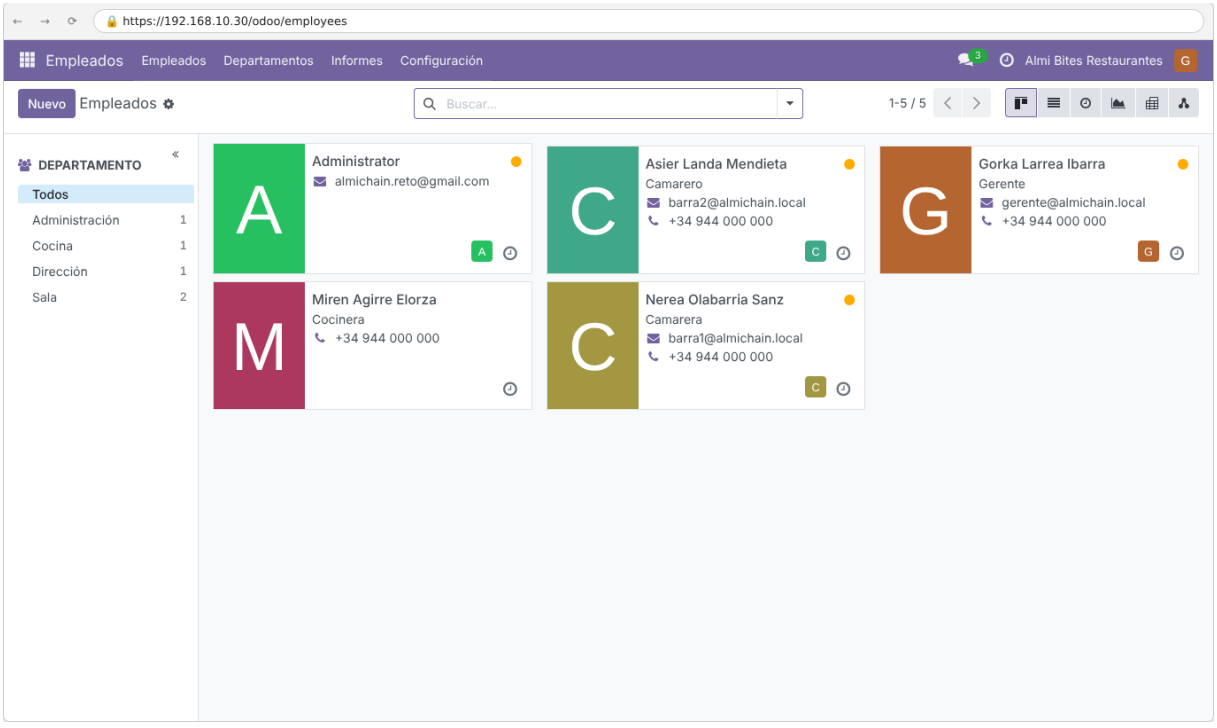


Figura 27. Empleados

Productos (E07): 4 bebidas y 4 platos, tipo Bienes con **Rastrear inventario**, **IVA 10 % (hostelería) con el precio con IVA incluido**, disponibles en el TPV en las categorías Bebidas y Platos, 50 unidades de stock inicial. **Clientes**: Ane Etxeberria, Jon Arrieta y Lucía Martín.

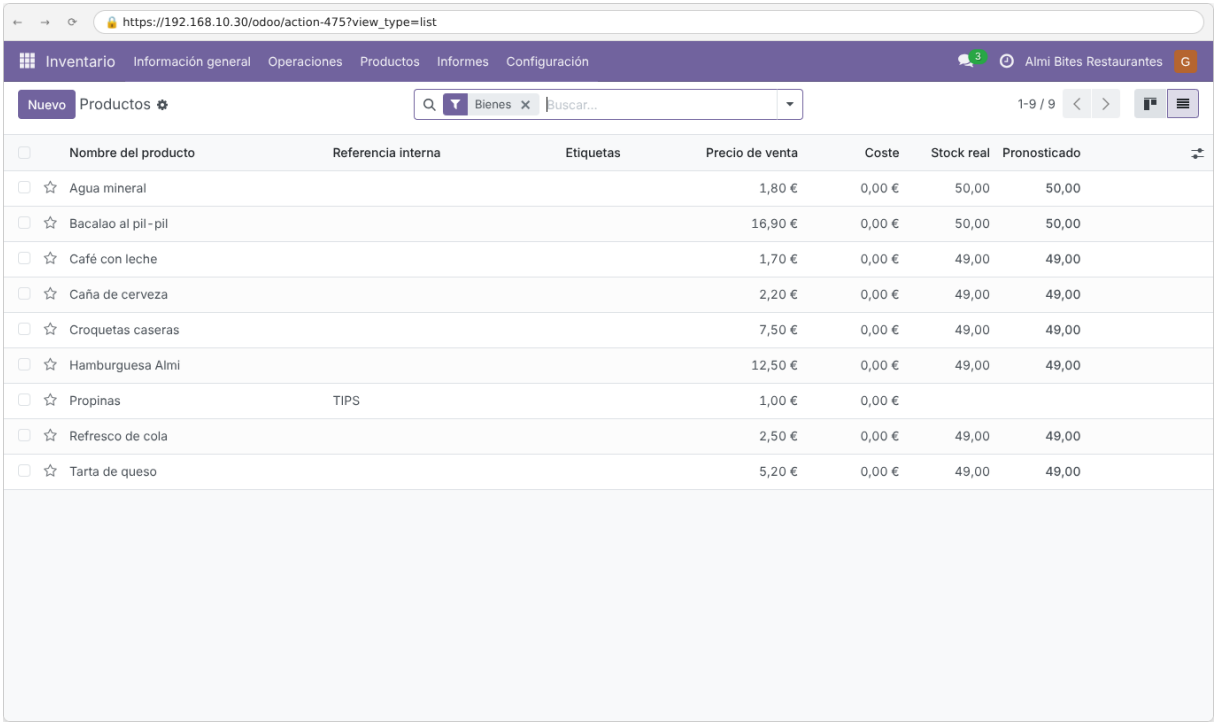


Figura 28. Productos con su stock (después de la prueba de venta)

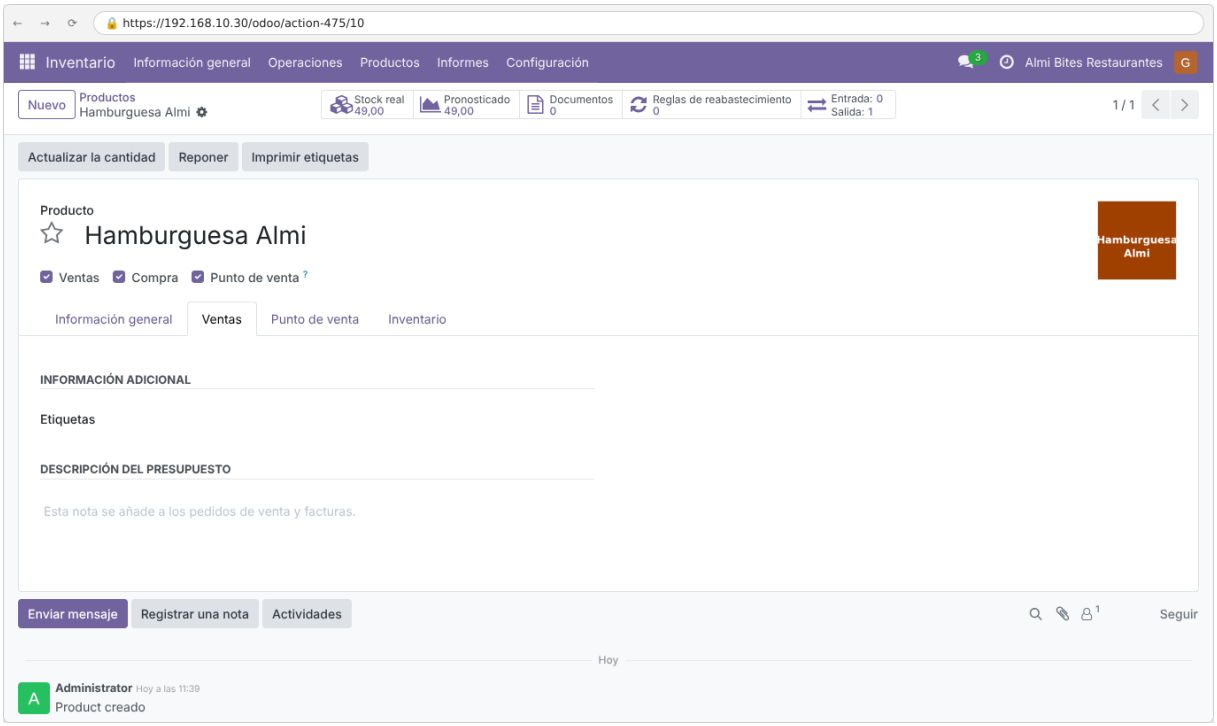


Figura 29. Producto con IVA 10 % hostelería

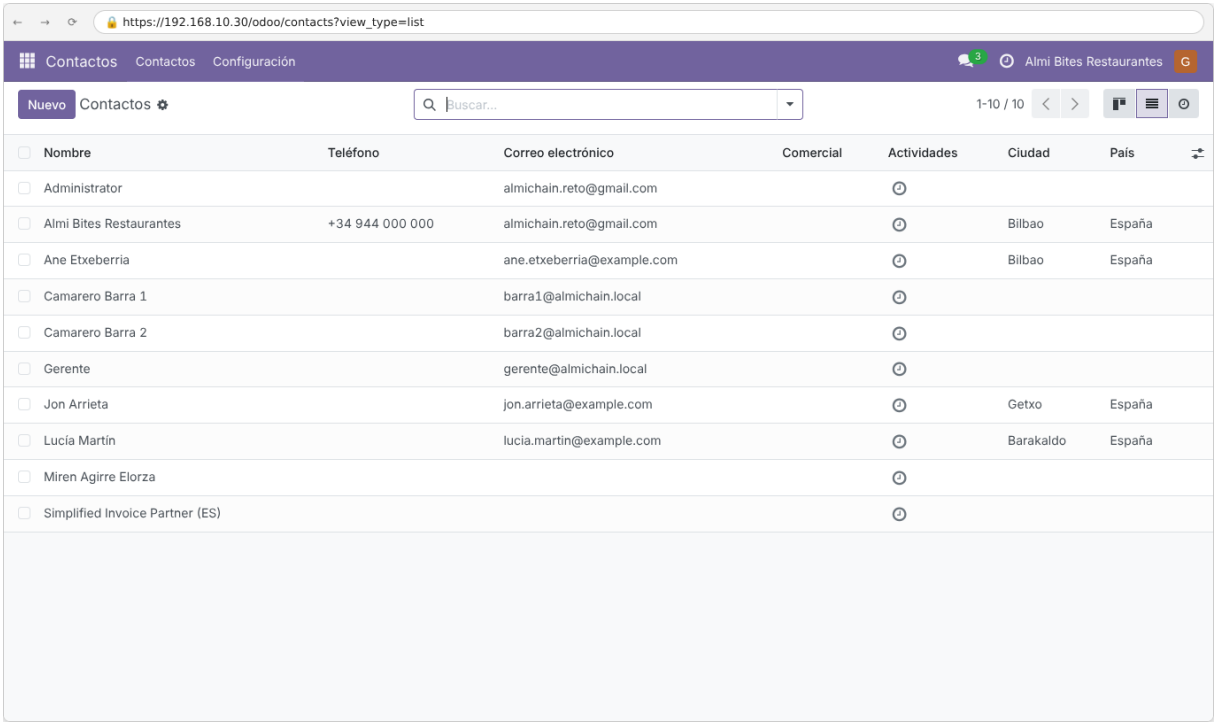


Figura 30. Clientes

TPV (E08): **Barra 1** (`config_id=1`) y **Barra 2** (`config_id=2`), cada uno con **Tarjeta** y su propia caja de **Efectivo** (Odoo no deja compartir un método de efectivo entre dos TPV), cabecera y pie del recibo e **impresión automática** sin vista previa.

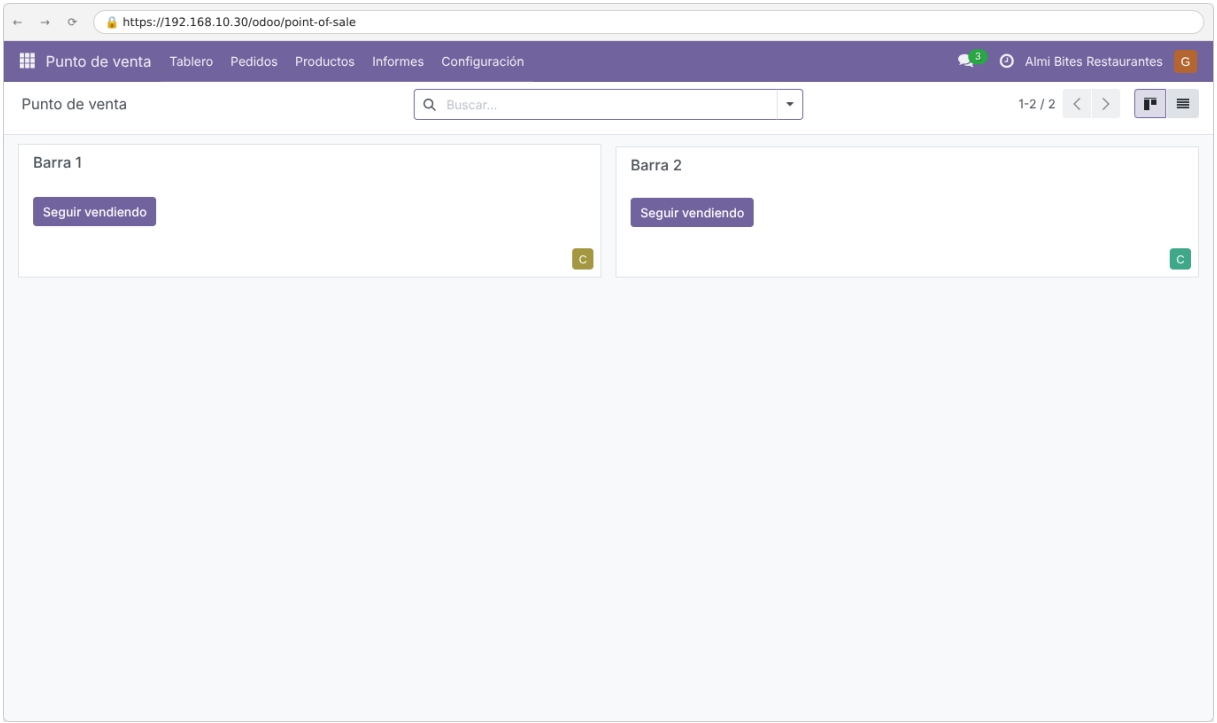


Figura 31. Panel del Punto de venta con Barra 1 y Barra 2

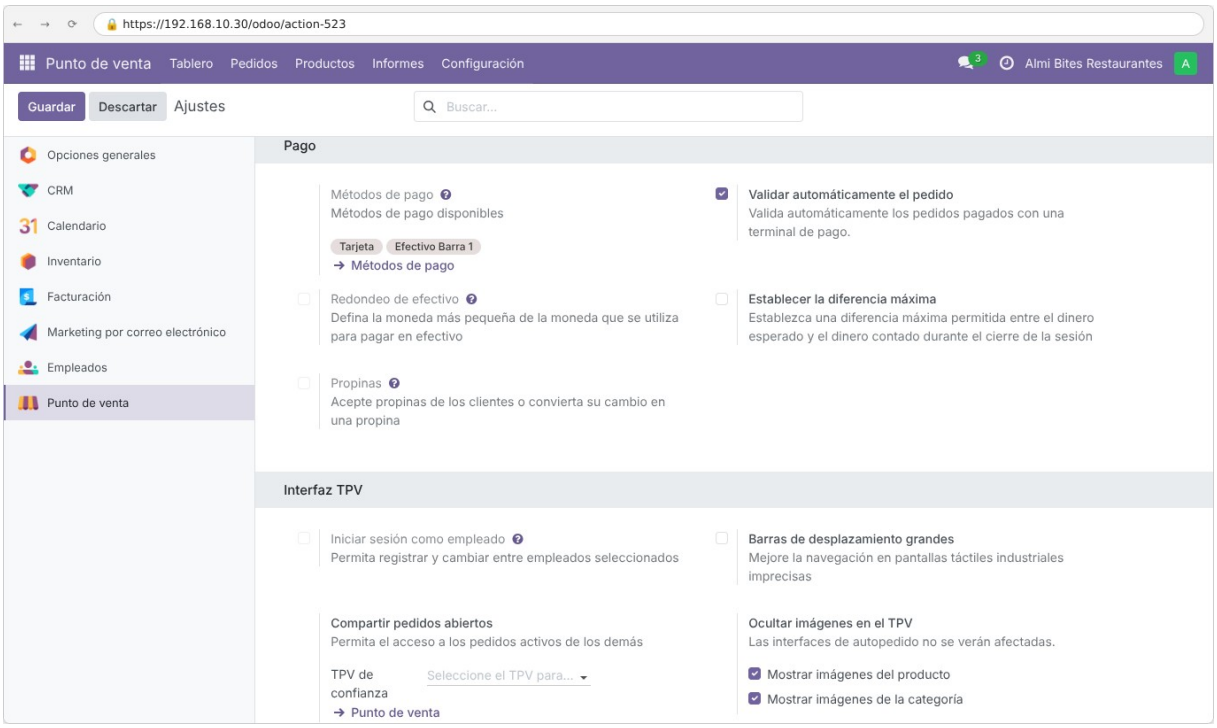


Figura 32. Ajustes de Barra 1: métodos de pago

Correo saliente (E09): servidor `smtp.gmail.com`, puerto 587, STARTTLS, usuario `almichain.reto@gmail.com`. La **contraseña de aplicación** de Gmail se escribe a mano en Odoo y se guarda en KeePassXC.

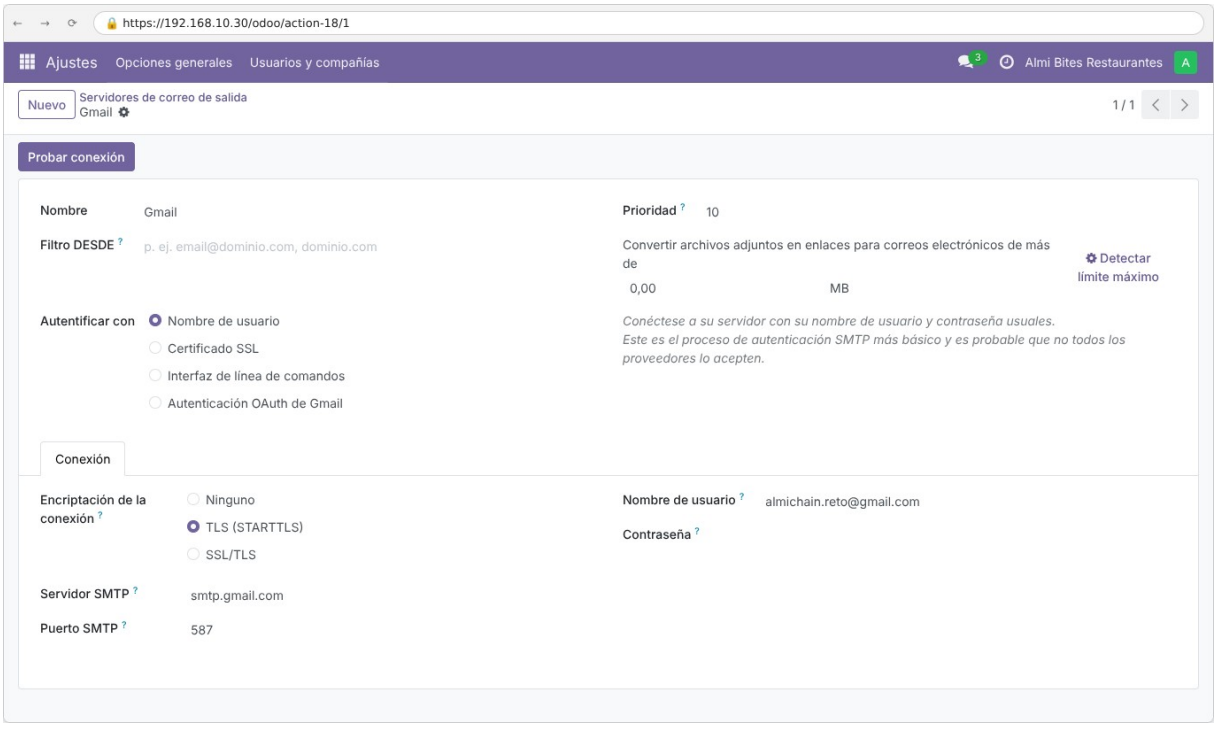


Figura 33. Servidor de correo de Gmail

Factura automática (E10): regla **TPV - Facturar pedidos con cliente** sobre *Pedidos de TPV*, que se dispara cuando el estado pasa a **Pagado**, con el filtro `[('partner_id', '!=', False), ('to_invoice', '=', False), ('account_move', '=', False)]`: el pedido tiene cliente, el camarero **no** ha marcado ya *Factura* en el TPV (si lo marca, Odoo la hace él) y todavía no tiene factura. Así nunca sale una factura doble:

```
for order in records:
    if order.partner_id and not order.account_move:
        order.action_pos_order_invoice()
```

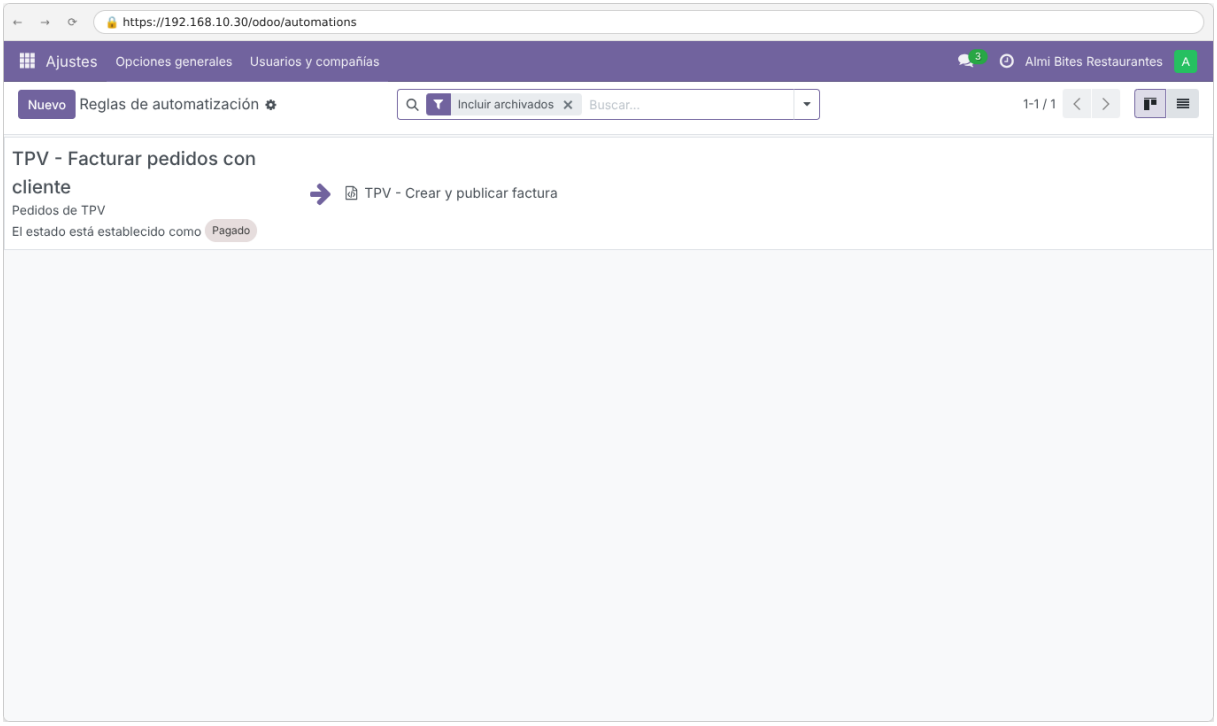


Figura 34. Reglas de automatización

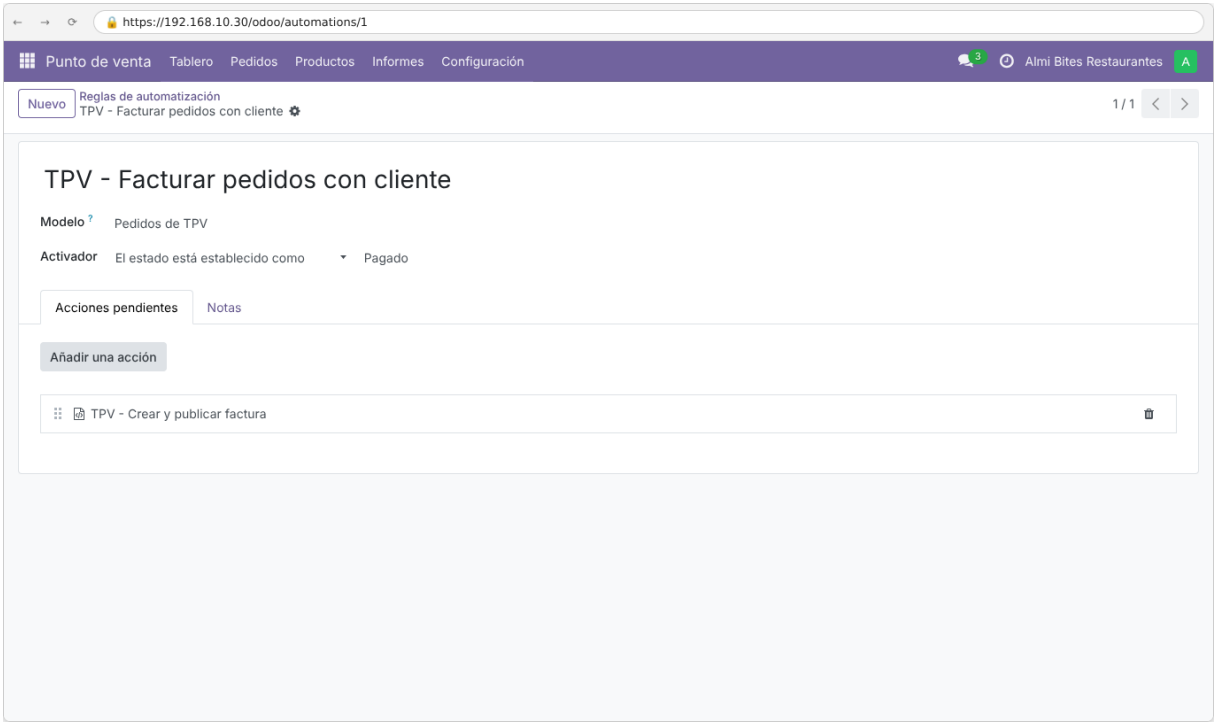


Figura 35. Código de la regla de factura automática

6. Prueba de principio a fin (E12)

Con el usuario **Camarero Barra 1** se abre la caja de **Barra 1**, se marcan una caña, un refresco y una hamburguesa, se elige la clienta **Ane Etxeberria** y se cobra en efectivo. Después, con **Camarero Barra 2**, una venta en **Barra 2** a **Jon Arrieta**.

Barra 1 (Camarero Barra 1, clienta Ane Etxeberria):

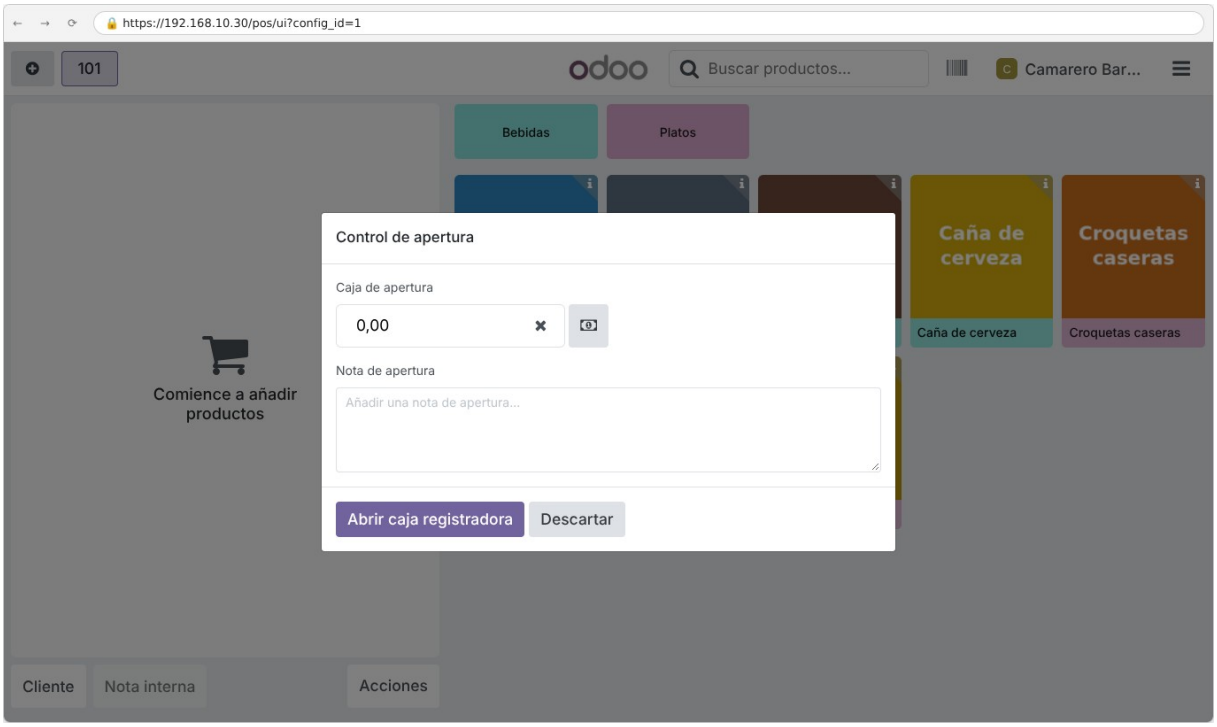


Figura 36. Barra 1: apertura de caja

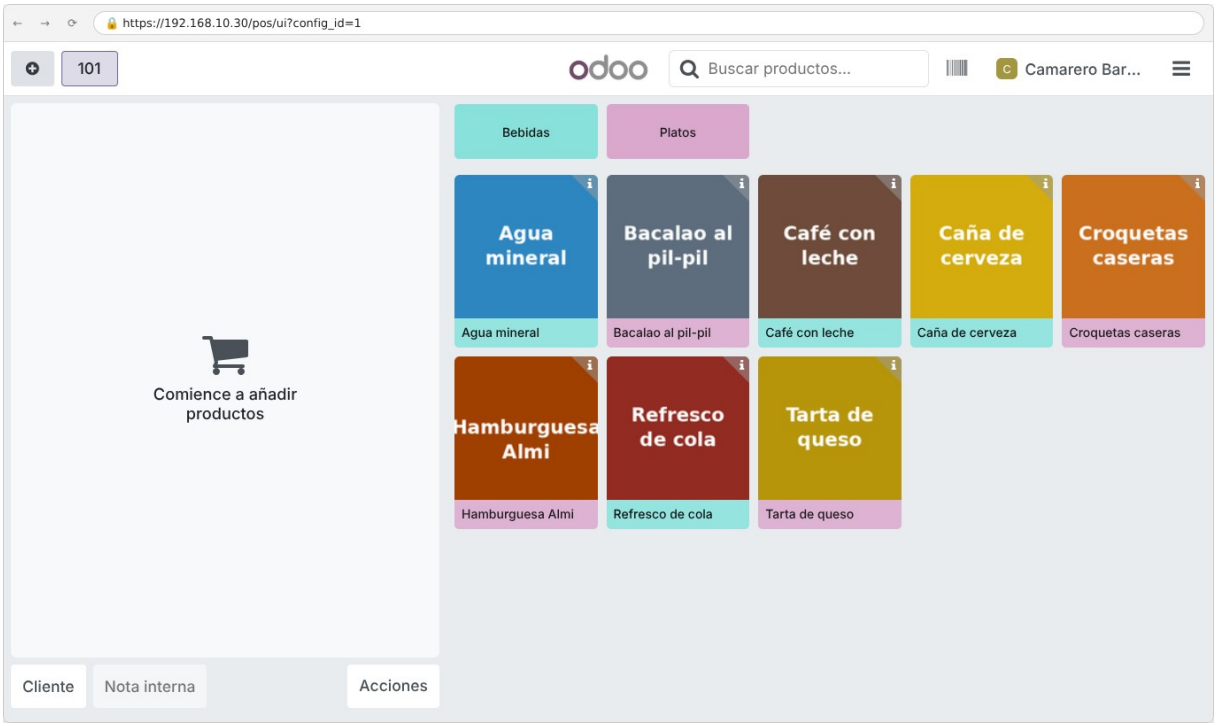


Figura 37. Barra 1: TPV abierto

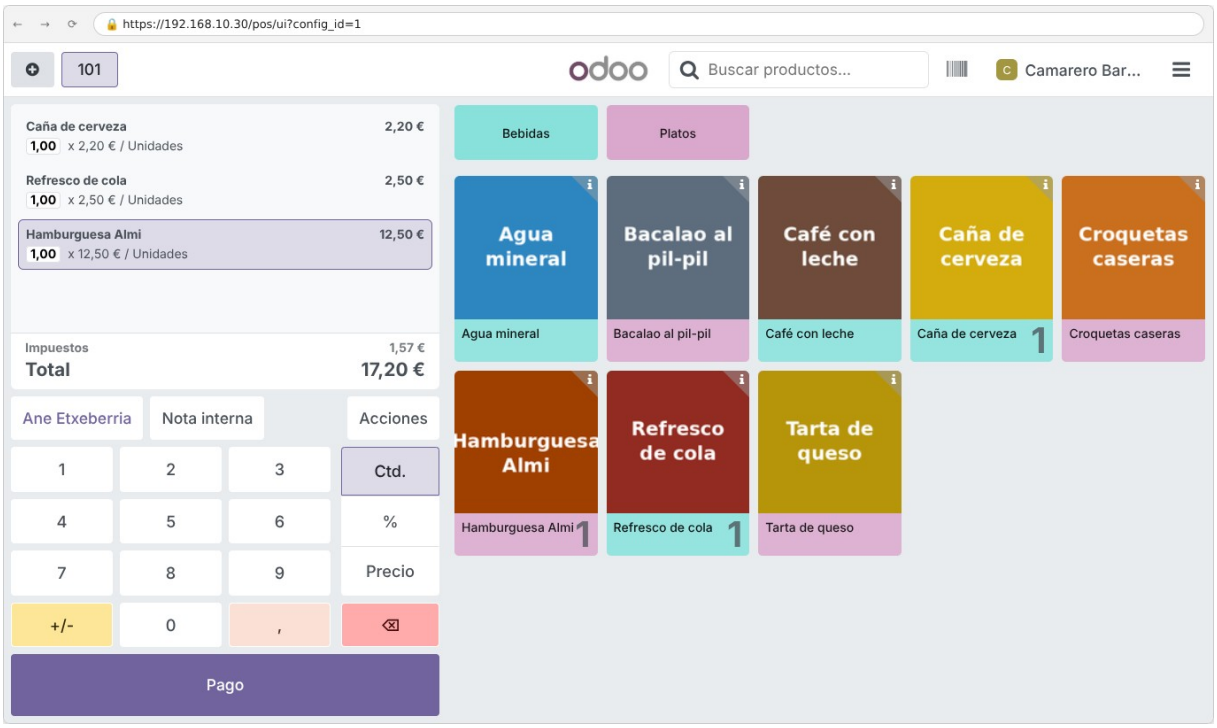


Figura 38. Barra 1: pedido con la clienta elegida

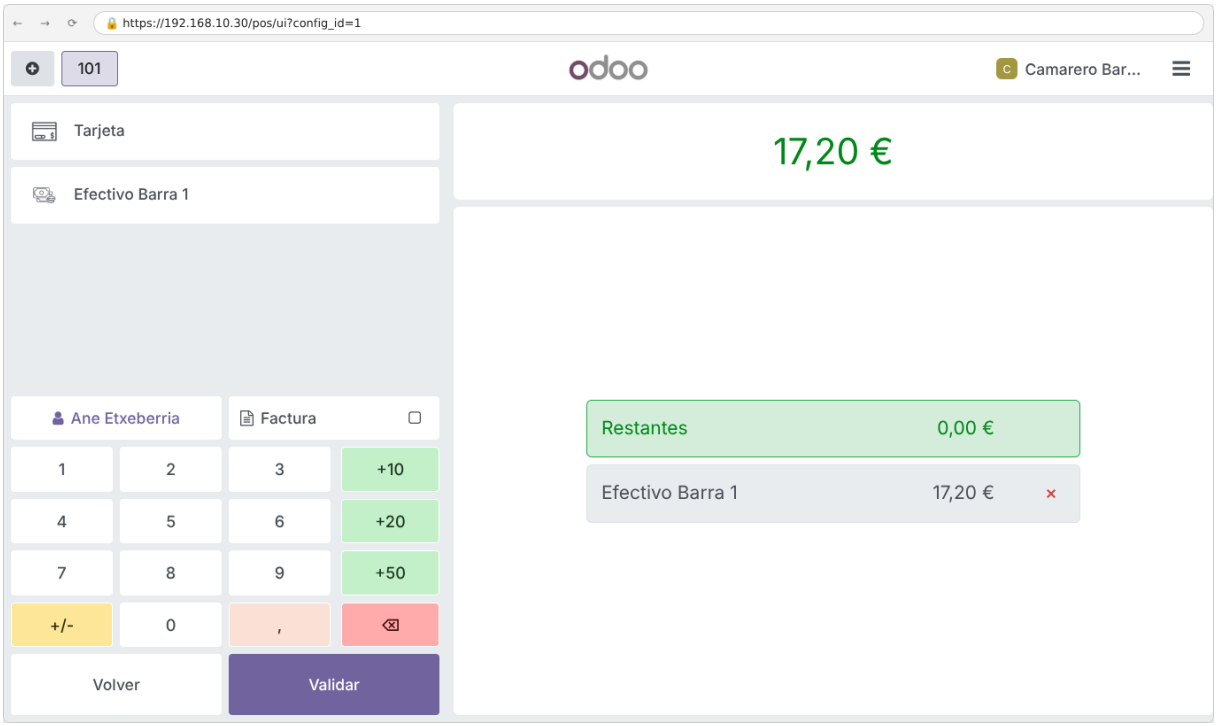


Figura 39. Barra 1: pago en efectivo



Figura 40. Barra 1: ticket que se manda a la impresora, con cabecera y pie

Barra 2 (Camarero Barra 2, cliente Jon Arrieta):

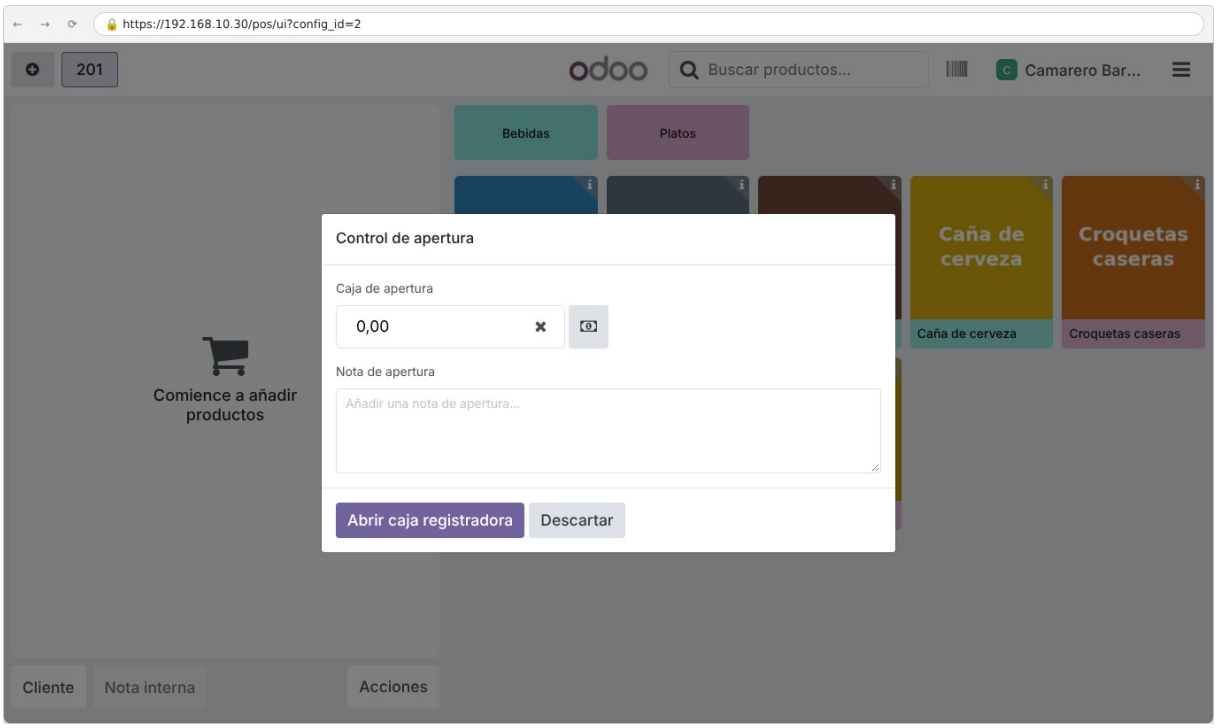


Figura 41. Barra 2: apertura de caja

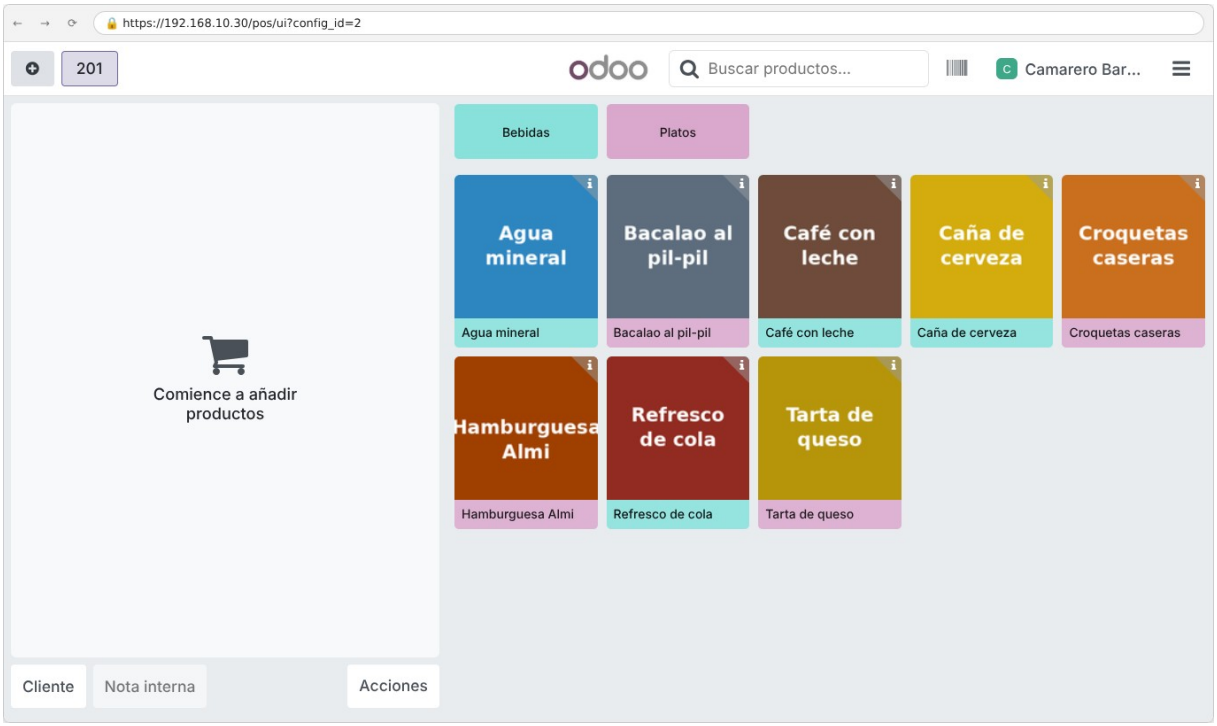


Figura 42. Barra 2: TPV abierto

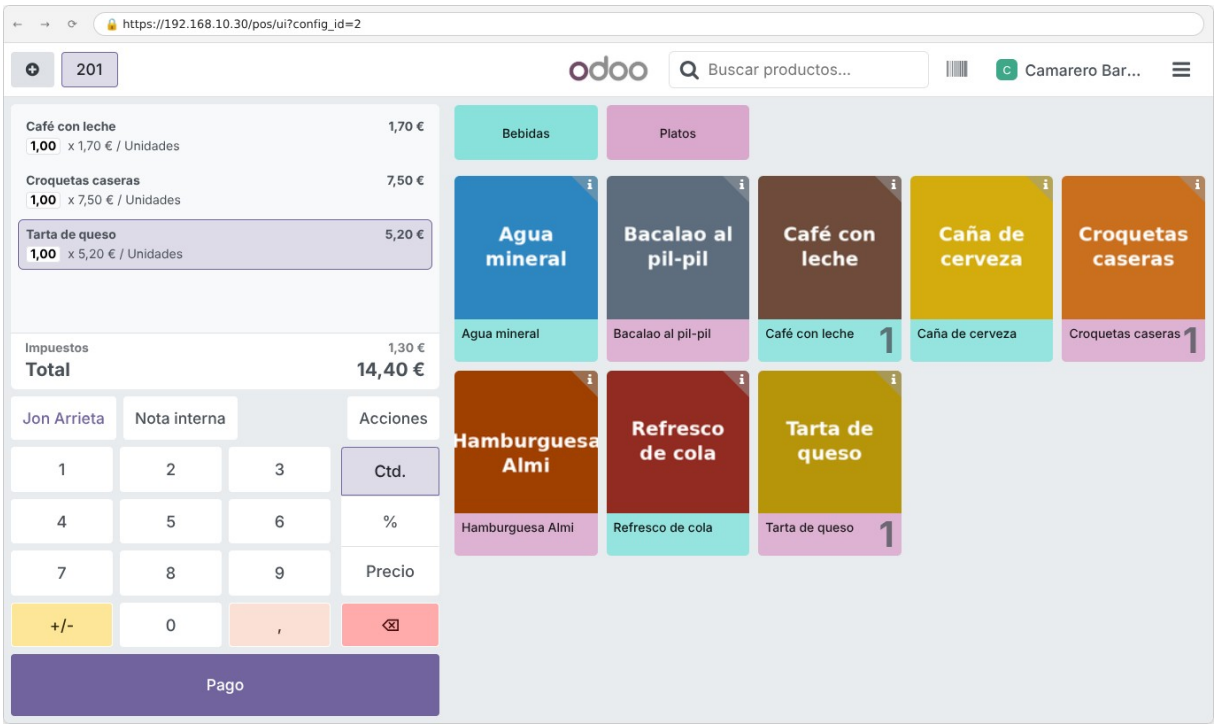


Figura 43. Barra 2: pedido con el cliente elegido

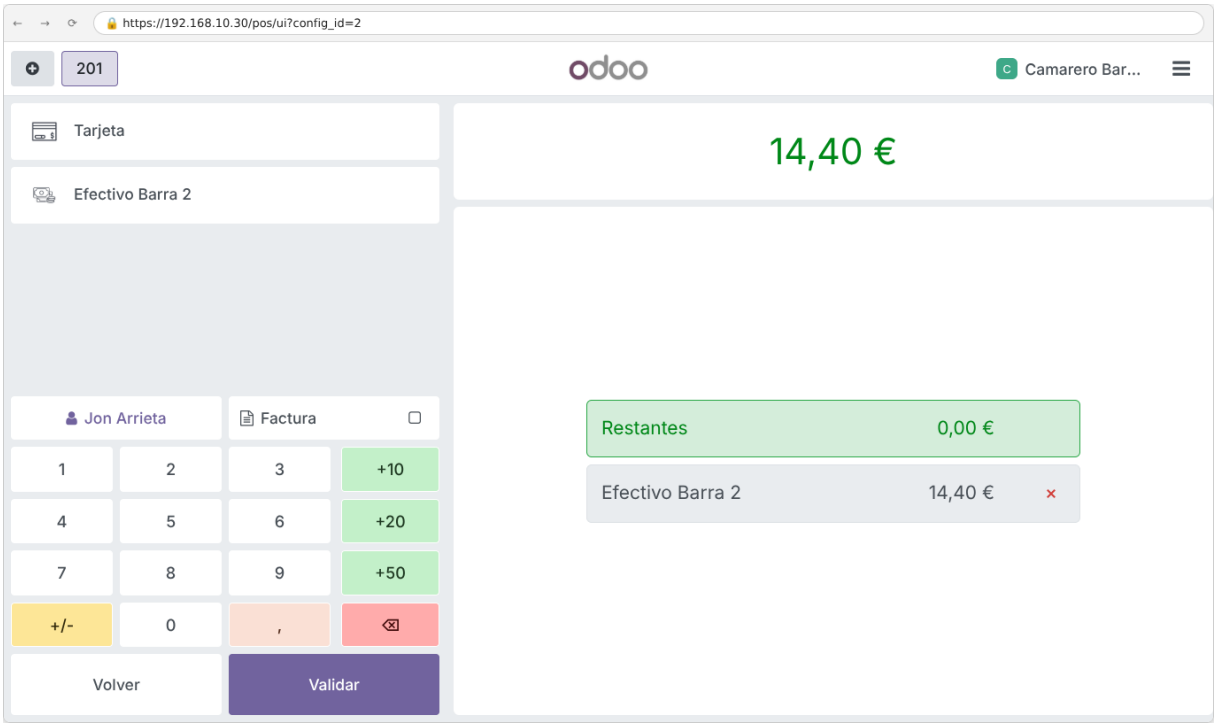


Figura 44. Barra 2: pago



Figura 45. Barra 2: ticket que se manda a la impresora

Resultado visto por el **Gerente**:

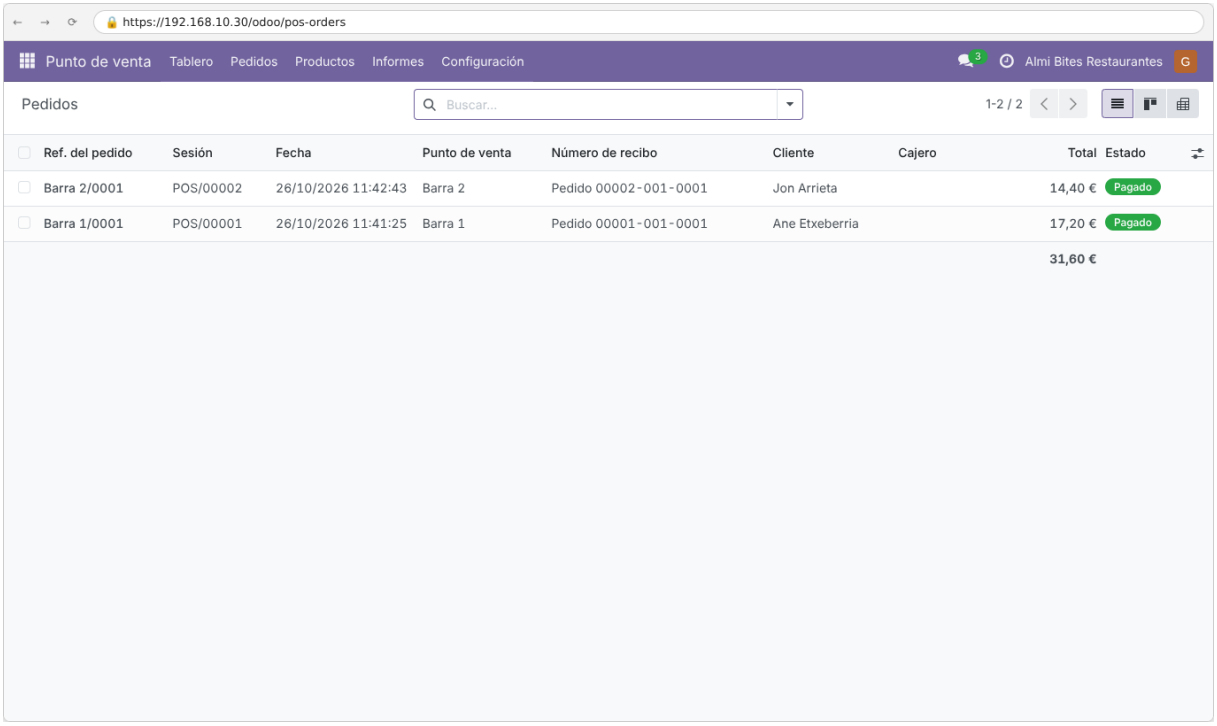


Figura 46. Los dos pedidos, cada uno en su TPV

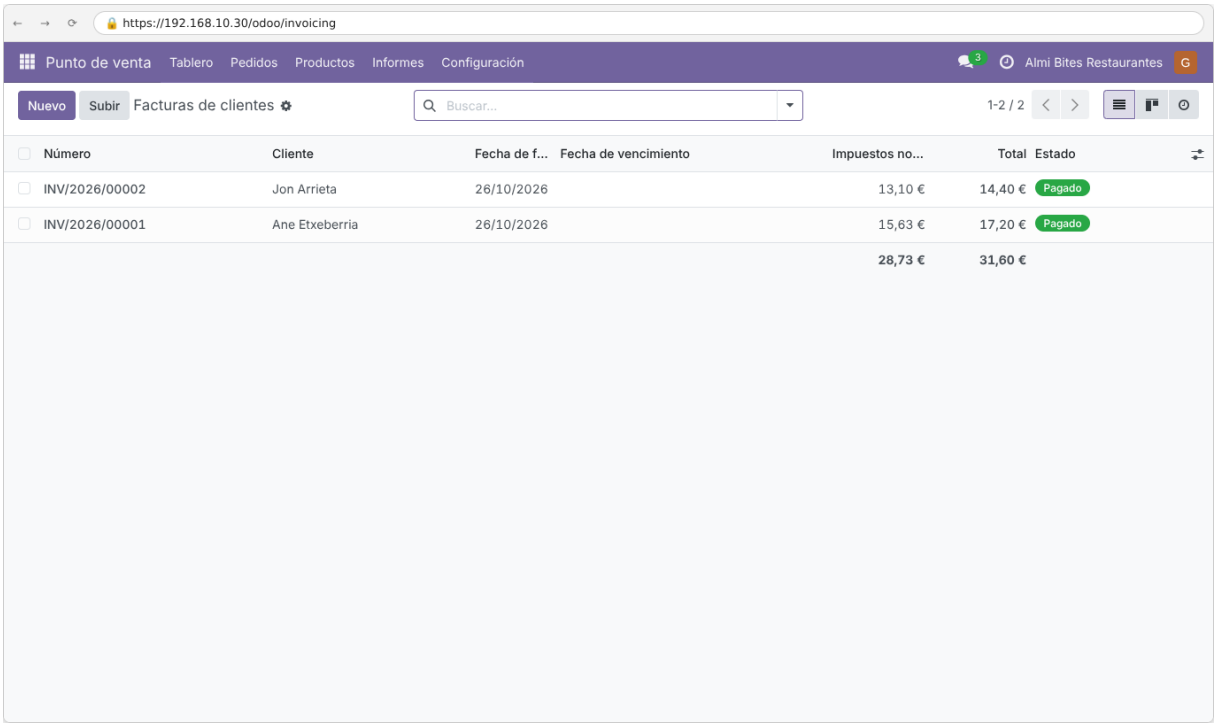


Figura 47. Las dos facturas creadas y publicadas solas

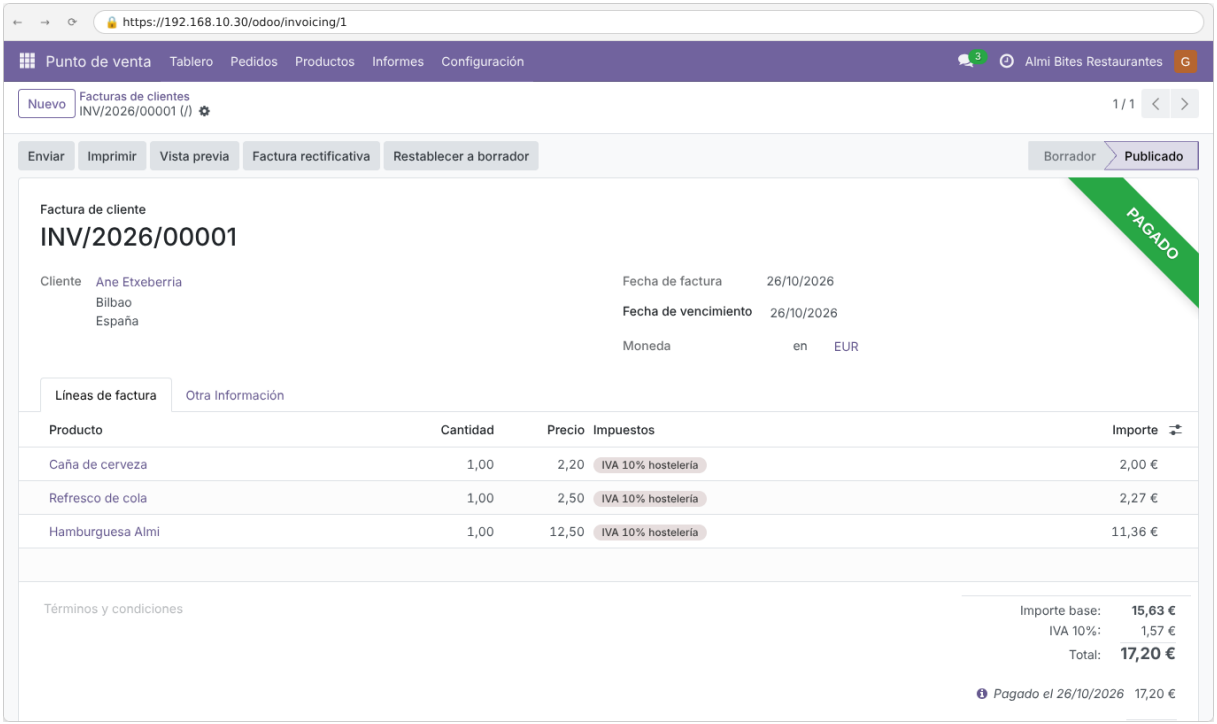


Figura 48. Factura INV/2026/00001 publicada y pagada, con IVA 10 %

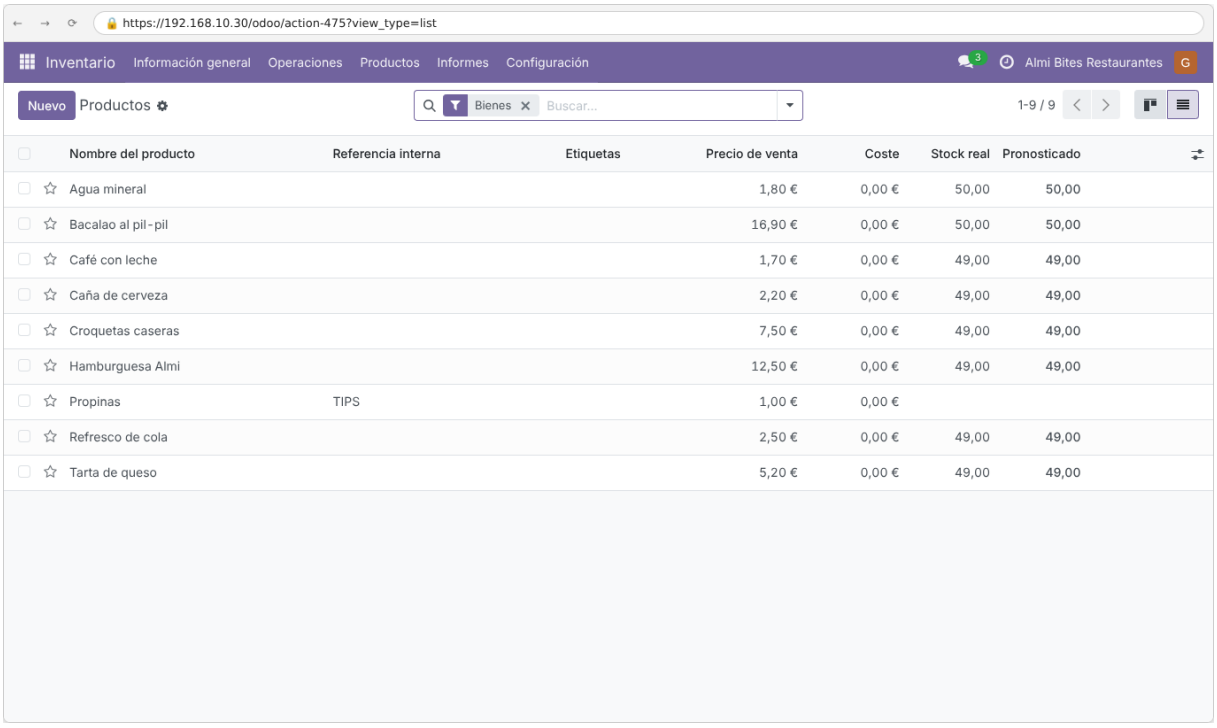


Figura 49. El stock ha bajado en lo vendido

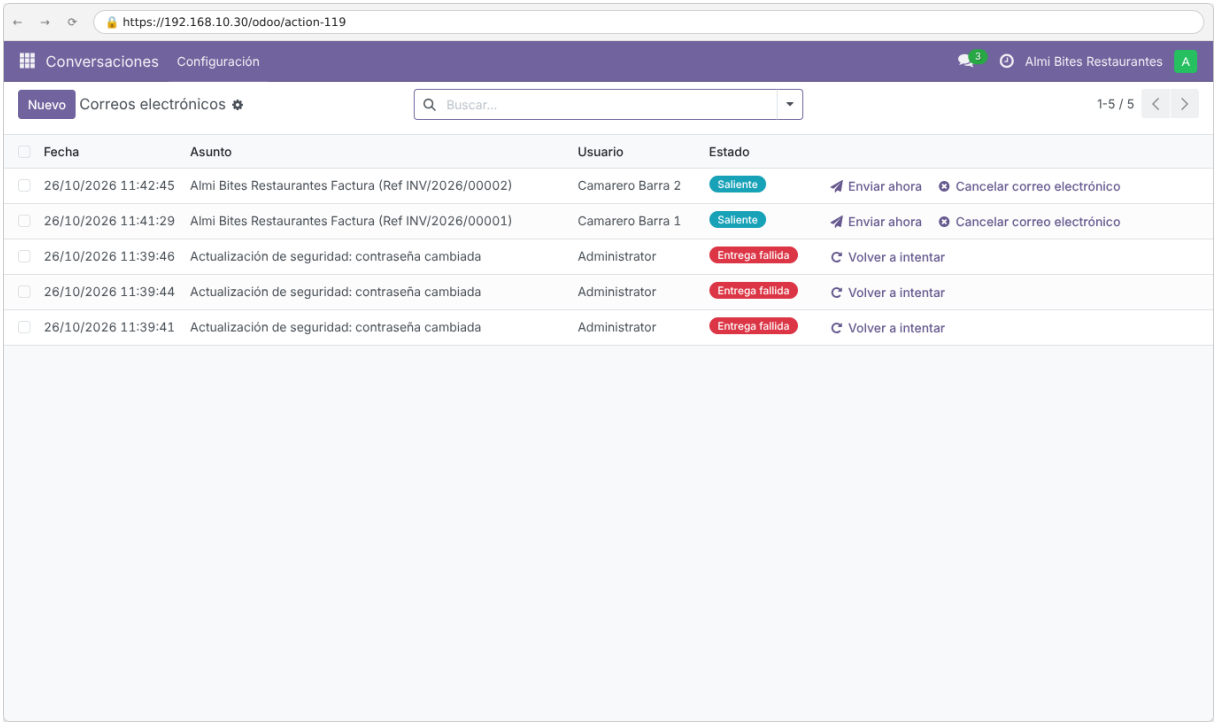


Figura 50. Correos de las facturas con el PDF adjunto, en la cola de salida

7. Copia de seguridad de Odoo (E11)

Script `05-erp-odoo/scripts/copia-odoo.sh` en `/usr/local/bin/`, todos los días a las **02:30** con cron de root. Guarda la base de datos (`pg_dump` en formato propio), los adjuntos (`filestore`), las huellas SHA-256, borra las copias de más de 7 días y lo manda todo a **ALMI-BD** por SSH con una clave que solo sirve para eso (ver 07-bd-raid-copias (`07-bd-raid-copias/README.md`)).

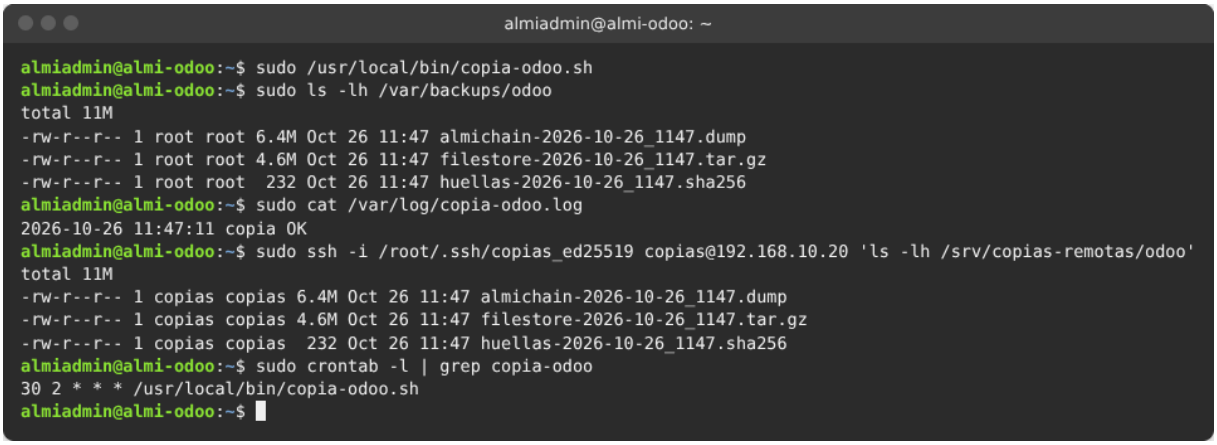


Figura 51. Copia hecha y programada en cron (servidor de pruebas: la «copia en ALMI-BD» iba al mismo servidor y el `ls` remoto que se ve ahora lo rechaza la clave restringida; la prueba entre dos máquinas está en el apartado 8)

Prueba de restauración en una base nueva `almichain_restaurada`: tiene el mismo número de pedidos que la original.

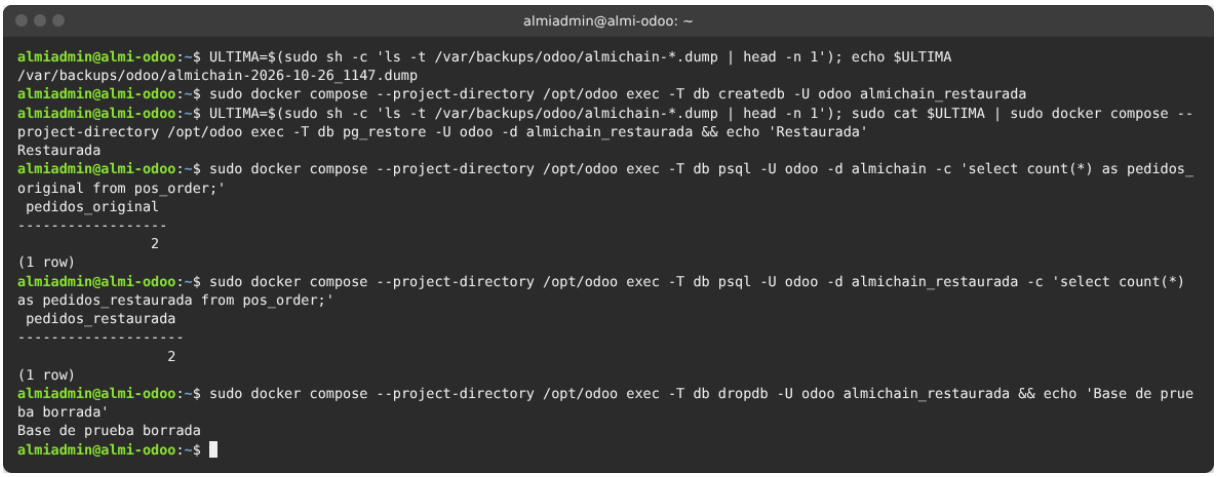


Figura 52. Restauración de la copia de Odoo

8. Máquina virtual exportada

ALMI-ODOO está exportada como `ALMI-ODOO.ova` (ver docs/maquinas-virtuales.md (`docs/maquinas-virtuales.md`)) y se construye con `herramientas/construir-vm` (`herramientas/README.md`). Se probó **arrancando su disco en QEMU** (no en VirtualBox) con `herramientas/probar-vm/pruebas-odoo.sh` (`herramientas/probar-vm/pruebas-odoo.sh`). En el primer arranque (`05-erp-odoo/scripts/primer-arranque-odoo.sh`) genera sus propios secretos, carga las imágenes de Docker que lleva dentro (no necesita Internet), restaura la base `almichain` con los datos de este bloque, arranca Odoo y nginx y aplica el cortafuegos. Cada captura tiene al lado su transcripción `.txt`.

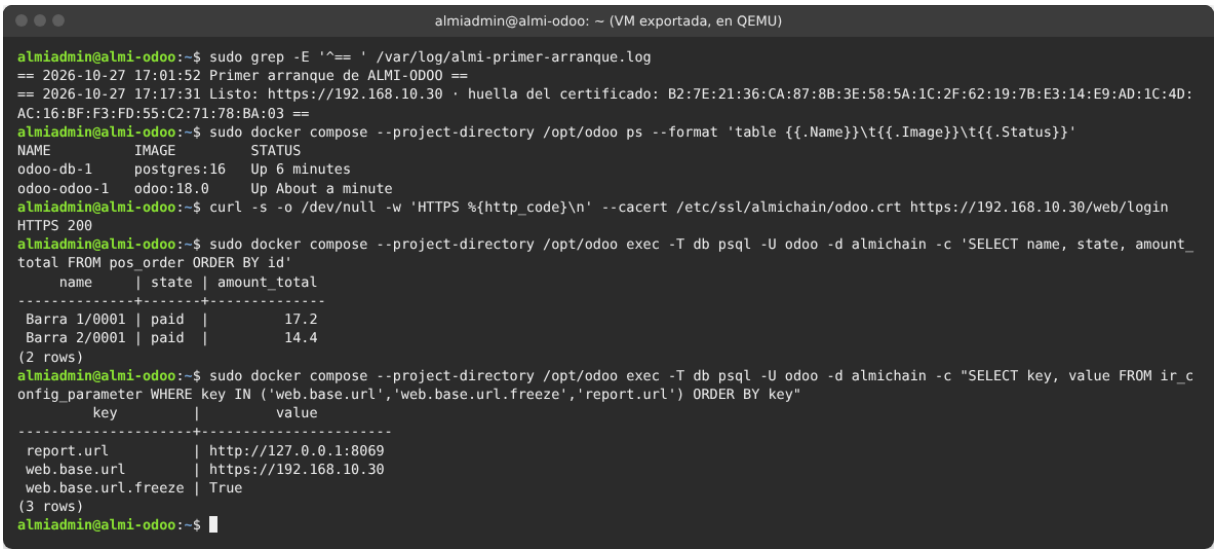


Figura 53. Primer arranque terminado: contenedores en marcha, HTTPS 200, los 2 pedidos restaurados y la dirección pública fija

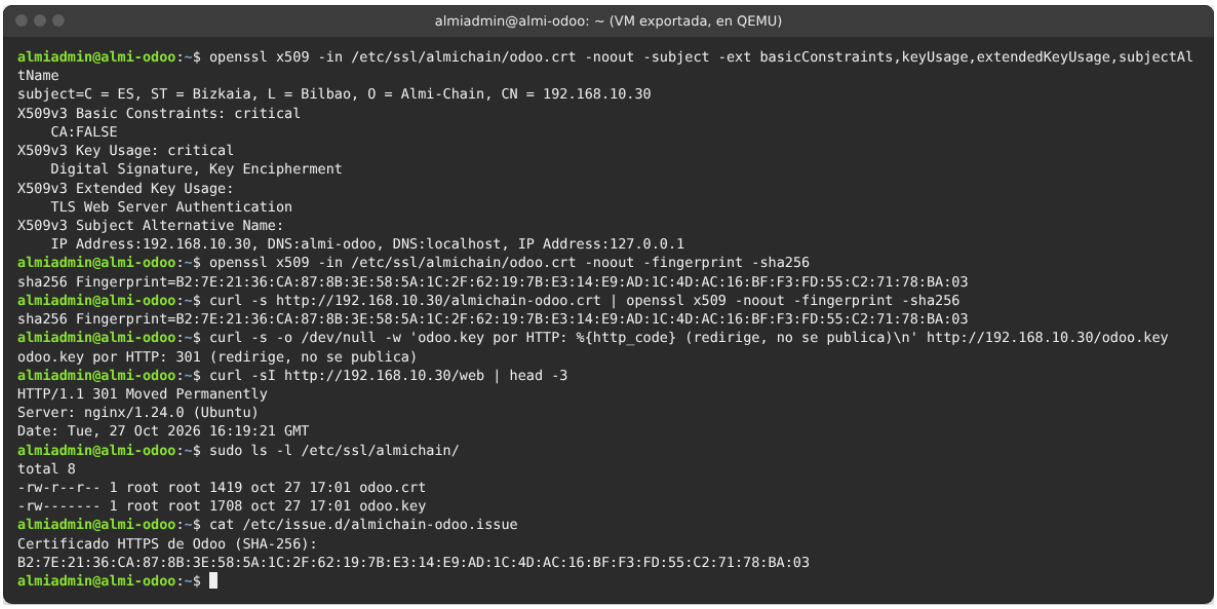


Figura 54. Certificado de servidor (CA:FALSE, serverAuth, SAN con la IP, almi-odoo y localhost), la misma huella publicada por HTTP, la clave no se publica y la huella en la consola

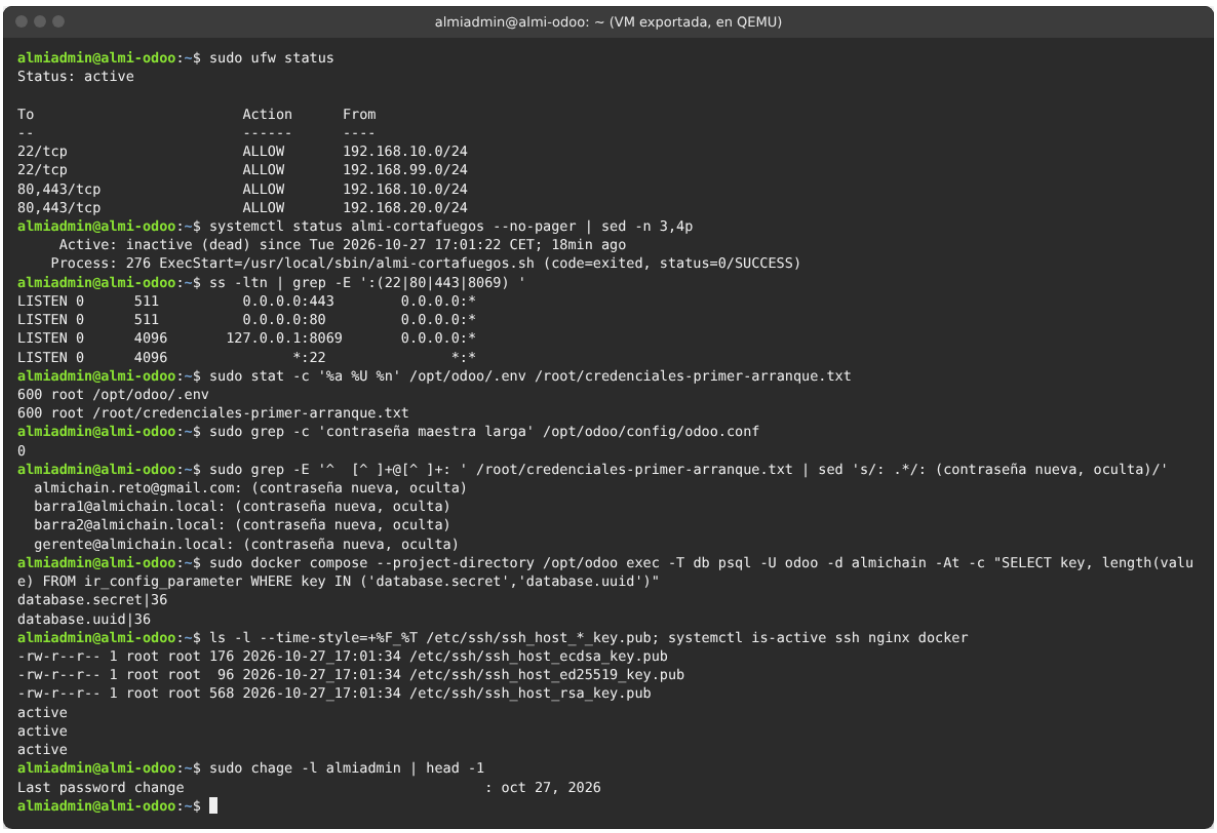


Figura 55. Cortafuegos con las redes del diseño puesto antes que la red (17:01:22, antes de crear las claves SSH a las 17:01:34), Odoo solo en 127.0.0.1:8069, secretos solo para root, contraseña maestra cambiada, contraseñas nuevas para los 4 usuarios de Odoo, secreto de sesiones nuevo, claves SSH propias y contraseña de almiadmin cambiada

Copias cruzadas entre las dos VM (ALMI-ODOO y ALMI-BD encendidas a la vez en la misma red): la copia de Odoo llega a ALMI-BD, la de la base de datos llega aquí, y la clave de copias **no puede** ejecutar órdenes, borrar, leer ni salir de su carpeta.

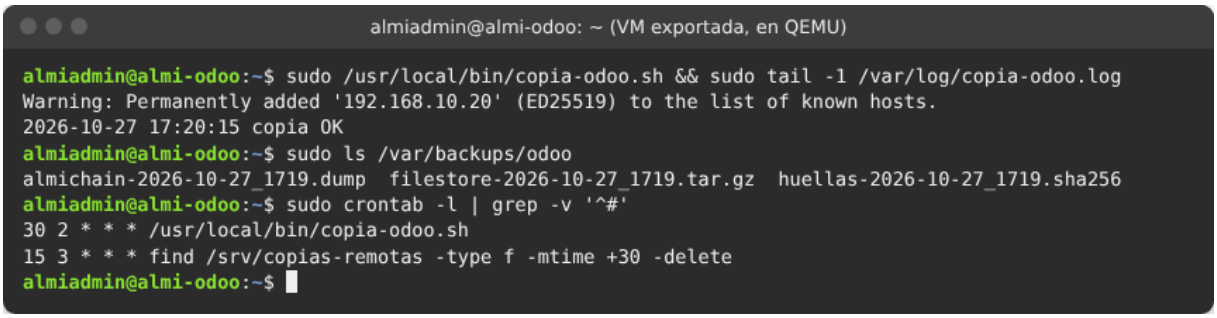


Figura 56. copia-odoo.sh: copia local, envío a ALMI-BD y cron

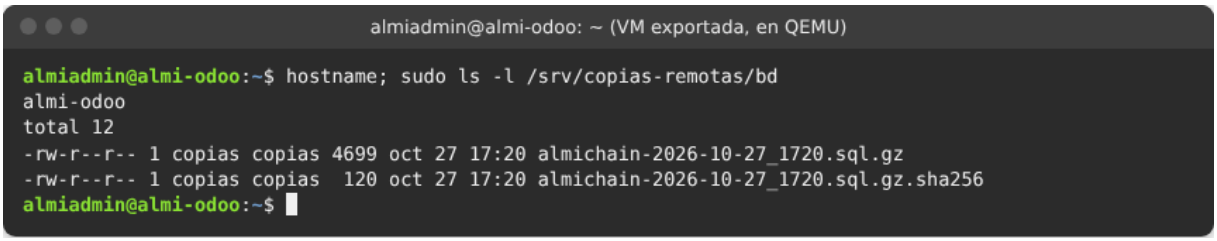


Figura 57. En ALMI-ODOO llegan las copias de la base de datos

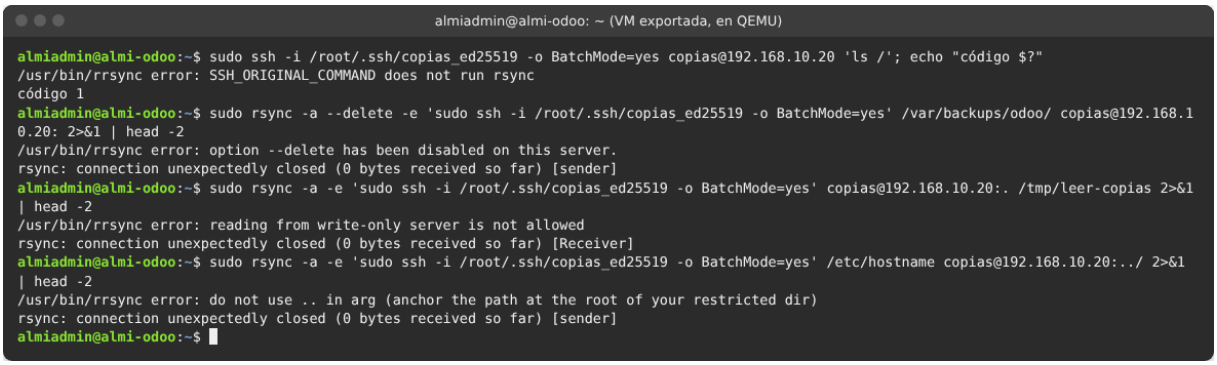


Figura 58. La clave de copias solo puede escribir: ls, –delete, leer y ../ rechazados por rrsync

Seguridad aplicada

- **Solo HTTPS**: nginx redirige el 80 al 443 (salvo la descarga de la parte pública del certificado); Odoo solo escucha en 127.0.0.1. Certificado de servidor (`CA:FALSE`) instalado como de confianza en los Windows tras comprobar su huella.
- **Cortafuegos** ufw: entrada denegada salvo SSH desde las VLAN 10 y 99, y HTTP/HTTPS desde las VLAN 10 y 20.
- **Secretos propios en cada VM importada**: en el primer arranque se generan la contraseña de PostgreSQL, la contraseña maestra de Odoo, contraseñas nuevas para los 4 usuarios de Odoo, el secreto con el que Odoo firma las sesiones (`database.secret`), el certificado con su clave y las claves de host SSH. La OVA no los lleva.
- **Cortafuegos desde el primer segundo**: en la VM exportada lo pone el servicio `almi-cortafuegos` (`05-erp-odoo/scripts/cortafuegos-odoo.sh`) antes de que haya red.
- **Gestor de bases de datos oculto** (`list_db = False`) y contraseña maestra larga.
- **Mínimo privilegio** en Odoo: los camareros solo usan el TPV; el gerente no tiene Ajustes; solo el administrador configura.
- **Secretos fuera del repositorio**: `.env`, `odoo.conf` real, clave del certificado y contraseñas en KeePassXC.
- `almiadmin` sin grupo `docker`; `/opt/odoo` y las copias con permisos 700.
- Copias diarias en otra máquina (regla 3-2-1 junto con las de la BD).

Pruebas

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
1	Odoo solo por HTTPS	`curl -I http://192.168.10.30/`	301 a https y candado	301 Moved Permanently → https	04 (`05-erp-odoo/capturas/04-nginx-https.png`)
2	Puerto 8069 cerrado desde fuera	`nc -zv 192.168.10.30 8069` (en Windows: `Test-NetConnection`)	Conexión rechazada	Connection refused (en el servidor de pruebas, desde la misma máquina); en la VM, 8069 solo escucha en 127.0.0.1 (38 (`05-erp-odoo/capturas/38-vm-secretos-cortafuegos.png`)). Desde un Windows: pendiente	05 (`05-erp-odoo/capturas/05-certificado.png`)
3	Gestor de bases de datos oculto	Abrir `/web/database/manager`	No se abre	“The database manager has been disabled”	08 (`05-erp-odoo/capturas/08-gestor-bd-oculto.png`)
4	Camareros sin facturación	Entrar como Camarero Barra 1	Sin menús de Facturación, Inventario ni CRM	Solo Punto de venta (y apps comunes: Conversaciones, Calendario, Contactos, Empleados)	31 (`05-erp-odoo/capturas/31-camarero-solo-tpv.png`)
5	Caso real completo	Venta con cliente en Barra 1 y Barra 2	Pedido facturado, factura publicada, correo con PDF y stock bajado	Factura, publicación y stock: correcto. **Correo: solo comprobado que queda en la cola con el PDF**; el envío real está pendiente de poner la contraseña de aplicación de Gmail	27 (`05-erp-odoo/capturas/27-pedidos-tpv.png`) 28 (`05-erp-odoo/capturas/28-facturas-publicadas.png`) 30 (`05-erp-odoo/capturas/30-stock-despues-de-vender.png`) 33 (`05-erp-odoo/capturas/33-cola-de-correos.png`)
6	Ticket impreso	Cobrar en cada barra	Ticket con cabecera y pie	Ticket generado en pantalla con cabecera y pie.	ticket 1 (`05-erp-odoo/capturas/25-barra1-`

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
				Impresión en la impresora física: pendiente — en clase (bloque 01)	`4-ticket.png`) ticket 2 (`05-erp-odoo/capturas/26-barra2-4-ticket.png`)
7	Copia y restauración	Restaurar en `almichain_restaurada`	Mismo número de pedidos	2 y 2	35 (`05-erp-odoo/capturas/35-restauracion-odoo.png`)
8	VM exportada	Arrancar el disco de `ALMI-ODOO.ova` desde cero (en QEMU)	Odoo por HTTPS con los datos del reto	HTTPS 200 y 2 pedidos	36 (`05-erp-odoo/capturas/36-vm-primer-arranque.png`)
9	Certificado de servidor	`openssl x509 -ext basicConstraints…` en la VM	`CA:FALSE`, `serverAuth`, SAN correcto; misma huella por HTTP	Correcto	37 (`05-erp-odoo/capturas/37-vm-certificado-servidor.png`)
10	Secretos propios y cortafuegos	`ufw status`, `ss -ltn`, permisos	Redes del diseño; 8069 solo en local; secretos 600	Correcto	38 (`05-erp-odoo/capturas/38-vm-secretos-cortafuegos.png`)
11	Copias cruzadas y clave restringida	`copia-odoo.sh`, `copia-bd.sh` y órdenes prohibidas con la clave	Las copias llegan; lo demás se rechaza	Correcto	39 (`05-erp-odoo/capturas/39-vm-copia-a-almi-bd.png`) 40 (`05-erp-odoo/capturas/40-vm-copias-recibidas-de-bd.png`) 41 (`05-erp-odoo/capturas/41-vm-clave-copias-restringida.png`)

Las pruebas 1 a 7 se hicieron en el servidor de pruebas (ver el origen de las evidencias en el README principal) y las 8 a 11 con la VM exportada en QEMU. En VirtualBox, y con los Windows de verdad, quedan por repetir en clase.

Problemas encontrados y solución

Problema	Causa	Solución
`cd /opt/odoo` da *Permission denied*	La carpeta tiene permisos 700 (solo root) a propósito	Usar `sudo docker compose --project-directory /opt/odoo …`
El cliente recibía **dos correos** por	En Odoo 18, al facturar un pedido	Se quitó la regla 2. Queda solo la

Problema	Causa	Solución
factura (uno sin PDF y con “Ref n/a”)	del TPV a un cliente con correo, Odoo ya envía la factura en PDF. La regla 2 de la guía (“enviar al publicar”) lo duplicaba, y además saltaba antes de que la factura tuviera número	regla 1 (facturar) y el envío lo hace Odoo
Al abrir caja salía “configure la dirección de correo del remitente”	Los usuarios de los camareros no tenían correo en su contacto	Poner correo a cada usuario (gerente@, barra1@, barra2@almichain.local)
Los PDF de las facturas tardaban un minuto	wkhtmltopdf, dentro del contenedor, pedía los estilos a `https://192.168.10.30`	Parámetro del sistema `report.url = http://127.0.0.1:8069`
No se podía compartir el efectivo entre Barra 1 y Barra 2	Odoo exige un método de efectivo por TPV	Un diario de caja por barra: “Efectivo Barra 1” y “Efectivo Barra 2”
La ACL de Packet Tracer de la guía solo dejaba el puerto 80 a los TPV	Odoo se usa por HTTPS	Se añade `eq 443` en la ACL `TPV-SOLO-ODOO` (bloque 04)
El camarero puede abrir la lista de facturas si escribe la dirección a mano	El grupo “Punto de venta: Usuario” necesita leer facturas para reimprimirlas	No tiene menú de Facturación ni puede modificarlas. Se documenta como riesgo aceptado (captura 32 (`05-erp-odoo/capturas/32-camarero-ve-facturas-riesgo-aceptado.png`))

Fuentes

- Documentación de Odoo 18: Punto de venta, Facturación y Reglas de automatización — https://www.odoo.com/documentation/18.0/
- Imagen oficial de Odoo en Docker Hub — https://hub.docker.com/_/odoo
- nginx como proxy inverso de Odoo — https://www.odoo.com/documentation/18.0/administration/on_premise/deploy.html
- Guías paso a paso del equipo: `00-gestion/Guias-paso-a-paso-Reto1-Almi-Chain.docx` (`00-gestion/Guias-paso-a-paso-Reto1-Almi-Chain.docx`), parte 6

6. Web en AWS con Terraform, DuckDNS y HTTPS

Objetivo

La web informativa del restaurante **Almi Bites** (carta, galería, mapa con **geolocalización** y formulario de contacto), preparada para publicarse en **AWS Academy** con **Terraform**, dominio gratuito de **DuckDNS** y **HTTPS** de Let's Encrypt en **https://almichain2.duckdns.org**. Con `terraform destroy` y `terraform apply` se rehace todo igual (despliegue reproducible).

> **Estado**: el código está completo y revisado (el HCL se ha comprobado con un analizador, el script de arranque con `bash -n` y shellcheck, y la configuración de nginx con `nginx -t`), pero **en este repositorio no hay todavía pruebas del despliegue**: ni salida de `terraform apply`, ni capturas, ni el informe de Lighthouse. Donde se preparó el repositorio no había Terraform ni acceso a AWS Academy. El despliegue y la tabla de pruebas se completan en clase (tareas W05-W09).

Tareas del tablero: W01 a W09. El código también está en su propio repositorio, krako829/almichain-web-terraform; esta carpeta es una copia para tener todo el reto junto.

Requisitos

Elemento	Valor
Nube	AWS Academy Learner Lab (región `us-east-1`), coste 0 €
Herramientas en el PC	Terraform 1.x, credenciales del lab en `~/.aws/credentials`
Dominio	`almichain2.duckdns.org` (token de DuckDNS en KeePassXC)
Servidor	EC2 Ubuntu 24.04 con nginx y Certbot, IP elástica

Arquitectura

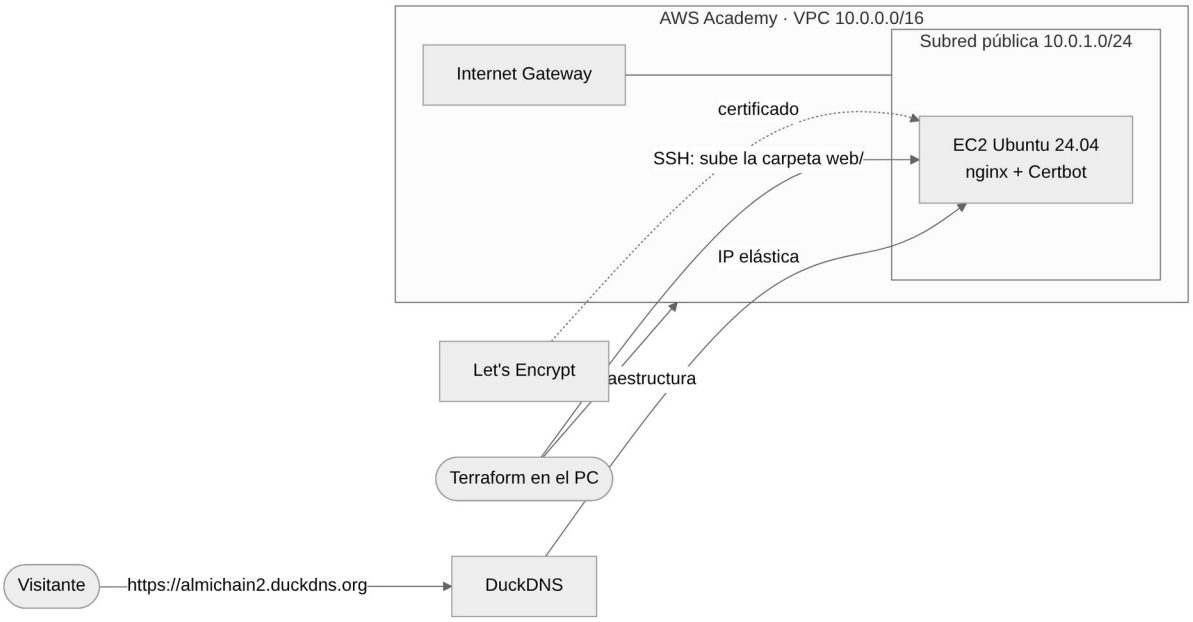


Figura 59. Diagrama

Terraform crea **13 recursos**: VPC, Internet Gateway, subred, tabla de rutas y su asociación, grupo de seguridad (22, 80, 443), clave privada, par de claves, fichero `.pem`, IP elástica, EC2, asociación de la IP y la subida de la web. Al arrancar, la EC2 instala nginx, actualiza la IP en DuckDNS, pide el certificado y fuerza HTTPS.

Archivos

Archivo	Qué es
`06-web-aws/versions.tf`	Versión de Terraform y proveedores (aws, tls, local)
`06-web-aws/variables.tf`	Variables (subdominio, token, correo, región, tipo de instancia)
`06-web-aws/main.tf`	Toda la infraestructura
`06-web-aws/outputs.tf`	IP, URL y comando SSH al terminar
`06-web-aws/user_data.sh.tftpl`	Script de arranque del servidor (nginx, DuckDNS, HTTPS)
`06-web-aws/terraform.tfvars.example`	Plantilla de datos personales (el `terraform.tfvars` real no se sube)
`06-web-aws/web`	La web: `index.html`, `estilos.css`, `geo.js` (geolocalización) e imágenes
`06-web-aws/docs/01-arquitectura.md`	Arquitectura en AWS
`06-web-aws/docs/02-guia-despliegue.md`	Guía de despliegue paso a paso
`06-web-aws/docs/03-explicacion-codigo.md`	Explicación del código de Terraform y del script de arranque
`06-web-aws/docs/04-pruebas.md`	Pruebas detalladas
`06-web-aws/docs/05-mantenimiento.md`	Mantenimiento: límites de Let’s Encrypt, recrear el servidor, quién despliega

Despliegue paso a paso

Guía completa: `06-web-aws/docs/02-guia-despliegue.md`. Resumen:

```
# Cada sesión del lab: credenciales nuevas (AWS Details → AWS CLI → Show)
nano ~/.aws/credentials

# Primera vez
cp terraform.tfvars.example terraform.tfvars && nano terraform.tfvars
terraform init

# Desplegar (tarda unos 5 minutos)
terraform plan       # Plan: 13 to add
terraform apply      # yes

# Prueba de despliegue reproducible
terraform destroy
terraform apply
```

Seguridad aplicada

- Solo HTTPS (HTTP redirige), certificado válido de Let's Encrypt con renovación automática.
- Grupo de seguridad con solo 22, 80 y 443; clave SSH generada por Terraform y guardada fuera del repositorio.
- `terraform.tfvars` (token de DuckDNS), `*.tfstate` y `*.pem` en `.gitignore`.

Pruebas

Detalladas en `06-web-aws/docs/04-pruebas.md`. Las capturas se hacen con la web desplegada desde la sesión del lab.

Nº	Qué se prueba	Resultado esperado	Resultado obtenido	Captura
1	`terraform plan`	13 to add	Pendiente — en clase	
2	`terraform apply`	Apply complete, url = https://almichain2.duckdns.org	Pendiente — en clase	
3	HTTPS con candado	Certificado de Let's Encrypt válido	Pendiente — en clase	
4	HTTP redirige a HTTPS	301 a https	Pendiente — en clase	
5	DNS	`nslookup almichain2.duckdns.org` = IP elástica	Pendiente — en clase	
6	SSL Labs	Nota A	Pendiente — en clase	
7	Geolocalización	El botón pide permiso y calcula la distancia al restaurante	Pendiente — en clase	
8	Vista móvil	Menú y secciones bien en 390 px	Pendiente — en clase	
9	Lighthouse	90 o más en las 4	**En local** (servidor de pruebas,	informe (`06-web-aws/pruebas/light`

Nº	Qué se prueba	Resultado esperado	Resultado obtenido	Captura
		categorías	`http://127.0.0.1`, móvil): rendimiento 92, accesibilidad 100, buenas prácticas 96, SEO 100. Sin HTTPS ni la red real, y con Google Fonts y el mapa bloqueados por el entorno. **Repetir con la web desplegada**	`house-local-movil.html`)
10	Destruir y volver a crear	La web vuelve igual	Pendiente — en clase	

Lighthouse en local

Antes de desplegar se pasó Lighthouse 12.2.1 a la carpeta `web/` servida en local (perfil móvil). Encontró dos problemas de **contraste** que se corrigieron: los precios de la carta (ámbar `#c8771e` sobre blanco, 3,4:1) pasan a `#9a5a12` (5,5:1) y el fondo verde del menú del día se oscurece para que el texto ámbar llegue a 4,8:1; además faltaba el icono de la página (`favicon`). Resultado: rendimiento 92, accesibilidad 100, buenas prácticas 96 y SEO 100. El informe completo está en `06-web-aws/pruebas/lighthouse-local-movil.html`. No sustituye a la prueba 9 con la web publicada (HTTPS, DuckDNS y la red real).

Problemas previstos y solución

La web todavía no se ha desplegado desde este repositorio, así que estos problemas son **previstos** (documentación de AWS Academy, Let's Encrypt y Terraform):

Problema	Solución
Las credenciales de AWS Academy caducan a las ~4 horas	Copiar las nuevas en `~/.aws/credentials` en cada sesión del lab (guía, paso 3 (`06-web-aws/docs/02-guia-despliegue.md`))
Let's Encrypt solo da 5 certificados por semana para el mismo dominio	Probar con `certbot_staging = true` y pasar a `false` solo para la versión final (mantenimiento (`06-web-aws/docs/05-mantenimiento.md`))
Al cambiar el script de arranque, la EC2 se paraba y arrancaba sin volver a ejecutarlo	`user_data_replace_on_change = true`: Terraform recrea la máquina
Abrir SSH a todo Internet	`ssh_cidr` obligatoria y sin `0.0.0.0/0`: solo la IP pública de quien despliega

Fuentes

- Terraform AWS Provider — https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- DuckDNS — https://www.duckdns.org/install.jsp
- Certbot — https://certbot.eff.org/
- API de geolocalización — https://developer.mozilla.org/es/docs/Web/API/Geolocation_API

7. Base de datos, RAID y copias de seguridad

Objetivo

Diseñar e implantar la base de datos de un local de Almi-Chain (clientes, reservas, mesas, comandas, carta, ingredientes, proveedores, personal y facturas) en un servidor **MariaDB** cuyos datos están en un **RAID 1**, con usuarios de mínimo privilegio, **copias automáticas** a otro servidor y **pruebas de restauración** y de **fallo de disco**.

Tareas del tablero: B01 a B12.

Requisitos

Elemento	Valor
VM	ALMI-BD · 2 GB de RAM · 2 núcleos · disco de sistema de 15 GB + **dos discos de 10 GB** para el RAID · Red NAT ALMICHAIN
Sistema	Ubuntu Server 24.04 LTS · IP 192.168.10.20/24
Software	MariaDB 10.11, mdadm, rsync, cron
Usuario de administración	`almiadmin` (contraseña en KeePassXC)

Arquitectura

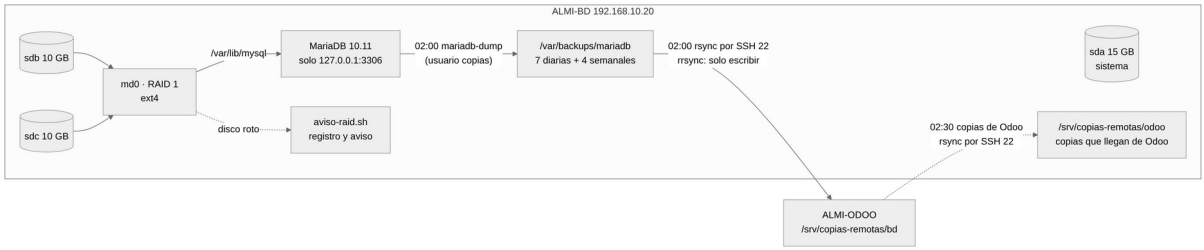


Figura 60. Diagrama

Modelo de datos

Modelo del equipo hecho en dbdiagram.io: `diagramas/modelo-er-equipo.pdf`. Es un **modelo relacional** (tablas, columnas y claves). El **modelo entidad/relación conceptual** está en `diagramas/modelo-er-conceptual.mmd` (fuente, se edita como texto) y su imagen: las tres tablas intermedias (`detalle_comanda`, `plato_ingrediente` y `suministro`) aparecen como relaciones N:M con sus atributos. Está hecho a partir del esquema; el equipo lo revisa y, si el profesor lo pide en notación de Chen (rombos), lo pasa a draw.io.

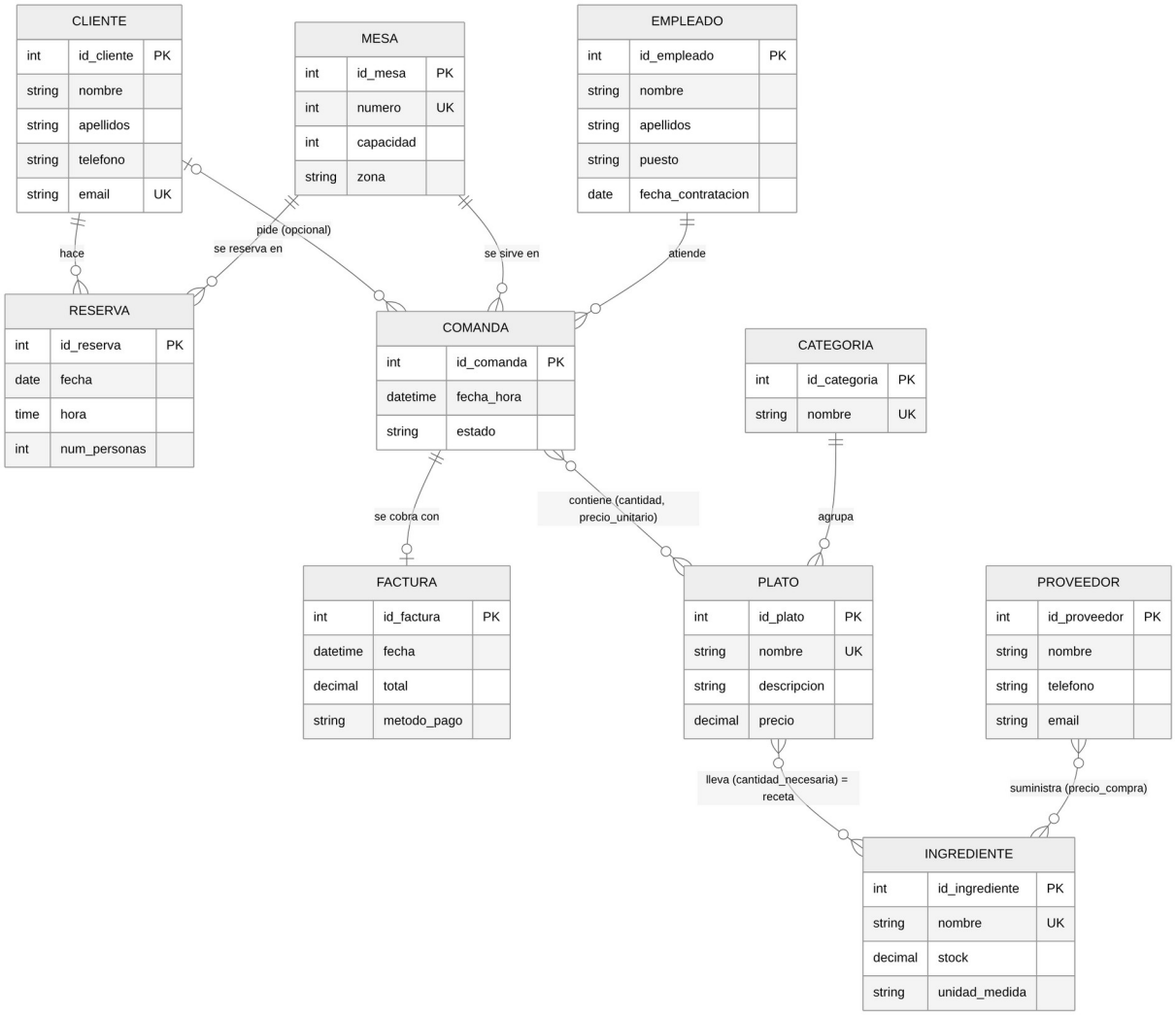


Figura 61. Modelo entidad/relación conceptual (notación de pata de gallo: | uno, o cero, pata varios)

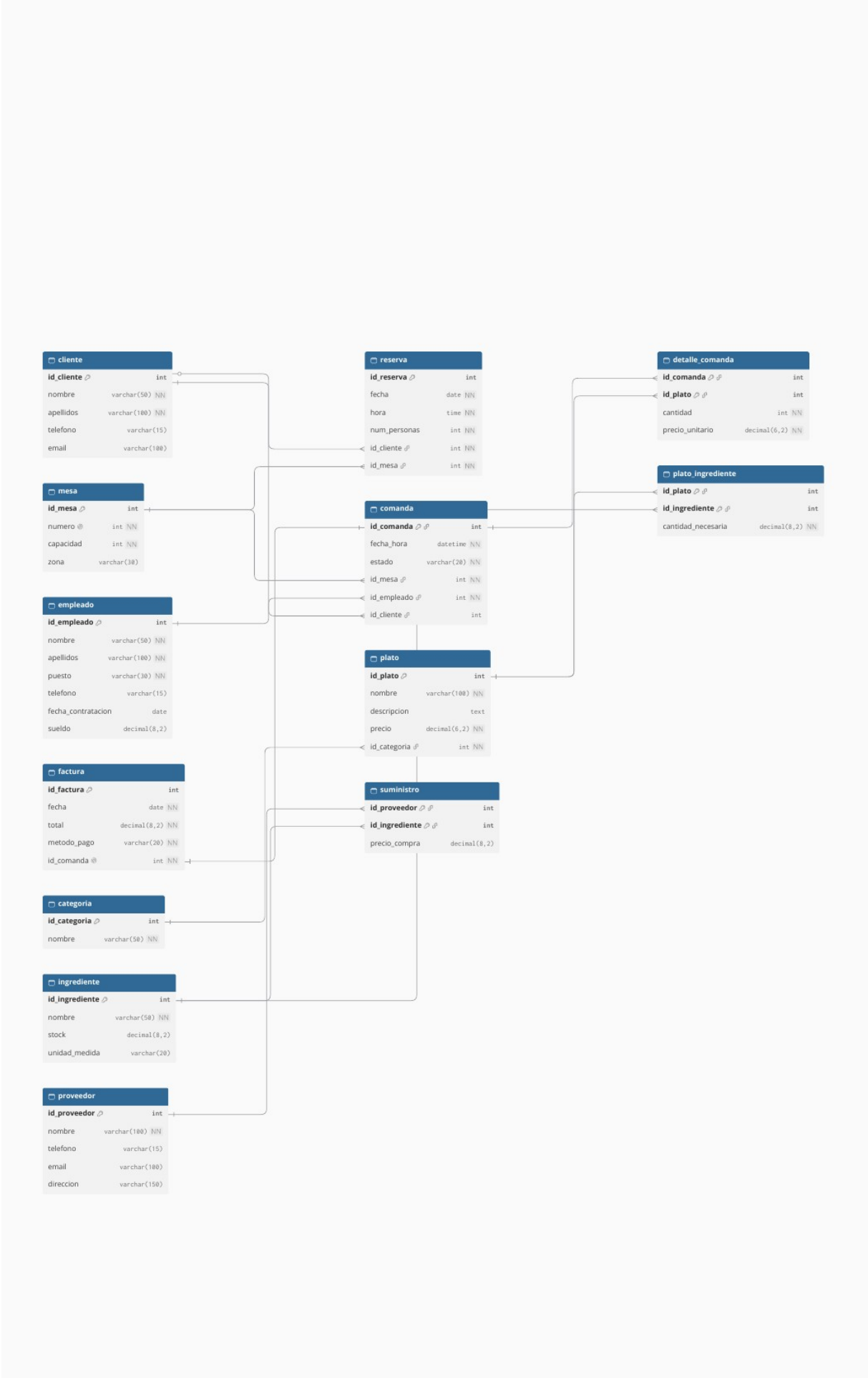

cliente
id_cliente int
nombre varchar(50) NN
apellidos varchar(100) NN
telefono varchar(15)
email varchar(100)
reserva
id_reserva int
fecha date NN
hora time NN
num_personas int NN
id_cliente int NN
id_mesa int NN
detalle_comanda
id_comanda int
id_plato int
cantidad int NN
precio_unitario decimal(6,2) NN
mesa
id_mesa int
numero int NN
capacidad int NN
zona varchar(30)
plato_ingrediente
id_plato int
id_ingrediente int
cantidad_necesaria decimal(8,2) NN
comanda
id_comanda int
fecha_hora datetime NN
estado varchar(20) NN
id_mesa int NN
id_empleado int NN
id_cliente int
empleado
id_empleado int
nombre varchar(50) NN
apellidos varchar(100) NN
puesto varchar(30) NN
telefono varchar(15)
fecha_contratacion date
sueldo decimal(8,2)
plato
id_plato int
nombre varchar(100) NN
descripcion text
precio decimal(6,2) NN
id_categoria int NN
factura
id_factura int
fecha date NN
total decimal(8,2) NN
metodo_pago varchar(20) NN
id_comanda int NN
suministro
id_proveedor int
id_ingrediente int
precio_compra decimal(8,2)
categoria
id_categoria int
nombre varchar(50) NN
ingrediente
id_ingrediente int
nombre varchar(50) NN
stock decimal(8,2)
unidad_medida varchar(20)
proveedor
id_proveedor int
nombre varchar(100) NN
telefono varchar(15)
email varchar(100)
direccion varchar(150)

Figura 62. Modelo relacional del equipo

13 tablas y una vista:

Tabla	Clave primaria	Claves ajenas	Notas
cliente	id_cliente	—	email único
mesa	id_mesa	—	número único; zona sala/terraza/barra
empleado	id_empleado	—	puesto, sueldo > 0
categoria	id_categoria	—	nombre único
plato	id_plato	id_categoria	precio > 0
ingrediente	id_ingrediente	—	stock ≥ 0
proveedor	id_proveedor	—	
reserva	id_reserva	id_cliente, id_mesa	una mesa no se reserva dos veces a la misma hora
comanda	id_comanda	id_mesa, id_empleado, id_cliente (opcional)	estado abierta/servida/pagada/anulada
detalle_comanda	(id_comanda, id_plato)	id_comanda, id_plato	N:M comanda-plato; guarda el precio del momento
plato_ingrediente	(id_plato, id_ingrediente)	id_plato, id_ingrediente	N:M, la receta
suministro	(id_proveedor, id_ingrediente)	id_proveedor, id_ingrediente	N:M, precio de compra
factura	id_factura	id_comanda (única)	1:1 con la comanda
v_importe_comanda (vista)			importe de cada comanda

Normalización: todas las columnas son atómicas (1FN); las tablas con clave compuesta (detalle_comanda, plato_ingrediente, suministro) solo tienen atributos que dependen de la clave entera (2FN); no hay dependencias entre atributos que no son clave, por ejemplo el nombre de la categoría está en `categoria` y no repetido en `plato` (3FN). El importe de la comanda no se guarda: se calcula con la vista, para que no pueda quedar desactualizado.

Archivos SQL en `07-bd-raid-copias/sql`:

Archivo	Qué hace
`07-bd-raid-copias/sql/01-esquema.sql`	Base `almichain` en utf8mb4, 13 tablas InnoDB con claves, restricciones CHECK, índices y la vista
`07-bd-raid-copias/sql/02-datos.sql`	Datos de ejemplo ficticios
`07-bd-raid-copias/sql/03-usuarios.sql`	Los 3 usuarios (las contraseñas se pasan al ejecutarlo, no están en el archivo)
`07-bd-raid-copias/sql/04-consultas.sql`	9 consultas de prueba
`07-bd-raid-copias/sql/05-recuentos.sql`	Recuento de filas para comparar antes y después de restaurar

Instalación y configuración paso a paso

1. VM con tres discos (B04)

Instalación base de Ubuntu Server igual que ALMI-ODOO (bloque 05, paso 1) con IP **192.168.10.20** y nombre `almi-bd`, **salvo el cortafuegos**: este servidor no tiene web, así que solo se abre el SSH y solo desde las VLAN 10 y 99:

```
sudo ufw default deny incoming && sudo ufw default allow outgoing
for red in 192.168.10.0/24 192.168.99.0/24; do sudo ufw allow from $red to any port 22 proto
tcp; done
sudo ufw enable
```

Con la VM apagada: **Configuración → Almacenamiento → Controlador SATA → Añadir disco duro → Crear** → `ALMI-BD-raid1`, VDI, 10 GB. Repetir con `ALMI-BD-raid2`.

2. MariaDB segura (B06)

```
sudo apt install -y mariadb-server
sudo mariadb-secure-installation
```

Se quitan los usuarios anónimos, la base `test` y el acceso remoto de root; root solo entra por el socket local (`unix_socket`). MariaDB escucha solo en `127.0.0.1`.

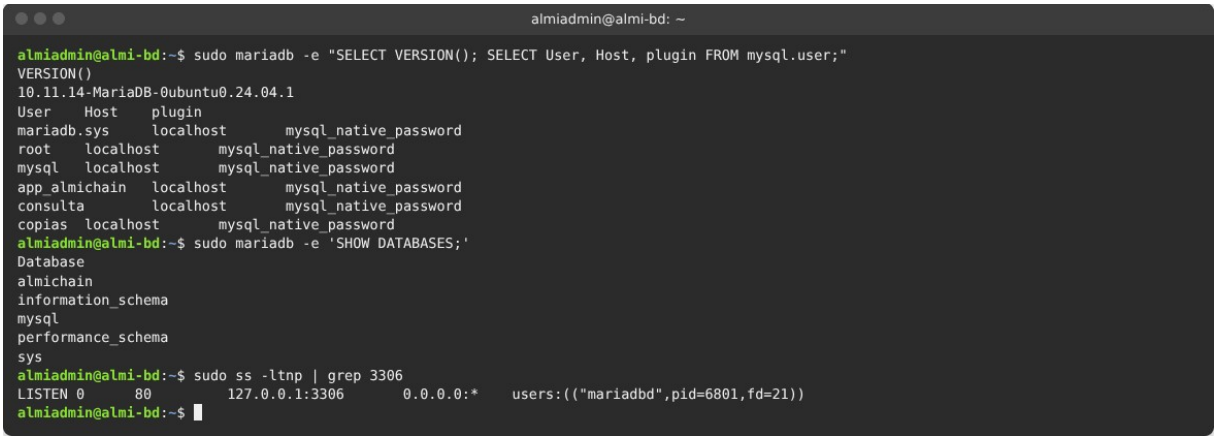


Figura 63. MariaDB sin usuarios anónimos ni base test, escuchando solo en 127.0.0.1

3. RAID 1 (B05)

Se hace **después** de instalar MariaDB, porque lo que se pone encima del RAID es su carpeta de datos (`/var/lib/mysql`). Los pasos están también en el script `07-bd-raid-copias/scripts/crear-raid.sh`, que comprueba que los discos están vacíos antes de tocar nada y que la copia es idéntica antes de vaciar la carpeta original:

```
sudo apt install -y mdadm
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc
cat /proc/mdstat                          # esperar a [UU] y 100 %
sudo mkfs.ext4 -L datos-bd /dev/md0
sudo systemctl stop mariadb
sudo mount /dev/md0 /mnt && sudo rsync -aAX /var/lib/mysql/ /mnt/ && sudo rm -rf
/mnt/lost+found
sudo diff -r --brief /var/lib/mysql/ /mnt/ && sudo umount /mnt   # sin salida = copia
idéntica
sudo find /var/lib/mysql -mindepth 1 -delete                     # vaciar el original
echo "UUID=$(sudo blkid -s UUID -o value /dev/md0) /var/lib/mysql ext4 defaults,nofail 0 2"
| sudo tee -a /etc/fstab
sudo mkdir -p /etc/systemd/system/mariadb.service.d
printf '[Unit]\nRequiresMountsFor=/var/lib/mysql\n' | sudo tee
/etc/systemd/system/mariadb.service.d/raid.conf
sudo systemctl daemon-reload
sudo mount /var/lib/mysql && sudo chown mysql:mysql /var/lib/mysql
```

```
sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
sudo update-initramfs -u
sudo systemctl start mariadb
```

> **Mejor con el script.** Los comandos de arriba hacen el RAID, pero **no** dos cosas que sí hace `07-bd-raid-copias/scripts/crear-raid.sh` (apartados 7 y 8): el **aviso** cuando falla un disco y el servicio que **arranca el RAID con un solo disco** si falta el otro. Por eso, en un servidor nuevo con los dos discos vacíos, lo recomendado es `sudo ./scripts/crear-raid.sh /dev/sdb /dev/sdc`, que además comprueba que los discos están vacíos.

- `nofail`: si el RAID fallara del todo, el servidor arranca igualmente y se puede reparar por SSH.
- `RequiresMountsFor`: pero **MariaDB no arranca** si el RAID no está montado. Por eso se vacía la carpeta original: si quedaran los datos viejos debajo del montaje, MariaDB arrancaría con ellos sin avisar y el restaurante trabajaría sobre datos antiguos.

Aviso si falla un disco: `mdadm` vigila el RAID (servicio `mdmonitor`) y, cuando un disco falla, llama al programa de la línea `PROGRAM` de `/etc/mdadm/mdadm.conf`. El script crea `/usr/local/sbin/aviso-raid.sh` (y reinicia `mdmonitor` para que lo use), que lo apunta en `/var/log/aviso-raid.log`, en el registro del sistema (`journalctl -t aviso-raid`) y en la pantalla de quien esté conectado, como mucho un minuto después del fallo (`mdmonitor` revisa el RAID cada 60 segundos). Sin esto, un disco roto podría pasar semanas sin que nadie se enterara y el RAID dejaría de proteger. El disco del sistema (el de 15 GB) **no** está en el RAID: si falla, se reinstala el servidor y se recupera la base desde el RAID o desde la copia en ALMI-ODOO.

> En la VM exportada (.ova) el RAID se crea solo la primera vez que arranca, con el servicio `almi-crear-raid`, porque los dos discos se entregan vacíos. El registro queda en `/var/log/almi-crear-raid.log`.

4. Esquema, datos y usuarios (B07)

```
cd ~/sql
sudo mariadb < 01-esquema.sql
sudo mariadb < 02-datos.sql
sudo ~/scripts/crear-usuarios-bd.sh 03-usuarios.sql
```

`07-bd-raid-copias/scripts/crear-usuarios-bd.sh` pide las tres contraseñas sin enseñarlas (mínimo 12 caracteres) y se las pasa a MariaDB por la entrada estándar, así no quedan en el historial de bash ni se ven con `ps`. `07-bd-raid-copias/sql/03-usuarios.sql` se para antes de tocar nada si falta alguna contraseña (probado: con `mariadb < 03-usuarios.sql` sin contraseñas sale con código 1 y los usuarios no cambian), usa `QUOTE()` (una comilla en la contraseña no rompe nada) y `CREATE OR REPLACE USER` (sirve también para cambiarlas). El script deja la contraseña de `copias` en `/root/.my.cnf` (600) para las copias automáticas.

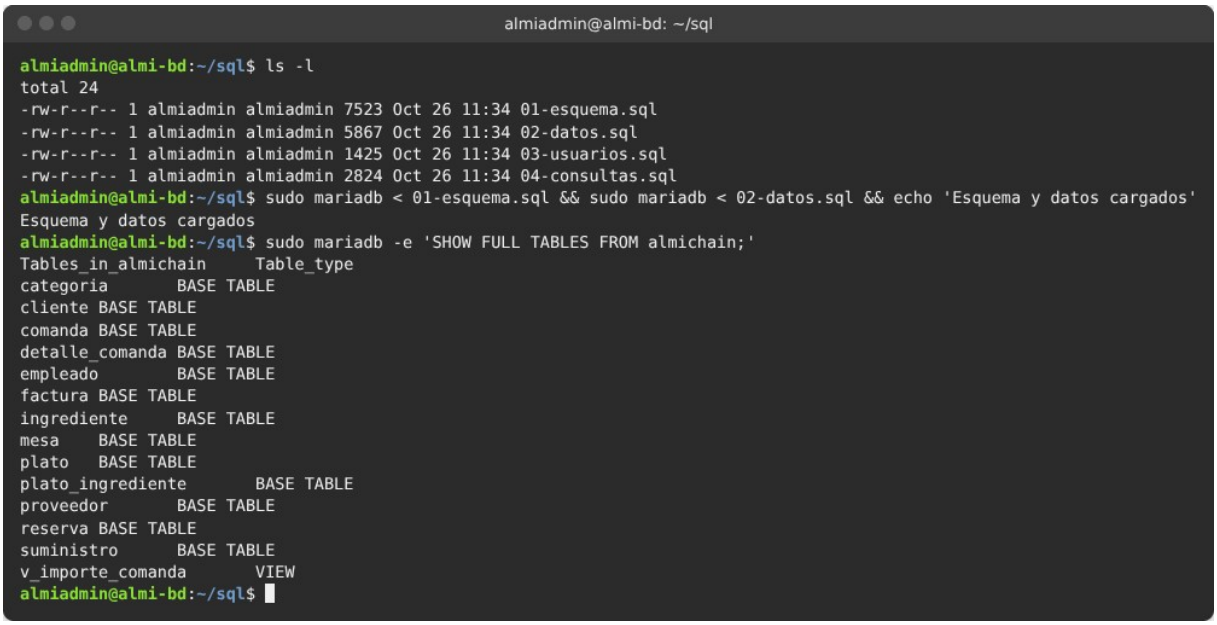


Figura 64. Esquema y datos cargados: 13 tablas y la vista

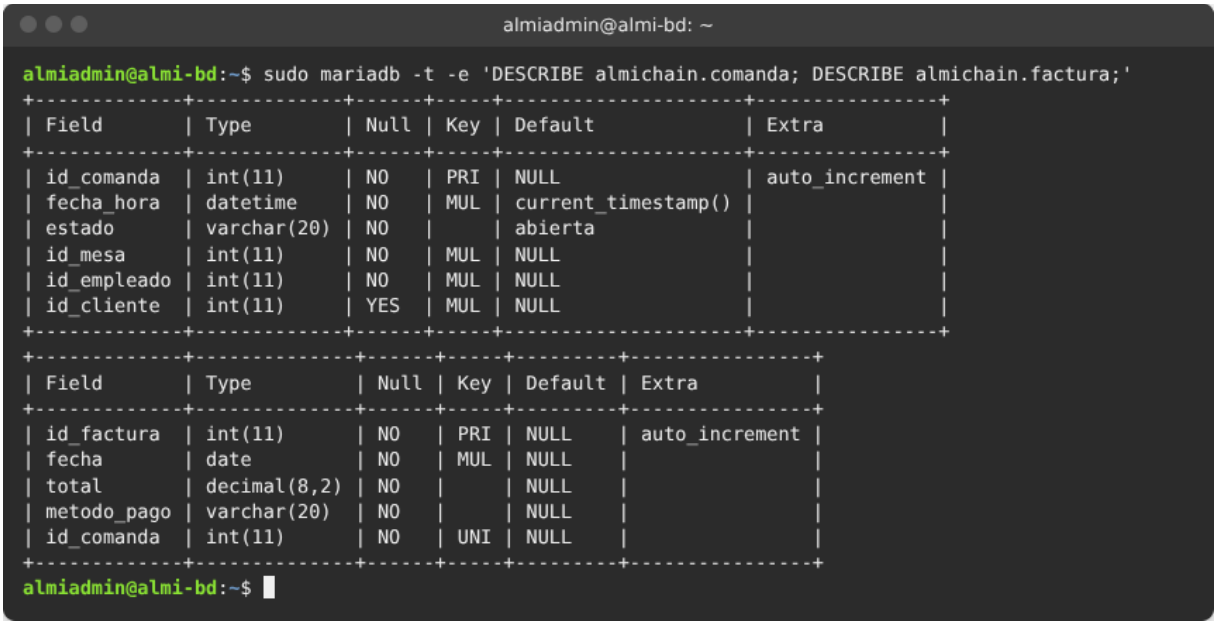


Figura 65. Estructura de comanda y factura con sus claves

Usuario	Permisos	Para qué
`app_almichain`	SELECT, INSERT, UPDATE, DELETE en `almichain.*`	La aplicación: trabaja con los datos, no puede cambiar la estructura
`consulta`	SELECT, SHOW VIEW en `almichain.*`	Informes de gerencia: solo lectura
`copias`	SELECT, SHOW VIEW, TRIGGER, LOCK TABLES, EVENT en `almichain.*` y SELECT en `mysql.proc`	Lo mínimo que necesita `mariadb-dump` (con `--routines`). No puede leer las demás bases ni `mysql.global_priv`, donde están los hashes de las contraseñas

Todos solo desde `localhost`.

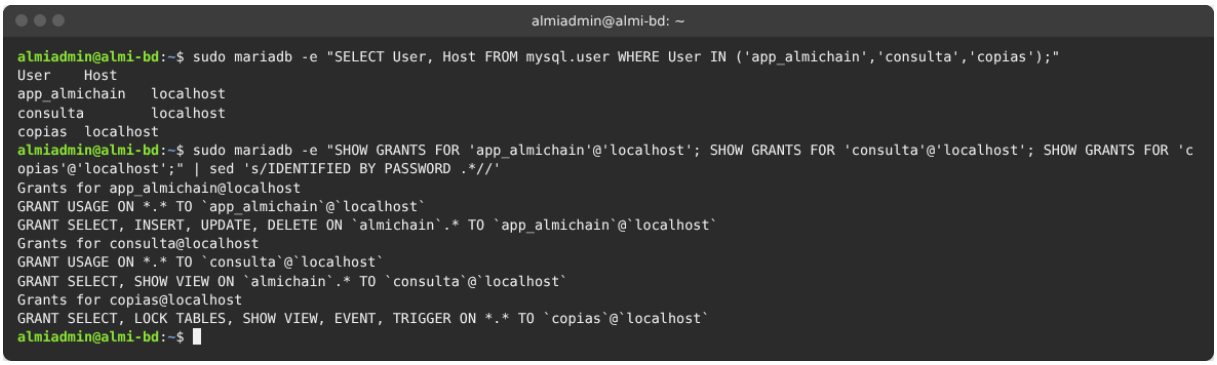


Figura 66. Usuarios y sus privilegios (servidor de pruebas, antes de limitar copias a almichain.; el estado actual está en la captura 19)*

5. Consultas (B07)

Las 9 consultas de `07-bd-raid-copias/sql/04-consultas.sql`, ejecutadas con el usuario de solo lectura `consulta`:

almiadmin@almi-bd: ~

```
almiadmin@almi-bd:~$ mariadb --defaults-extra-file=~/.consulta.cnf -t -vv < ~/sql/04-consultas.sql | sed -n '1,59p'
--------------
SELECT c.nombre AS categoria, p.nombre AS plato, p.precio
FROM plato p JOIN categoria c ON c.id_categoria = p.id_categoria
ORDER BY c.id_categoria, p.precio
--------------

+-------------+----------------------+--------+
| categoria   | plato                | precio |
+-------------+----------------------+--------+
| Bebidas     | Café con leche       |   1.70 |
| Bebidas     | Agua mineral 50 cl   |   1.80 |
| Bebidas     | Caña de cerveza      |   2.20 |
| Bebidas     | Refresco de cola     |   2.50 |
| Entrantes   | Croquetas caseras (6) |   7.50 |
| Entrantes   | Ensalada de la casa  |   8.90 |
| Principales | Pasta boloñesa       |  10.50 |
| Principales | Hamburguesa Almi     |  12.50 |
| Principales | Bacalao al pil-pil   |  16.90 |
| Postres     | Tarta de queso       |   5.20 |
| Postres     | Goxua                |   5.50 |
+-------------+----------------------+--------+
11 rows in set

--------------
SELECT r.hora, CONCAT(cl.nombre, ' ', cl.apellidos) AS cliente,
       r.num_personas, m.numero AS mesa, m.zona
FROM reserva r
JOIN cliente cl ON cl.id_cliente = r.id_cliente
JOIN mesa m     ON m.id_mesa = r.id_mesa
WHERE r.fecha = '2026-10-24'
ORDER BY r.hora
--------------

+----------+--------------------------+--------------+------+------+
| hora     | cliente                  | num_personas | mesa | zona |
+----------+--------------------------+--------------+------+------+
| 14:00:00 | Ane Etxeberria Goikoetxea |            4 |    2 | sala |
| 14:30:00 | Jon Arrieta Uriarte      |            2 |    1 | sala |
| 21:00:00 | Lucía Martín Gómez       |            6 |    4 | sala |
+----------+--------------------------+--------------+------+------+
3 rows in set

--------------
SELECT * FROM v_importe_comanda ORDER BY id_comanda
--------------

+------------+---------------------+------+---------------------+---------+---------+
| id_comanda | fecha_hora          | mesa | camarero            | estado  | importe |
+------------+---------------------+------+---------------------+---------+---------+
|          1 | 2026-10-24 14:05:00 |    2 | Nerea Olabarria Sanz | pagada  |   47.30 |
|          2 | 2026-10-24 14:35:00 |    1 | Asier Landa Mendieta | pagada  |   42.90 |
|          3 | 2026-10-24 21:10:00 |    4 | Nerea Olabarria Sanz | pagada  |  100.90 |
|          4 | 2026-10-25 09:15:00 |    8 | Asier Landa Mendieta | pagada  |    1.70 |
|          5 | 2026-10-25 14:10:00 |    6 | Nerea Olabarria Sanz | pagada  |  103.40 |
|          6 | 2026-10-25 21:40:00 |    5 | Asier Landa Mendieta | servida |   22.80 |
|          7 | 2026-10-26 10:00:00 |    9 | Nerea Olabarria Sanz | abierta |    3.40 |
+------------+---------------------+------+---------------------+---------+---------+
7 rows in set
almiadmin@almi-bd:~$
```

Figura 67. Consultas 1 a 3: carta, reservas del día e importe de cada comanda

```
almiadmin@almi-bd: ~
almiadmin@almi-bd:~$ mariadb --defaults-extra-file=~/.consulta.cnf -t -vv < ~/sql/04-consultas.sql | sed -n '60,116p'
--------------
SELECT fecha, metodo_pago, COUNT(*) AS facturas, SUM(total) AS total
FROM factura
GROUP BY fecha, metodo_pago WITH ROLLUP
--------------

+------------+-------------+----------+--------+
| fecha      | metodo_pago | facturas | total  |
+------------+-------------+----------+--------+
| 2026-10-24 | efectivo    |        1 |  42.90 |
| 2026-10-24 | tarjeta     |        2 | 148.20 |
| 2026-10-24 | NULL        |        3 | 191.10 |
| 2026-10-25 | efectivo    |        1 |   1.70 |
| 2026-10-25 | tarjeta     |        1 | 103.40 |
| 2026-10-25 | NULL        |        2 | 105.10 |
| NULL       | NULL        |        5 | 296.20 |
+------------+-------------+----------+--------+
7 rows in set

--------------
SELECT p.nombre, SUM(d.cantidad) AS unidades,
       SUM(d.cantidad * d.precio_unitario) AS ingresos
FROM detalle_comanda d JOIN plato p ON p.id_plato = d.id_plato
GROUP BY p.id_plato, p.nombre
ORDER BY unidades DESC
LIMIT 5
--------------

+-------------------+----------+----------+
| nombre            | unidades | ingresos |
+-------------------+----------+----------+
| Caña de cerveza   |        9 |    19.80 |
| Hamburguesa Almi  |        7 |    87.50 |
| Tarta de queso    |        6 |    31.20 |
| Pasta boloñesa    |        5 |    52.50 |
| Bacalao al pil-pil |        4 |    67.60 |
+-------------------+----------+----------+
5 rows in set

--------------
SELECT CONCAT(e.nombre, ' ', e.apellidos) AS camarero,
       COUNT(DISTINCT v.id_comanda) AS comandas, SUM(v.importe) AS vendido
FROM v_importe_comanda v
JOIN comanda c  ON c.id_comanda = v.id_comanda
JOIN empleado e ON e.id_empleado = c.id_empleado
GROUP BY e.id_empleado
ORDER BY vendido DESC
--------------

+---------------------+----------+---------+
| camarero            | comandas | vendido |
+---------------------+----------+---------+
| Nerea Olabarria Sanz |        4 |  255.00 |
| Asier Landa Mendieta |        3 |   67.40 |
+---------------------+----------+---------+
2 rows in set
almiadmin@almi-bd:~$
```

Figura 68. Consultas 4 a 6: facturación por día y método de pago, platos más vendidos y ventas por camarero

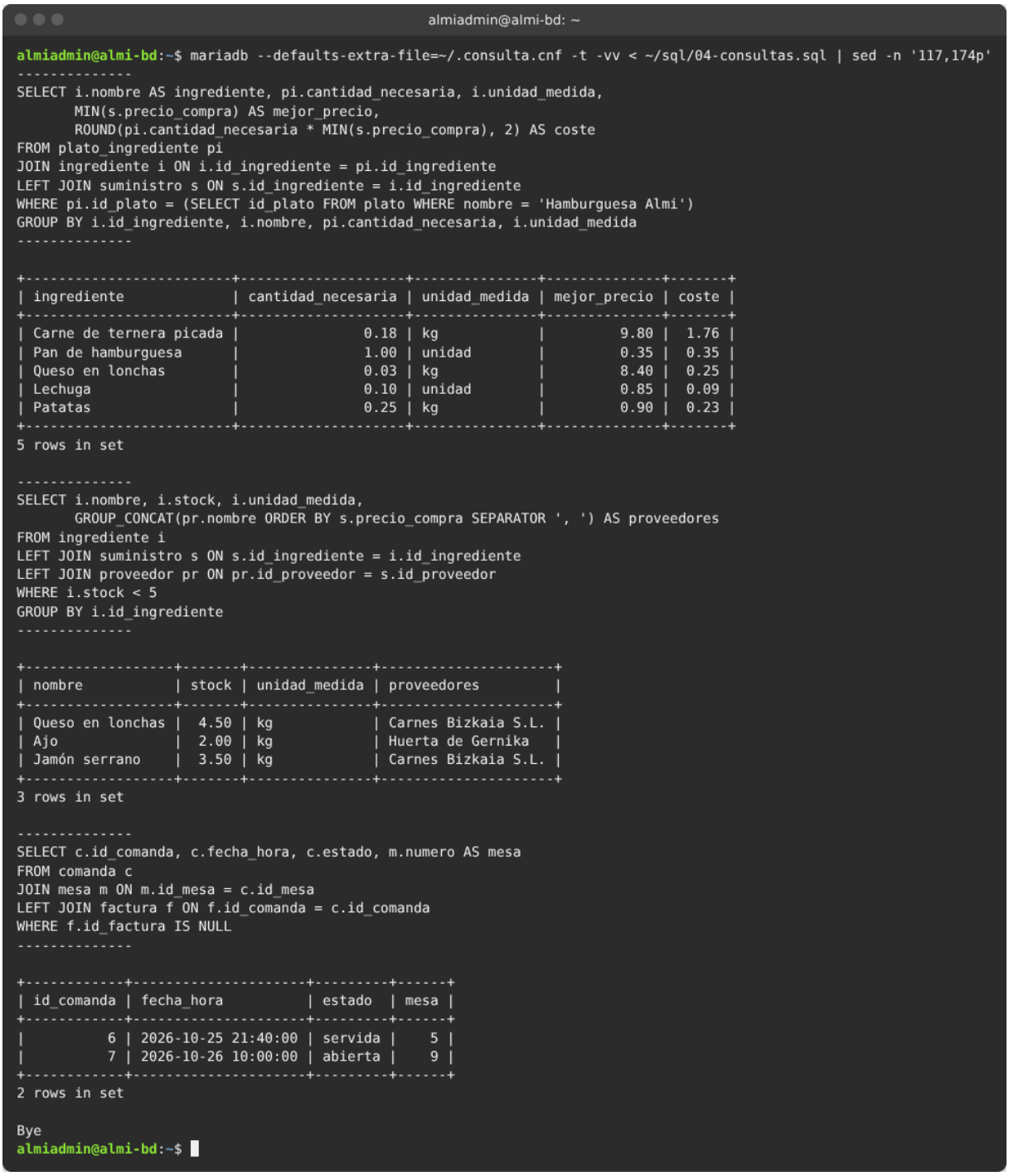


Figura 69. Consultas 7 a 9: coste de la receta, ingredientes con poco stock y comandas sin cobrar

El usuario `consulta` no puede borrar ni modificar:



Figura 70. consulta no puede hacer DELETE ni DROP: ERROR 1142

6. Plan de copias (B08)

Qué	Cómo	Cuándo	Retención	Dónde
Base `almichain` (MariaDB)	`mariadb-dump --single-transaction` comprimido + huella SHA-256	Diaria, 02:00	7 diarias + 4 semanales (domingo)	ALMI-BD `/var/backups/mariadb` y **copia en ALMI-ODOO**
Base y adjuntos de Odoo	`pg_dump -Fc` + `tar` del filestore + huellas	Diaria, 02:30	7 días	ALMI-ODOO `/var/backups/odoo` y **copia en ALMI-BD**
Configuración y scripts	Repositorio Git	En cada cambio	Historial completo	GitHub

Regla 3-2-1: 3 copias (datos en el RAID + copia local + copia en el otro servidor), en 2 soportes distintos (dos máquinas) y 1 fuera del local (el repositorio para la configuración; para los datos, se recomienda al restaurante un disco externo semanal o un almacenamiento en la nube cifrado).

7. Copias cruzadas automáticas (B09)

En **los dos** servidores:

- Usuario `copias` sin contraseña (`passwd -l`) que solo recibe archivos en `/srv/copias-remotas/` (permisos 700).
- Clave SSH `/root/.ssh/copias_ed25519` para enviar. En el `authorized_keys` del otro servidor la clave va limitada a la IP de origen, sin terminal y **solo para escribir en su carpeta** con `rrsync` (viene con rsync en Ubuntu 24.04):

  ```
  command="/usr/bin/rrsync -wo -no-del
  /srv/copias-remotas/bd/",from="192.168.10.20",restrict ssh-ed25519 …
  ```

  Así, aunque alguien se hiciera con un servidor, desde él **no puede borrar ni leer las copias** que hay en el otro, ni ejecutar órdenes allí. Probado entre las dos VM: la copia normal funciona; `ssh … 'ls /'`, `rsync --delete`, leer y `../` se rechazan (captura 41 de Odoo (`05-erp-odoo/capturas/41-vm-clave-copias-restringida.png`) y 26 (`07-bd-raid-copias/capturas/26-vm-clave-copias-en-almi-bd.png`), con la línea de `authorized_keys` de ALMI-BD).
- A mano (lo mismo que hace `herramientas/construir-vm/lib.sh`). Los archivos del repositorio se llevan a la VM con `git clone` o `scp -r`. En **ALMI-ODOO**, que envía a ALMI-BD:

```
sudo ssh-keygen -t ed25519 -N '' -C copias-odoo -f /root/.ssh/copias_ed25519
sudo cat /root/.ssh/copias_ed25519.pub          # se copia para el otro servidor
sudo install -m 700 05-erp-odoo/scripts/copia-odoo.sh /usr/local/bin/copia-odoo.sh
```

En **ALMI-BD**, que la recibe:

```
sudo useradd -m -s /bin/bash -c 'Copias de seguridad' copias && sudo passwd -l copias
sudo install -d -o copias -g copias -m 700 /srv/copias-remotas
/srv/copias-remotas/odoo /home/copias/.ssh
echo 'command="/usr/bin/rrsync -wo -no-del
/srv/copias-remotas/odoo/",from="192.168.10.30",restrict ssh-ed25519 AAAA…pegar…' \
  | sudo tee /home/copias/.ssh/authorized_keys
sudo chown copias:copias /home/copias/.ssh/authorized_keys && sudo chmod 600
/home/copias/.ssh/authorized_keys
```

Y lo mismo al revés (clave en ALMI-BD, `copia-bd.sh`, carpeta `bd` y `from="192.168.10.20"` en ALMI-ODOO). En el cron de root de cada uno (`sudo crontab -e`): `0 2 * * * /usr/local/bin/copia-bd.sh` (o `30 2 … copia-odoo.sh`) y la limpieza de abajo. La primera copia se lanza a mano (`sudo /usr/local/bin/copia-bd.sh`) para que `ssh` guarde la clave de host del otro.

- El que recibe las copias es quien borra las viejas: en el cron de root, `find /srv/copias-remotas -type f -mtime +30 -delete` (30 días).
- La primera vez que un servidor envía, guarda la clave de host del otro (`StrictHostKeyChecking=accept-new`); si después cambiara, `ssh` se niega a conectar.
- Contraseña del usuario `copias` de MariaDB en `/root/.my.cnf` (permisos 600).

Script `07-bd-raid-copias/scripts/copia-bd.sh` en `/usr/local/bin/`, en el cron de root a las 02:00:

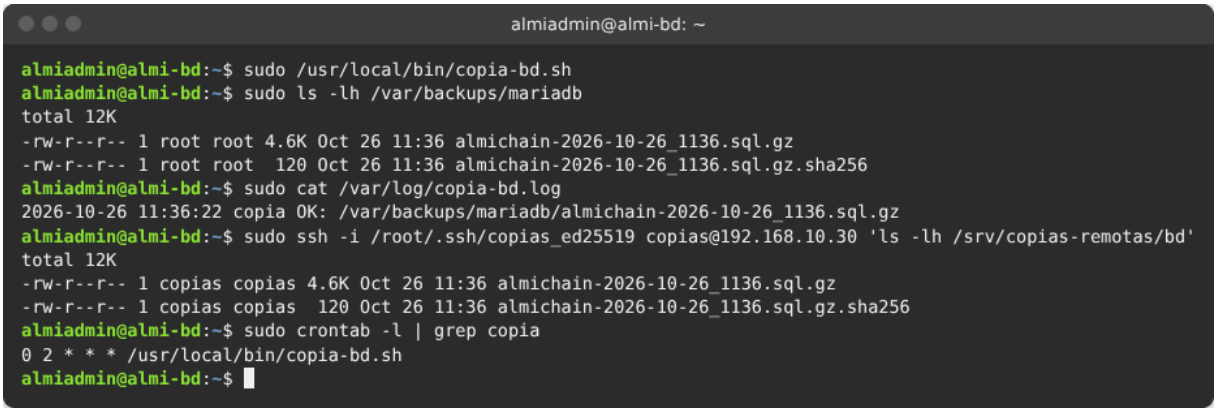


Figura 71. Copia hecha, comprobada, enviada y programada en cron (servidor de pruebas, antes de restringir la clave con rrsync: el `ls` remoto que se ve ahora se rechaza; ver las capturas 24 y 25)

Pruebas de restauración (B10)

Se simula un desastre: se borra la base de datos entera y se recupera de la copia.

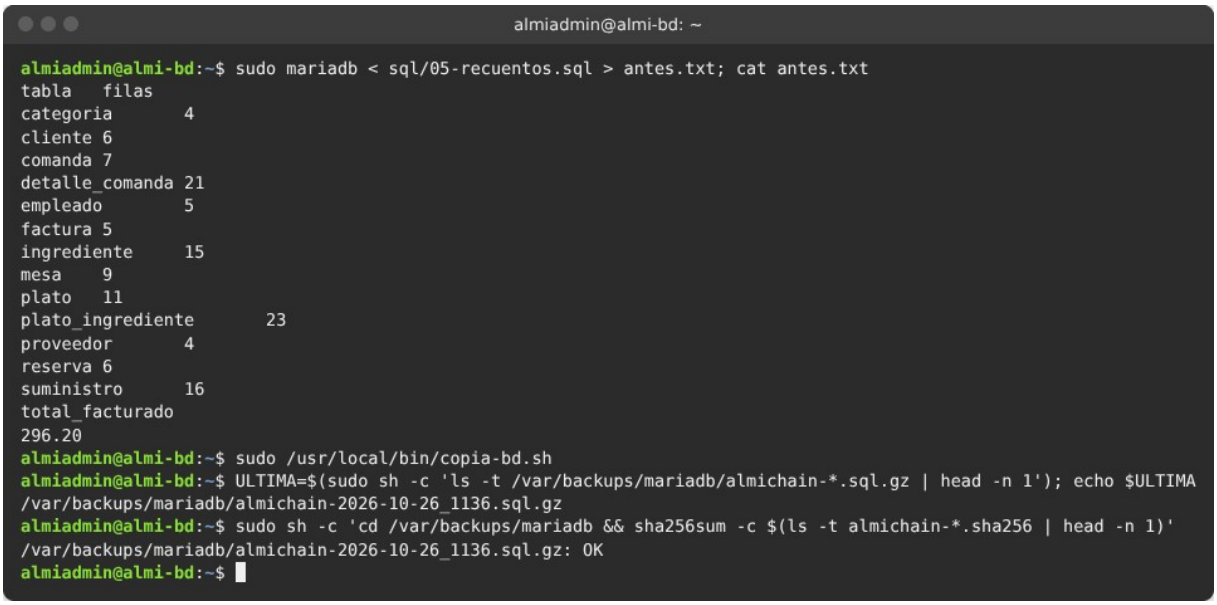


Figura 72. 1. Recuento antes, copia nueva y huella correcta



Figura 73. 2. Se borra la base de datos



Figura 74. 3. Restauración en menos de un segundo, datos idénticos y los usuarios siguen funcionando

Tiempo de recuperación (RTO): menos de 1 segundo con esta cantidad de datos. **Pérdida máxima de datos (RPO)**: 24 horas (copia diaria).

La VM exportada y la prueba de fallo de disco (B11)

Hecho arrancando los discos de `ALMI-BD.ova` (el de sistema y los dos de 10 GB) **en QEMU**, no en VirtualBox, con `herramientas/probar-vm/pruebas-bd.sh` (`herramientas/probar-vm/pruebas-bd.sh`). Cada captura tiene al lado su transcripción `.txt` con las órdenes y la salida real.

Primer arranque. `almi-crear-raid` crea el RAID 1 con los dos discos vacíos, copia encima los datos de MariaDB, comprueba la copia, **vacía la carpeta original** (debajo del montaje quedan 0 archivos)

y deja MariaDB dependiendo del montaje. Después, `almi-primer-arranque-bd` crea los tres usuarios con contraseñas nuevas, solo para esta VM, y pone el cortafuegos.

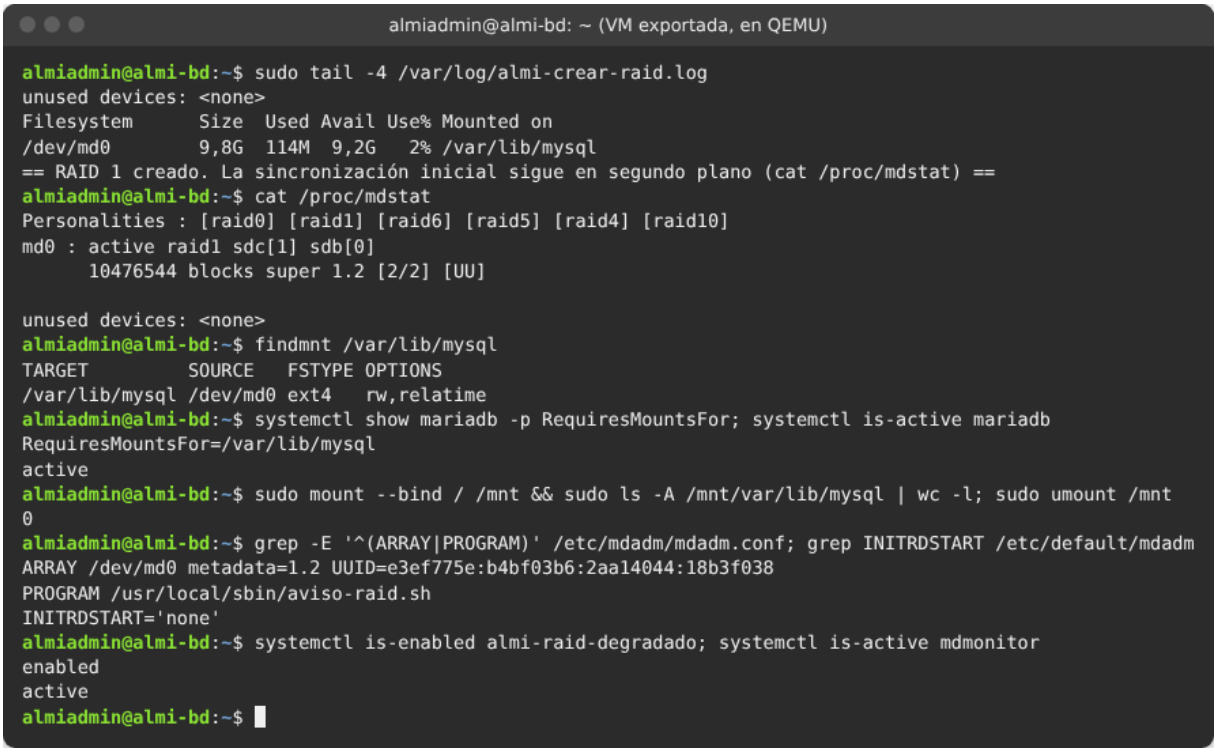


Figura 75. Primer arranque: RAID 1 en UU montado en /var/lib/mysql, MariaDB requiere el montaje, carpeta original vacía, aviso configurado

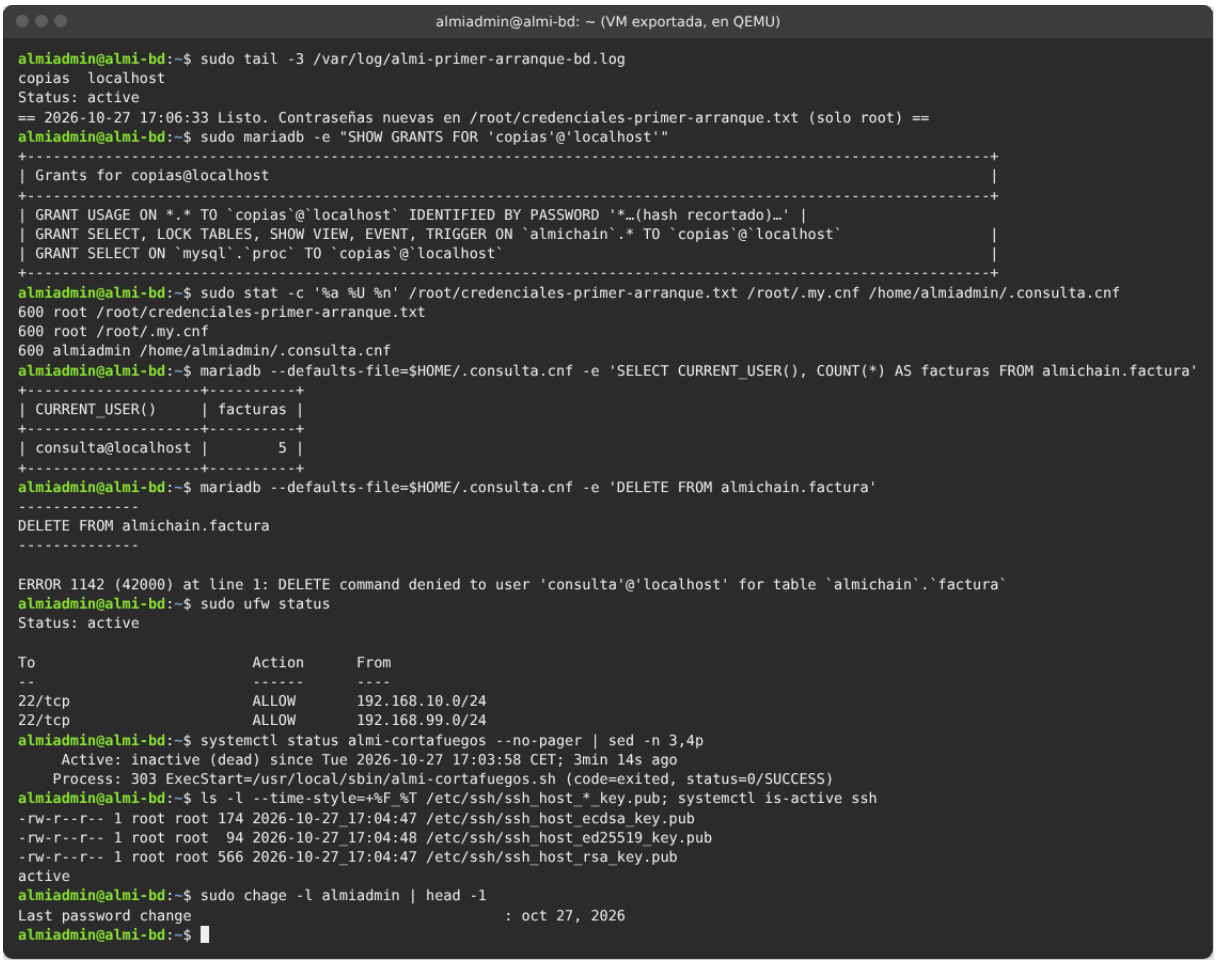


Figura 76. Usuarios con contraseñas nuevas: copias solo sobre almichain y mysql.proc, secretos solo para root, consulta no puede borrar, cortafuegos solo SSH desde las VLAN 10 y 99, claves SSH propias

Prueba A, un disco se rompe con el servidor en marcha. Se marca `sdc` como roto: el RAID pasa a `[U_]` y la base de datos **sigue funcionando y se puede escribir**. Antes de un minuto llega el **aviso** (registro y mensaje en pantalla).

```
sudo mdadm /dev/md0 --fail /dev/sdc
cat /proc/mdstat
sudo mariadb -e "INSERT INTO almichain.cliente (nombre, apellidos, telefono) VALUES
('Prueba', 'RAID', '699999999');"
sudo tail /var/log/aviso-raid.log
```

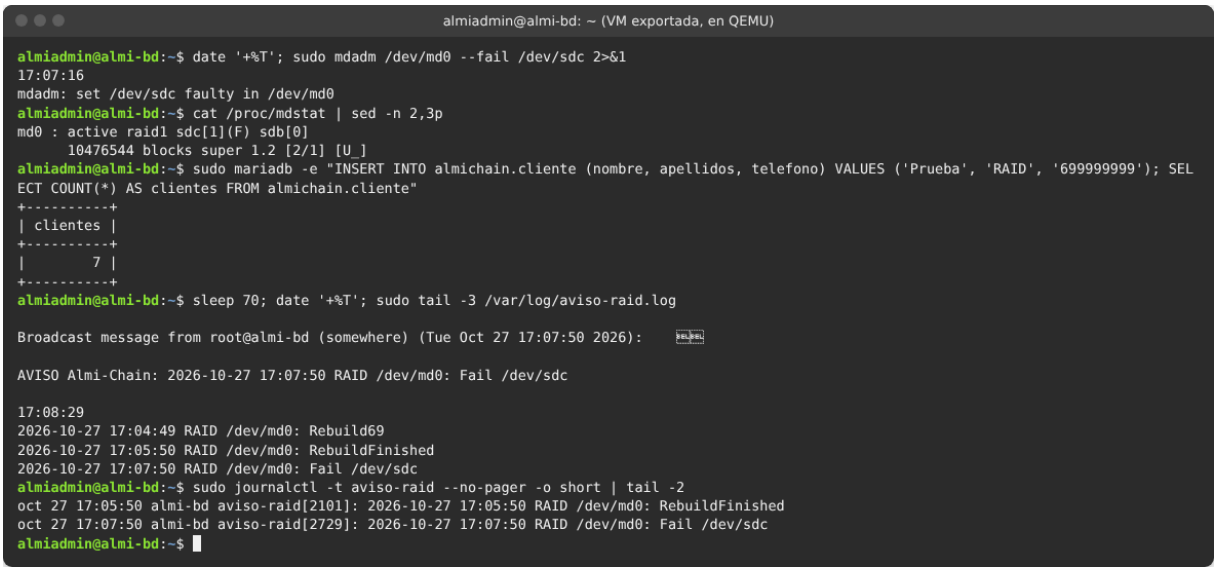


Figura 77. Disco sdc roto a las 17:07:16: U_, MariaDB sigue escribiendo y el aviso llega a las 17:07:50 (34 s) al registro propio, al del sistema y a la pantalla

Cambio del disco y reconstrucción: se quita del RAID y se vuelve a añadir; mdadm copia los datos del disco bueno al otro.

```
sudo mdadm /dev/md0 --remove /dev/sdc
sudo mdadm /dev/md0 --add /dev/sdc
watch cat /proc/mdstat
```

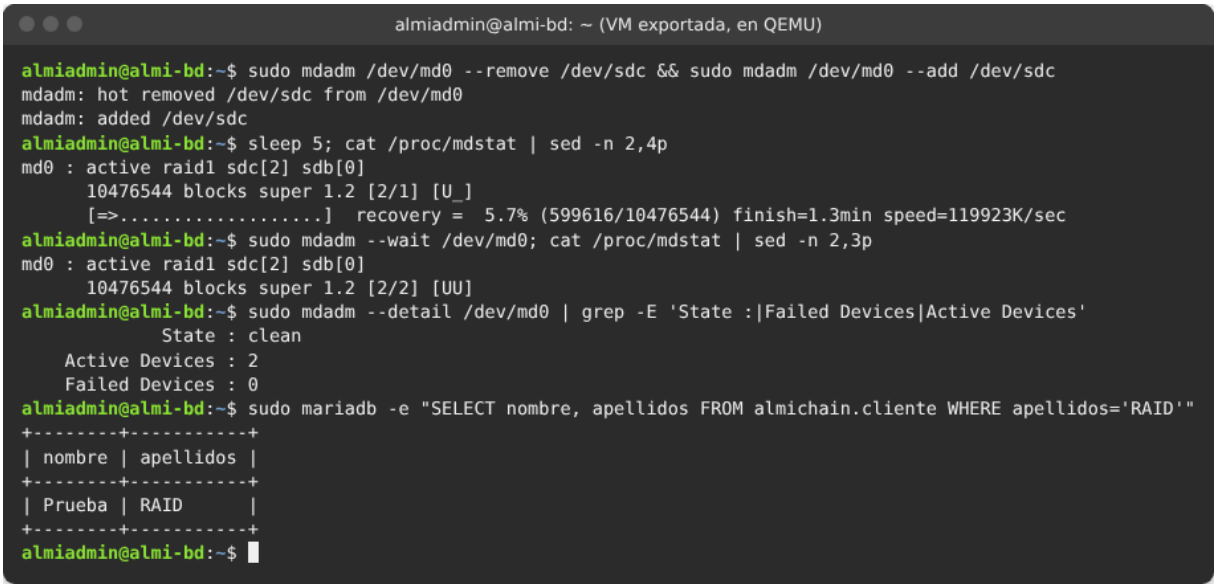


Figura 78. Reconstrucción hasta UU, 0 discos fallidos y el dato escrito con un solo disco sigue ahí

Prueba B, arrancar sin un disco. Con la VM apagada se quita el segundo disco y se arranca: el RAID arranca **degradado** con un solo disco, se monta y MariaDB funciona con los datos. Ubuntu no siempre lo arranca solo cuando falta un disco (se quedaba `inactive`); por eso hay un servicio, `almi-raid-degradado`, que espera 20 s y, si sigue inactivo, lo arranca con el disco que haya, monta los datos y arranca MariaDB.

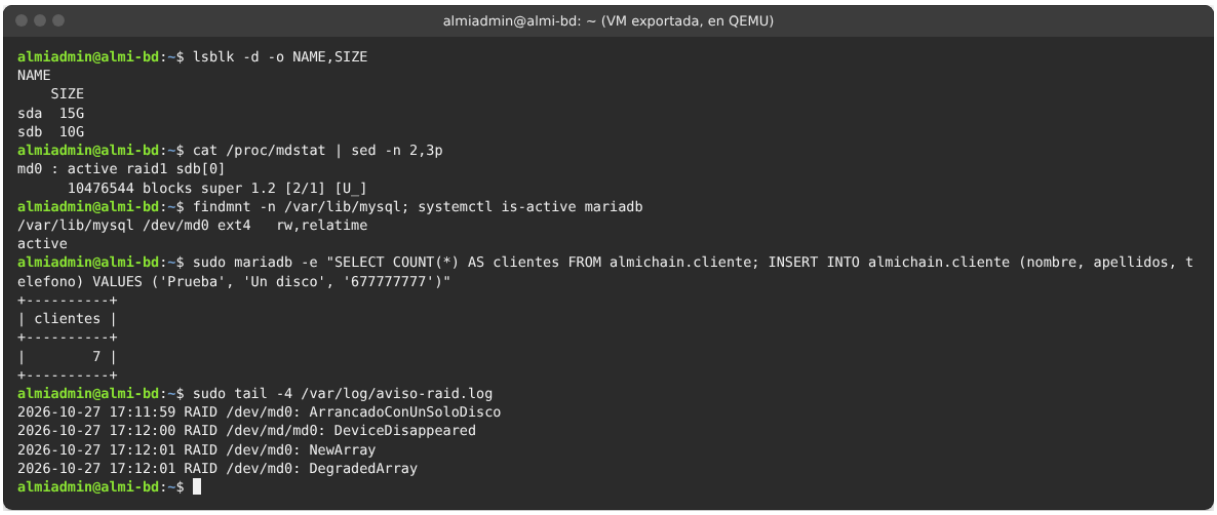


Figura 79. Arranque con un solo disco: md0 degradado, datos montados y MariaDB funcionando

Después se apaga, se pone un **disco nuevo y vacío** y se añade al RAID:

```
sudo mdadm /dev/md0 --add /dev/sdc
```

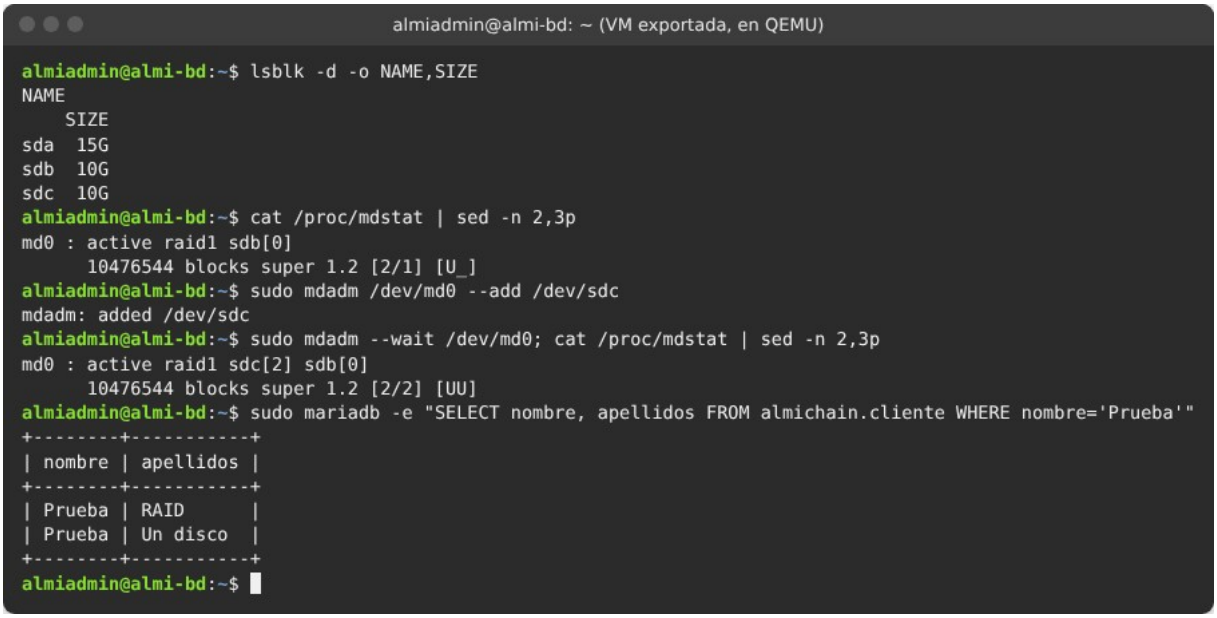


Figura 80. Disco nuevo añadido: reconstrucción hasta UU y los datos siguen

En VirtualBox es igual: VM apagada → **Configuración** → **Almacenamiento** → quitar `ALMI-BD-raid2` → arrancar y comprobar → apagar, **Añadir disco duro** → **Crear** uno nuevo de 10 GB → arrancar y `sudo mdadm /dev/md0 --add /dev/sdc`.

Copias cruzadas entre las dos VM (con ALMI-ODOO y ALMI-BD encendidas a la vez en la misma red): cada una envía su copia a la otra y la clave de copias solo puede escribir.



Figura 81. En ALMI-BD llegan las copias de Odoo

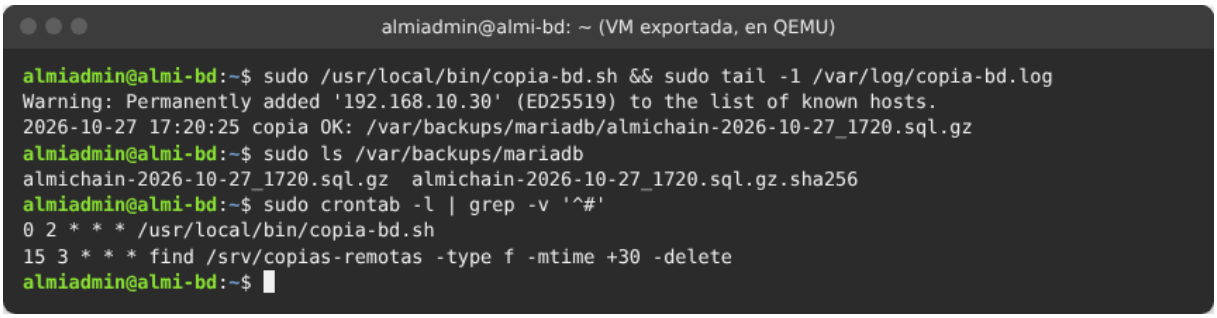


Figura 82. ALMI-BD hace su copia y la envía a ALMI-ODOO

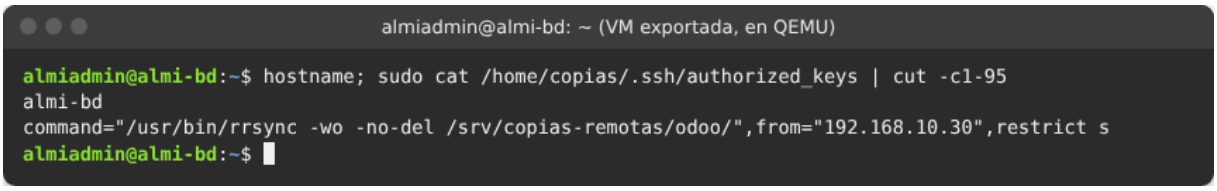


Figura 83. La clave de Odoo en ALMI-BD solo puede escribir en su carpeta (rrsync -wo -no-del) y solo desde 192.168.10.30

Seguridad aplicada

- MariaDB solo escucha en 127.0.0.1; root solo por socket local; sin usuarios anónimos ni base `test`.
- Tres usuarios con el mínimo privilegio necesario; ninguno con permisos globales de escritura.
- Contraseñas fuera del repositorio (variables al cargar `03-usuarios.sql`, `/root/.my.cnf` con permisos 600).
- Copias con permisos 700, huella SHA-256 y enviadas con una clave SSH que solo sirve desde la IP del otro servidor, sin terminal y solo para escribir en la carpeta de copias (`rrsync -wo -no-del`).
- MariaDB no arranca si el RAID no está montado: nunca trabaja con datos viejos por error.
- Cortafuegos ufw: solo SSH (22) desde la VLAN 10 (gestión) y la 99 (administración). En la VM exportada lo pone el servicio `almi-cortafuegos` (`07-bd-raid-copias/scripts/cortafuegos-bd.sh`) antes de que haya red.
- Secretos propios en cada VM importada: claves de host SSH y contraseñas de los 3 usuarios de MariaDB se generan en el primer arranque (ver docs/maquinas-virtuales.md (`docs/maquinas-virtuales.md`)).
- RAID 1: si falla un disco, la base de datos sigue funcionando.

Pruebas

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
1	MariaDB segura	`SELECT User, Host FROM mysql.user`	Sin anónimos ni root remoto	Sin anónimos; todos los usuarios en localhost (root, mysql, mariadb.sys y los 3 de la aplicación)	01 (`07-bd-raid-copias/capturas/01-mariadb-segura.png`)
2	Solo escucha en local	`ss -ltnp | grep 3306`	127.0.0.1:3306	127.0.0.1:3306	01 (`07-bd-raid-copias/capturas/01-mariadb-segura.png`)

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
3	Esquema completo	`SHOW FULL TABLES`	13 tablas y 1 vista	13 BASE TABLE y 1 VIEW	02 (`07-bd-raid-copias/capturas/02-esquema-cargado.png`)
4	Mínimo privilegio	`SHOW GRANTS` de los 3 usuarios	Como en la tabla de usuarios	Correcto en la VM (19 (`07-bd-raid-copias/capturas/19-vm-usuarios-secretos-cortafuegos.png`)). La captura 04 es anterior: ahí `copias` todavía tenía `SELECT` en `*.*`	04 (`07-bd-raid-copias/capturas/04-usuarios-privilegios.png`) 19 (`07-bd-raid-copias/capturas/19-vm-usuarios-secretos-cortafuegos.png`)
5	Consultas	`04-consultas.sql` como `consulta`	9 resultados correctos	9 resultados	05 (`07-bd-raid-copias/capturas/05-consultas-1-a-3.png`) 06 (`07-bd-raid-copias/capturas/06-consultas-4-a-6.png`) 07 (`07-bd-raid-copias/capturas/07-consultas-7-a-9.png`)
6	Solo lectura	`DELETE` y `DROP` como `consulta`	ERROR 1142	ERROR 1142 en los dos	08 (`07-bd-raid-copias/capturas/08-consulta-no-puede-borrar.png`)
7	Copia automática	`copia-bd.sh` y `crontab -l`	Copia local, huella, copia en ALMI-ODOO y cron a las 02:00	Correcto en el servidor de pruebas (la «otra máquina» era el mismo servidor); entre dos VM, prueba 15	09 (`07-bd-raid-copias/capturas/09-copia-automatica.png`)
8	Restauración	Borrar la base y restaurar	`diff` sin diferencias	“RESTAURACION CORRECTA: datos identicos”	10 (`07-bd-raid-copias/capturas/10-restauracion-1-antes.png`) 11 (`07-bd-raid-copias/capturas/11-`

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
					`restauracion-2-borrado.png`) 12 (`07-bd-raid-copias/capturas/12-restauracion-3-recuperada.png`)
9	Fallo de un disco del RAID	`mdadm --fail` y escribir en la base	`[U_]` y la base sigue funcionando	`[U_]`; INSERT correcto	20 (`07-bd-raid-copias/capturas/20-vm-raid-fallo-y-aviso.png`)
10	Aviso del fallo	Esperar tras el `--fail`	Aviso en el registro y en pantalla en menos de un minuto	“Fail /dev/sdc” en `/var/log/aviso-raid.log`, en el registro del sistema y en pantalla	20 (`07-bd-raid-copias/capturas/20-vm-raid-fallo-y-aviso.png`)
11	Reconstrucción	`mdadm --remove` y `--add`	Vuelve a `[UU]`	`[UU]`, 0 fallidos, el dato sigue	21 (`07-bd-raid-copias/capturas/21-vm-raid-reconstruccion.png`)
12	Arrancar sin un disco (prueba B)	Apagar, quitar el segundo disco y arrancar	RAID degradado, datos montados y MariaDB funcionando	`[U_]`, montado, MariaDB activa y se puede escribir; lo arrancó `almi-raid-degradado`	22 (`07-bd-raid-copias/capturas/22-vm-raid-sin-un-disco.png`)
13	Disco nuevo	Apagar, poner un disco vacío y `mdadm --add`	Vuelve a `[UU]` con todos los datos	`[UU]` y están los dos clientes de prueba	23 (`07-bd-raid-copias/capturas/23-vm-raid-disco-nuevo.png`)
14	MariaDB no usa datos viejos	Carpeta original bajo el montaje	Vacía; MariaDB depende del montaje	0 archivos; `RequiresMountsFor=/var/lib/mysql`	18 (`07-bd-raid-copias/capturas/18-vm-primer-arranque-raid.png`)
15	Copias cruzadas entre las dos VM	`copia-bd.sh` en ALMI-BD y `copia-odoo.sh` en ALMI-ODOO	Cada copia llega a la otra máquina	Ver capturas	24 (`07-bd-raid-copias/capturas/24-vm-copias-recibidas-de-odoo.png`) 25 (`07-bd-raid-copias/capturas/25-vm-copia-`

Nº	Qué se prueba	Cómo	Resultado esperado	Resultado obtenido	Captura
					`a-almi-odoo.png`)

Las pruebas 1 a 8 se hicieron en el servidor de pruebas (ver el origen de las evidencias en el README principal); las 9 a 15, con la VM exportada arrancada en QEMU. En VirtualBox están pendientes de repetir en clase.

Problemas encontrados y solución

Problema	Causa	Solución
El diagrama del equipo y la guía tenían modelos distintos	La guía proponía 15 tablas; el equipo diseñó el suyo	Se implementa el modelo del equipo (13 tablas)
Los datos de MariaDB tienen que estar en el RAID desde el principio	Si se instala MariaDB antes del RAID, los datos quedan en el disco de sistema	Se copian con `rsync -aAX` con el servicio parado y se monta el RAID en `/var/lib/mysql`
El servicio que crea el RAID en el primer arranque se quedaba “activating” para siempre	Hacía `systemctl start mariadb` desde un servicio que está ordenado *antes* de mariadb: cada uno esperaba al otro	`systemctl start --no-block mariadb`
`mariadb-dump` necesita contraseña en cron	cron no puede escribirla	`/root/.my.cnf` con permisos 600 y el usuario `copias` de solo lectura

Fuentes

- Documentación de MariaDB: mariadb-dump, CREATE USER y GRANT — https://mariadb.com/kb/
- `man mdadm` y wiki de RAID de Linux — https://raid.wiki.kernel.org/
- Guías paso a paso del equipo, parte 8

Presupuesto global (G11)

Precios con IVA en tiendas online, con el enlace y la fecha en que se miraron, porque cambian a menudo. Las filas que todavía están *por completar* las rellena el responsable de cada bloque (P02, R02, S01). Por eso el total de abajo es **parcial**.

Equipos del restaurante (P02)

Elemento	Cant.	Modelo elegido	Características clave	Precio unidad (€)	Total (€)	Enlace y fecha
TPV táctil de barra	2	Geon Zazoo POS 360	Intel N150, 8 GB, SSD 128 GB, táctil 15” Full HD, Windows 11, 4×USB 3.0, LAN	556,94	1.113,88	PcComponentes · 26/10/2026
Impresora de tickets	2	Epson TM-T20III (USB + serie)	Térmica 80 mm, ESC/POS, corte automático	251,58	503,16	PcComponentes · 26/10/2026
Cajón portamonedas	2	AVPos Cash-U280	Acero, 5 billetes y 8 monedas, conector RJ11 (lo abre la impresora)	45,99	91,98	PcComponentes · 27/10/2026
Equipo del gerente	1	*por completar* (4 núcleos, 16 GB, SSD NVMe 500 GB, **Windows 11 Pro incluido**)				
Monitor del gerente	1	LG 24BA560-B	23,8” IPS Full HD, altura ajustable, altavoces	132,97	132,97	PcComponentes · 27/10/2026
SAI	1	APC Back-UPS BE850G2-GR	850 VA, 8 enchufes Schuko, 2 USB (router, switch y servidores)	144,99	144,99	PcComponentes · 27/10/2026
Subtotal equipos					**1.986,98**	Sin el equipo del gerente

Impresora: según las opiniones de compradores, el modelo USB + serie de la TM-T20III no trae conexión para el cajón; para el restaurante real conviene el modelo con **Ethernet**

(C31CH51012), que además permite ponerla en la VLAN de TPV. No se ha encontrado su precio en una tienda española: *por completar* y sustituir la fila.

Licencias de Windows: los TPV vienen con Windows 11 y el equipo del gerente se elige con Windows 11 Pro incluido, así que no hace falta comprar licencias aparte. Hay que comprobar que la de los TPV sea **Pro** (las directivas de grupo del quiosco no existen en Home); si fuera Home, sumar la actualización a Pro. Microsoft ya no vende licencias de Windows 10 (BleepingComputer). En el laboratorio se usa Windows 10 Pro/Education con la licencia del centro.

Red (R02, R04)

Elemento	Cant.	Modelo / tarifa	Características	Precio (€)	Enlace y fecha
Línea de fibra para empresas	1	*por completar* (comparar 4 operadores: Movistar Empresas, Vodafone Business, Orange Empresas, Euskaltel/Digi; cuota mensual × 12)	Simétrica ≥ 300 Mb/s, IP fija, soporte 24 h, respaldo 4G/5G	/mes	
Router del local	1	MikroTik hEX (RB750Gr3)	5 puertos Gigabit, VLAN 802.1Q, NAT, cortafuegos, SSH	61,90	Galaxus (Austria) · 27/10/2026. **Precio con IVA austriaco y sin envío; falta buscarlo en una tienda española**
Switch gestionable 24 puertos	1	TP-Link TL-SG3428	24 Gigabit + 4 SFP, gestionable L2, VLAN 802.1Q, seguridad de puerto, SSH	214,81	PcComponentes · 27/10/2026
Punto de acceso / router Wi-Fi	2	Los 2 routers del laboratorio	WDS, WPA2-AES	0	—
Cable Cat 6 U/UTP, conectores, tomas y panel	1 lote	*por completar*	T568B		
Subtotal red				**276,71**	Sin la línea ni el cableado

Arcade stick (S01)

Elemento	Cant.	Detalle	Precio (€)	Enlace y fecha
Placa Arduino	1	La que da el centro	0	—
Kit de joystick y botones	1	Eurojoystick 2 de 8 direcciones, 10 botones de 28 mm (8 de juego, 1 jugador y moneda), cables y microinterruptores. Trae un codificador USB que no hace falta (se usa el Arduino)	33,05	Arcadexpress · 27/10/2026
Caja	1	MDF o impresión 3D: *por completar*		
Subtotal arcade stick			**33,05**	Sin la caja

Software (P03)

Todo el software del reto es gratuito: Ubuntu, MariaDB, PostgreSQL, Odoo Community (LGPL), Docker, nginx, MAME (GPL), KeePassXC, Arduino IDE, Terraform y AWS Academy (créditos del curso). Windows va incluido en los equipos (ver la nota de licencias).

Total

Apartado	Total (€)
Equipos	1.986,98
Red	276,71
Arcade stick	33,05
Total parcial (con IVA)	**2.296,74**
Línea de fibra	*por completar* €/mes (gasto mensual, aparte)

Falta para el total completo: equipo del gerente, impresora con Ethernet (sustituye a la USB), cableado, caja del stick y la cuota de la línea.

Resumen de pruebas, conclusiones y trabajo pendiente

Resumen de las pruebas

Cada bloque tiene su tabla de pruebas con el resultado obtenido y la captura. Este es el resumen a fecha de 27 de octubre de 2026; el origen de cada evidencia (servidor de pruebas, QEMU o VirtualBox) está en la Introducción (`README.md`).

Bloque	Pruebas hechas	Pendientes	Dónde se hicieron
01 Windows gerente y TPV	0 de 11	Todas	— (necesita las VM de Windows de clase)
02 Recreativa	7 de 9	Benchmark en la VM de VirtualBox y prueba con el stick	QEMU (VM exportada) y anfitrión (benchmark)
03 Arcade stick	0 de 6 (los programas compilan)	Todas	— (necesita la placa y el material)
04 Red	0 de 14	Todas	— (necesita Packet Tracer y los routers)
05 ERP Odoo	8 de 11 completas, 3 a medias	Envío real del correo y ticket en la impresora física	Servidor de pruebas y QEMU (VM exportada)
06 Web en AWS	0 de 10 (Lighthouse hecho solo en local)	Todas	— (necesita AWS Academy)
07 Base de datos	15 de 15	Repetir en VirtualBox las del RAID	Servidor de pruebas (1-8) y QEMU, VM exportada (9-15)
VM exportadas	Primer arranque de las tres en QEMU, copias cruzadas entre ALMI-ODOO y ALMI-BD	Importación en VirtualBox	QEMU

Conclusiones

Lo que funciona y está demostrado. El ERP Odoo con los dos TPV factura solo al cobrar a un cliente, publica la factura, la deja lista para enviar por correo con el PDF y descuenta el stock; todo publicado solo por HTTPS. La base de datos propia tiene 13 tablas normalizadas, usuarios de mínimo privilegio, copias diarias comprobadas con su huella, copia cruzada a la otra máquina y restauración probada; el RAID 1 aguanta la pérdida de un disco y se reconstruye. La recreativa arranca sola en MAME con 1 núcleo, 1 GB y sin swap, y no usa swap mientras se juega. Las tres VM Linux se pueden reconstruir con `herramientas/` (`herramientas/README.md`) (hacen falta las contraseñas iniciales, acceso a los repositorios de Ubuntu y, para Odoo, un Odoo ya configurado del que sacar los datos) y generan sus propias claves y contraseñas de servicio al importarse; solo comparten la contraseña inicial de `almiadmin` (obligatorio cambiarla) y la clave de las copias cruzadas.

Lo que está preparado pero falta probar. Los puestos Windows, la red (Packet Tracer y routers físicos), el arcade stick y la web en AWS tienen su procedimiento, sus scripts y sus configuraciones revisados, pero no se han ejecutado todavía: dependen de material, licencias o cuentas que se usan en clase. Sus tablas de pruebas están marcadas como «Pendiente — en clase».

Lo que se aprendió de la revisión. La auditoría interna (carpeta `auditoria/` (`auditoria`)) encontró y se corrigieron, entre otros: un certificado HTTPS que podía firmar certificados para cualquier web, una clave de copias que permitía borrar las copias del otro servidor, un orden de pasos en la base de datos que dejaba los datos ocultos bajo el RAID, una directiva de contraseñas que se aplicaba después de crear los usuarios y documentación que daba por hechas cosas que faltaban. La lección principal: **cada afirmación tiene que tener su prueba**, y lo que no se ha probado se dice.

Valoración del equipo: *por escribir por el equipo al terminar el reto (qué ha ido bien, qué cambiaríamos y qué ha aprendido cada uno).*

Trabajo pendiente antes de la entrega (13 de noviembre)

1. Montar en clase los bloques 01, 03, 04 y 06 y rellenar sus tablas de pruebas con capturas.
2. Importar las tres OVA en VirtualBox y repetir allí las pruebas principales (prueba B del RAID, benchmark de la recreativa).
3. Poner la contraseña de aplicación de Gmail en Odoo y probar el envío real; imprimir un ticket en la impresora física.
4. Gestión: repartir las tareas en el tablero, exportarlo, Anexo I firmado (sin datos personales en el repositorio), Anexo II y commits de los cinco.
5. Presupuesto: completar las filas que faltan y el total.
6. Diagramas que faltan: plano del local (draw.io), archivo `.pkt` de Packet Tracer y **diseño conceptual (boceto) de la web**.
7. Recreativa: 3 ROMs legales de mamedev.org y un tercer juego en el benchmark.
8. Windows: `Test-NetConnection` al 443 y al 8069 de Odoo desde ALMI-TPV (prueba 2 de Odoo desde Windows).
9. Web: hacer público el repositorio de la web (W04) si lo pide el profesor; hoy es privado.
10. Presentación final (G15) y valoración del equipo (apartado de conclusiones).

Bibliografía

Documentación oficial consultada (cada capítulo tiene además sus propias fuentes):

- Odoo 18: documentación oficial, https://www.odoo.com/documentation/18.0/, e instalación en servidor, https://www.odoo.com/documentation/18.0/administration/on_premise/deploy.html.
- Imagen de Docker de Odoo: https://hub.docker.com/_/odoo.
- MariaDB Knowledge Base: https://mariadb.com/kb/.
- Linux RAID (mdadm): https://raid.wiki.kernel.org/.
- rsync y rrsync: páginas de manual `rsync(1)` y `rrsync(1)` de Ubuntu 24.04.
- OpenSSL: página de manual `openssl-req(1)` y `x509v3_config(5)`.
- nginx: https://nginx.org/en/docs/.
- Ubuntu Server 24.04: https://documentation.ubuntu.com/server/.
- Terraform, proveedor de AWS: https://registry.terraform.io/providers/hashicorp/aws/latest/docs.
- Let's Encrypt y Certbot: https://certbot.eff.org/; DuckDNS: https://www.duckdns.org/install.jsp.
- MDN, API de geolocalización: https://developer.mozilla.org/es/docs/Web/API/Geolocation_API.
- MAME: https://docs.mamedev.org/ y ROMs legales https://www.mamedev.org/roms/.
- Arduino, librería Keyboard: https://docs.arduino.cc/language-reference/en/functions/usb/Keyboard/; ArduinoJoystickLibrary: https://github.com/MHeironimus/ArduinoJoystickLibrary.
- Cisco Packet Tracer y documentación de IOS de Cisco (VLAN, ACL, NAT, SSH).

- Microsoft: documentación de PowerShell (módulo `Microsoft.PowerShell.LocalAccounts`) y de directivas de grupo.

Índice de anexos

Anexo	Qué es	Dónde
A	Máquinas virtuales exportadas: importar, primer acceso y secretos	`docs/maquinas-virtuales.md` (`docs/maquinas-virtuales.md`)
B	Routers reales y repetidor WDS	`04-red/routers/configuracion-routers.md` (`04-red/routers/configuracion-routers.md`)
C y D	Web: guía de despliegue y explicación del código	`06-web-aws/docs/` (`06-web-aws/docs`)
E	Herramientas: cómo se hicieron las VM, las pruebas y las capturas	`herramientas/README.md` (`herramientas/README.md`)
—	Guías paso a paso del equipo (planificación previa)	`00-gestion/Guias-paso-a-paso-Reto1-Almi-Chain.docx` (`00-gestion/Guias-paso-a-paso-Reto1-Almi-Chain.docx`)
—	Informe de la auditoría interna	`auditoria/` (`auditoria`)
—	Anexo I y Anexo II del reto	Pendientes (ver 00-gestion (`00-gestion/README.md`))

Anexos

Documentos que amplían algún capítulo. Están también en el repositorio, en la ruta que se indica en cada uno.

Anexo A. Máquinas virtuales exportadas

Las tres máquinas Linux del reto están exportadas en formato **OVA** (Open Virtualization Format, OVF 1.0) para importarlas en VirtualBox ya configuradas. Las .ova no están dentro del repositorio (GitHub no admite archivos de ese tamaño): se entregan aparte, partidas en trozos de menos de 400 MB (`ALMI-ODOO.ova.parte1`, `parte2`…) cuando hace falta, cada una con su huella `.sha256`. Las huellas también están en `docs/ova-sha256.txt` para comprobar que lo descargado es lo entregado.

> **Cómo se han hecho y probado.** Las OVA no se crearon con el instalador de Ubuntu en VirtualBox, sino con los scripts de `herramientas/construir-vm` (`herramientas/README.md`) (Ubuntu instalado con `debootstrap`, con los mismos paquetes y archivos que los README). Se han probado arrancando sus discos en **QEMU**, no en VirtualBox, que no estaba disponible donde se prepararon. **La importación en VirtualBox está pendiente de comprobar en clase** (prueba 1 de la tabla de abajo).

Unir los trozos

Descargar todos los trozos, el `.sha256` de cada .ova y los scripts en **la misma carpeta**:

- **Zorin / Linux**: `bash unir-ova-linux.sh` → une los trozos, comprueba la huella de **todas** las .ova (también las que no van partidas) y borra los trozos si está bien.
- **Windows**: doble clic en `unir-ova-windows.bat` → une los trozos y comprueba las huellas con PowerShell (`OK` o `HUELLA DISTINTA`).

A mano en Linux: `cat ALMI-ODOO.ova.parte1 ALMI-ODOO.ova.parte2 ALMI-ODOO.ova.parte3 > ALMI-ODOO.ova` y `sha256sum -c ALMI-ODOO.ova.sha256`.

Archivo	Máquina	Hardware	Red	Qué hace la primera vez que arranca
`ALMI-ODOO.ova`	Servidor del ERP	3 GB · 2 CPU · disco de 25 GB	IP fija 192.168.10.30	Genera sus secretos (certificado HTTPS, contraseñas de PostgreSQL y maestra de Odoo, claves SSH), carga Odoo 18 y PostgreSQL 16 en Docker, restaura la base `almichain` con los datos del reto y aplica el cortafuegos. Tarda unos minutos. Después: `https://192.168.10.30`
`ALMI-BD.ova`	Servidor de base de	2 GB · 2 CPU ·	IP fija	Crea el **RAID 1** con

Archivo	Máquina	Hardware	Red	Qué hace la primera vez que arranca
	datos	disco de 15 GB + **2 discos de 10 GB**	192.168.10.20	los dos discos vacíos y mueve encima los datos de MariaDB; crea los 3 usuarios de MariaDB con contraseñas nuevas; cortafuegos; claves SSH
`ALMI-ARCADE.ova`	Recreativa	**1 GB · 1 CPU** · disco de 8 GB · sin swap · tarjeta de sonido	DHCP	Entra sola como `arcade` y abre MAME

Los dos Windows (ALMI-GERENTE y ALMI-TPV) no se exportan: necesitan la ISO y la licencia de Windows de clase. Se preparan con los scripts de `01-windows-gerente-tpv` (`01-windows-gerente-tpv/README.md`).

Importar

1. Crear la **Red NAT ALMICHAIN** si no existe (00-gestion (`00-gestion/README.md`)), con el reenvío de puertos en **127.0.0.1**.
2. **Archivo** → **Importar servicio virtualizado** → elegir la `.ova` → **Siguiente**.
3. En la lista de ajustes, en **Política de dirección MAC** elegir *Generar nuevas direcciones MAC para todos los adaptadores de red* → **Terminar**.
4. Con la VM importada y apagada: **Configuración** → **Red** → **Adaptador 1** → **Conectado a: Red NAT** → **Nombre: ALMICHAIN**.
5. Solo en ALMI-ARCADE: comprobar **Configuración** → **Pantalla** → **Controlador gráfico: VMSVGA** y **Audio: habilitado** (el archivo OVF estándar no guarda el controlador gráfico; VirtualBox pone el de Ubuntu de 64 bits). Para el stick, **USB** → **filtro** con el Arduino.
6. Arrancar. ALMI-ODOO y ALMI-BD tienen que estar en el **mismo PC** y en la misma Red NAT para que funcionen las copias cruzadas.

Primer acceso y secretos

Las VM no llevan claves de servidor ni contraseñas de servicio propias: cada copia importada las genera la primera vez que arranca, así dos equipos que importen la misma OVA no comparten esas contraseñas ni certificados. Lo que **sí** comparten todas las copias está al final de este apartado.

Máquina	Qué se genera	Dónde queda
ALMI-ODOO y ALMI-BD	Cortafuegos, **antes de que haya red** (servicio `almi-cortafuegos`)	`ufw status`
ALMI-ODOO y ALMI-BD	Claves de host SSH (`ssh-keygen -A` justo antes de arrancar el servidor SSH)	`/etc/ssh/ssh_host_*`
ALMI-ODOO	Contraseña de PostgreSQL	`/opt/odoo/.env` (solo root)
ALMI-ODOO	Contraseña maestra de Odoo, contraseñas nuevas de los 4 usuarios de Odoo (administrador,	`/root/credenciales-primer-arranque.txt` (solo root) y `/etc/ssl/almichain/`. La **huella**

Máquina	Qué se genera	Dónde queda
	gerente y camareros), secreto con el que Odoo firma las sesiones (`database.secret`) y certificado HTTPS (`CA:FALSE`)	**del certificado** sale también en la consola, encima de `login:`, para comprobarla en los Windows
ALMI-BD	Contraseñas de `app_almichain`, `consulta` y `copias`	`/root/credenciales-primer-arranque.txt` (solo root); la de `copias` en `/root/.my.cnf` y la de `consulta` en `~almiadmin/.consulta.cnf`

Pasos la primera vez:

1. Entrar como `almiadmin` con la **contraseña inicial**, que el equipo entrega aparte (no está en el repositorio). El sistema **obliga a cambiarla** en ese momento (`chage -d 0`).
2. `sudo cat /root/credenciales-primer-arranque.txt` → pasar los secretos a KeePassXC → `sudo shred -u /root/credenciales-primer-arranque.txt`.
3. En ALMI-ODOO, las contraseñas de los usuarios de Odoo (administrador, gerente y los dos camareros) también están en ese archivo: son nuevas, distintas en cada copia. Cada uno puede cambiar la suya en Odoo (**Preferencias** → **Seguridad de la cuenta**).

Lo que sí comparten todas las copias: (1) la **contraseña inicial de `almiadmin`**, que hay que cambiar al entrar (el sistema obliga), y (2) la pareja de claves de las copias cruzadas (`/root/.ssh/copias_ed25519`), para que funcionen nada más importar las dos VM. Esa clave solo sirve desde la IP del otro servidor y solo para escribir en su carpeta de copias (`rrsync -wo -no-del`). Para cambiarla:

```
# En ALMI-ODOO (y lo mismo al revés en ALMI-BD, con sus IP y carpetas)
sudo ssh-keygen -t ed25519 -N '' -C copias-odoo -f /root/.ssh/copias_ed25519   # sobrescribir: y
sudo cat /root/.ssh/copias_ed25519.pub
# En ALMI-BD, sustituir la clave de la línea de /home/copias/.ssh/authorized_keys, sin tocar el principio:
# command="/usr/bin/rrsync -wo -no-del /srv/copias-remotas/odoo/",from="192.168.10.30",restrict ssh-ed25519 <clave nueva>
```

Usuarios

Máquina	Usuarios
Las tres	`almiadmin` (administrador, sudo; cambia la contraseña inicial al entrar)
ALMI-ARCADE	`arcade` (sin privilegios, entra solo)
Odoo (web)	administrador `almichain.reto@gmail.com`, `gerente@almichain.local`, `barra1@almichain.local`, `barra2@almichain.local`
MariaDB	`app_almichain`, `consulta`, `copias` (root solo por `sudo mariadb`)

Acceso desde el PC de clase

Con el reenvío de puertos de la red ALMICHAIN (ver 00-gestion (`00-gestion/README.md`)):

```
ssh -p 2230 almiadmin@localhost     # ALMI-ODOO
ssh -p 2220 almiadmin@localhost     # ALMI-BD
# Odoo en el navegador del PC: https://localhost:8443 (el certificado incluye localhost)
```

Pruebas de las OVA

Nº	Qué se prueba	Dónde	Resultado
1	Importar cada .ova en VirtualBox 7 y arrancarla	VirtualBox en clase	**Pendiente — en clase**
2	Huellas y formato	`sha256sum -c` con `docs/ova-sha256.txt` y lectura del `.ovf`	Correcto (huellas de las 3 OVA; cada VMDK igual a su disco probado)
3	Primer arranque de cada VM desde cero	QEMU	Ver las pruebas con la VM exportada de los bloques 02 (`02-linux-recreativa/README.md`), 05 (`05-erp-odoo/README.md`) y 07 (`07-bd-raid-copias/README.md`)

Si algo no va

Síntoma	Solución
La VM no tiene red	Comprobar que el adaptador está en *Red NAT → ALMICHAIN* y `ip -br a` dentro de la VM. La red se configura para cualquier interfaz `en*`
Odoo no responde tras 15 minutos	`sudo systemctl status almi-primer-arranque` y `sudo tail -50 /var/log/almi-primer-arranque.log`
nginx sale como fallido justo después del primer arranque	Es normal mientras el primer arranque no ha creado el certificado; al terminar lo reinicia. Si sigue: `sudo systemctl restart nginx`
ALMI-BD no crea el RAID	Comprobar que tiene los dos discos de 10 GB en el controlador SATA (`lsblk`). Registro: `/var/log/almi-crear-raid.log`. Se puede lanzar a mano: `sudo /usr/local/sbin/almi-crear-raid.sh`
MariaDB no arranca en ALMI-BD	A propósito no arranca si el RAID no está montado: `cat /proc/mdstat`, `findmnt /var/lib/mysql`
La recreativa se queda en negro	Controlador gráfico VMSVGA. Para entrar a administrar: Ctrl + Alt + F2

Anexo B. Routers reales y repetidor WDS

1. Configuración

Los menús cambian según la marca. Los nombres de abajo son los habituales (TP-Link entre paréntesis). Apunta modelo y versión de firmware de cada router.

Router principal (R1), conectado a la toma de red de clase por su puerto WAN:

1. Reinícialo de fábrica: con él encendido, mantén pulsado el botón RESET 10 segundos.
2. Conecta el PC por cable a un puerto LAN. Mira la puerta de enlace con `ip route | grep default` (por ejemplo 192.168.0.1) y ábrela en el navegador. Usuario y contraseña de fábrica: en la pegatina del router.

3. Cambia la contraseña de administración (Herramientas del sistema → Contraseña). Guárdala en KeePassXC.
4. Actualiza el firmware si hay versión nueva (Herramientas del sistema → Actualización de firmware).
5. Wi-Fi (Inalámbrico → Configuración):
   - SSID: AlmiChain-Clientes.
   - Región: España.
   - Canal: **fijo** (no automático). Elige el 1, el 6 o el 11, el menos usado. Para verlo desde Zorin: `nmcli -f SSID,CHAN,SIGNAL dev wifi list`.
   - Ancho de canal: **20 MHz** (más estable con WDS).
6. Seguridad Wi-Fi: **WPA2-PSK** con cifrado **AES** y una clave de 12 caracteres o más. No uses WPA3: la mayoría de routers no admiten WDS con WPA3.
7. Desactiva **WPS** (Inalámbrico → WPS → Deshabilitar).
8. Desactiva la **administración remota** (Seguridad → Administración remota: IP 0.0.0.0 o desactivado) y **UPnP** (Reenvío → UPnP).
9. Si tiene opción de **aislamiento de clientes** (AP Isolation), actívala: los clientes del Wi-Fi no se ven entre ellos.
10. Mira el rango DHCP (DHCP → Configuración), por ejemplo .100 a .199. Lo necesitas para el paso siguiente.
11. Guarda y haz capturas de cada pantalla.

Router secundario (R2), sin cable al principal; solo por Wi-Fi:

1. Reinícialo de fábrica y conéctate a él por cable desde el PC.
2. Cambia su contraseña de administración y guárdala.
3. Cambia su IP de LAN (Red → LAN) a una de la red de R1 que **no** esté en el rango DHCP de R1. Ejemplo: si R1 es 192.168.0.1, pon R2 en 192.168.0.2. Guarda. El router se reinicia: entra de nuevo con la IP nueva.
4. Desactiva su **servidor DHCP** (DHCP → Deshabilitar). Solo reparte IPs R1.
5. Inalámbrico → Configuración:
   - SSID: AlmiChain-Clientes-2 (distinto para las pruebas; al final puede ser el mismo que R1).
   - Canal: **el mismo** que R1. Ancho: 20 MHz.
   - Marca **Habilitar puente WDS** (Enable WDS Bridging).
   - Pulsa **Buscar** (Survey) → en la lista, **Conectar** en AlmiChain-Clientes.
   - Tipo de clave: **WPA2-PSK**. Contraseña: la de R1. Guarda.
6. Seguridad Wi-Fi del propio R2: WPA2-PSK, AES, clave de 12 caracteres o más.
7. Desactiva WPS, administración remota y UPnP, como en R1.
8. Reinicia R2. En **Estado** (Status), el estado WDS tiene que poner **Run** o “Conectado”.

9. Si vuestro modelo no tiene WDS, busca “Modo repetidor” o “Extensor de cobertura” y documenta que se usó ese modo.

2. Pruebas del repetidor

1. Conecta un portátil o móvil a AlmiChain-Clientes-2 (R2). Desconecta el cable del PC si lo tiene.
2. Comprueba que la IP la da R1 (misma red y puerta de enlace = IP de R1):

```
ip -4 addr show
ip route | grep default
```

3. Comprueba que llega a R1 y a Internet sin pérdidas (tiene que poner 0% packet loss):

```
ping -c 20 192.168.0.1
ping -c 20 8.8.8.8
```

4. Velocidad con iperf3: en un PC conectado **por cable a R1**, `sudo apt install -y iperf3` y luego `iperf3 -s`. En el portátil (también con iperf3 instalado), primero conectado a R1 y después a R2 (cambiando `IP_DEL_PC_SERVIDOR` por la IP de ese PC):

```
iperf3 -c IP_DEL_PC_SERVIDOR -t 20
```

5. Es normal que a través de R2 salga más o menos la mitad: con WDS cada paquete viaja dos veces por el aire. Escríbelo en la documentación.
6. Cobertura: elige 4 puntos (al lado de R1, mitad del pasillo, al lado de R2, el punto más lejano). En cada uno mide la señal con `nmcli -f SSID,SIGNAL,CHAN dev wifi list` o con una app de móvil (WiFi Analyzer) y rellena:

Punto	Señal de R1 (%)	Señal de R2 (%)	Velocidad iperf3 (Mb/s)
Junto a R1	Pendiente — en clase		
Mitad del pasillo	Pendiente — en clase		
Junto a R2	Pendiente — en clase		
Punto más lejano	Pendiente — en clase		

Estado: los routers físicos todavía no se han configurado; esta tabla y las capturas se rellenan en clase.

Anexo C. Web: guía de despliegue

← Volver al README (`06-web-aws/README.md`)

Esta guía parte de cero, en un PC con **Zorin OS** (o Ubuntu). Todos los comandos se escriben en la **terminal**.

> 💡 **¿Dónde estoy?** En la terminal, lo que aparece entre los `:` y el `$` es la carpeta en la que estás. Por ejemplo, `alberto@pc:~/reto1/almichain-web-terraform$` significa que estás en `~/reto1/almichain-web-terraform`. Todos los comandos de Terraform se ejecutan **dentro de la carpeta del proyecto**.

Paso 1. Instalar Terraform

Comprueba si ya está instalado:

```
terraform -version
```

Si aparece `Terraform v1.x.x`, pasa al paso 2. Si aparece `orden no encontrada`, instálalo:

```
# 1. Descargar la clave de HashiCorp, para que el sistema confíe en sus paquetes
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg

# 2. Añadir el repositorio de HashiCorp a apt
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(. /etc/os-release && echo $UBUNTU_CODENAME) main" |
sudo tee /etc/apt/sources.list.d/hashicorp.list

# 3. Instalar
sudo apt update && sudo apt install -y terraform

# 4. Comprobar
terraform -version
```

En **Windows**: `winget install Hashicorp.Terraform` y abrir una terminal nueva.

Paso 2. Descargar el proyecto

Clónalo en una carpeta fija (no en Descargas, para no borrarlo sin querer):

```
mkdir -p ~/reto1
cd ~/reto1
git clone https://github.com/krako829/almichain-web-terraform.git
cd almichain-web-terraform
ls
```

Tienen que aparecer `main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `user_data.sh.tftpl`, `terraform.tfvars.example`, `README.md` y las carpetas `web` y `docs`.

> Si el repositorio es privado, GitHub pedirá usuario y un *token* personal en lugar de la contraseña. También puedes descargarlo como ZIP desde GitHub: **Code → Download ZIP**.

Paso 3. Credenciales de AWS Academy

Terraform necesita permiso para crear cosas en la cuenta del lab.

1. Entra en AWS Academy → tu curso → **Learner Lab** → **Start Lab**. Espera a que el círculo junto a “AWS” se ponga **verde**.
2. Pulsa **AWS Details**:
   - Apunta la **Region** que aparece (normalmente `us-east-1`).
   - Junto a **AWS CLI**, pulsa **Show** y copia el bloque entero, desde `[default]` hasta el final de `aws_session_token`.
3. Crea el fichero de credenciales y pega el bloque:

   ```
   mkdir -p ~/.aws
   nano ~/.aws/credentials
   ```

   - Pega con **Ctrl + Shift + V**. En la terminal, Ctrl + V solo no pega.
   - Guarda con **Ctrl + O** y **Enter**. Sal con **Ctrl + X**.
4. Comprueba que se ha guardado:

   ```
   cat ~/.aws/credentials
   ```

   Tienen que salir 4 líneas: `[default]`, `aws_access_key_id=...`, `aws_secret_access_key=...` y `aws_session_token=...`.

⚠️ Las credenciales **caducan a las ~4 horas**. Cada vez que inicies el lab, repite este paso con las nuevas.

⚠️ Usa `nano` **sin** **`sudo`**. Con `sudo`, el fichero se guarda en la carpeta de root y Terraform no lo encuentra.

Paso 4. Token de DuckDNS

1. Entra en https://www.duckdns.org e inicia sesión con la cuenta donde está el subdominio **almichain2**.
2. Copia el **token**. Si el navegador traduce la página al español, aparece como **“simbólico”**.
3. Comprueba que en la lista de dominios de esa cuenta aparece **almichain2**. Si no aparece, créalo: escribe `almichain2` en *sub domain* y pulsa **add domain**.

El token solo sirve para la cuenta que tiene el dominio. Con el token de otra cuenta, DuckDNS responde `KO` y el HTTPS no se activa.

Paso 5. Rellenar `terraform.tfvars`

Este fichero contiene los datos personales. No está en el repositorio: se crea a partir de la plantilla.

```
cp terraform.tfvars.example terraform.tfvars
nano terraform.tfvars
```

Déjalo así, con tus datos:

```
duckdns_subdominio = "almichain2"
duckdns_token      = "TU-TOKEN-DE-DUCKDNS"
email              = "tu-correo@gmail.com"
ssh_cidr           = "81.34.10.7/32"   # TU IP pública + /32
region             = "us-east-1"   # la que pone en AWS Details
```

El correo lo usa Let’s Encrypt para avisar si el certificado va a caducar.

`ssh_cidr` es obligatorio: el puerto 22 de la EC2 solo se abre a tu IP. Para saber tu IP pública: `curl -s https://checkip.amazonaws.com`. Terraform no acepta `0.0.0.0/0`. Si cambias de red (por ejemplo, de clase a casa), actualiza el valor y vuelve a hacer `terraform apply`.

Paso 6. `terraform init`

```
terraform init
```

Descarga los proveedores que usa el proyecto: `aws`, `tls` (para generar la clave SSH) y `local` (para guardarla en un fichero). Tiene que terminar con:

```
Terraform has been successfully initialized!
```

Solo hace falta la primera vez, o cuando cambie `versions.tf`.

Paso 7. `terraform plan`

```
terraform plan
```

Todavía **no crea nada**: solo enseña lo que va a hacer. En un despliegue desde cero, al final pone:

```
Plan: 13 to add, 0 to change, 0 to destroy.
```

Los 13 recursos son: VPC, Internet Gateway, subred, tabla de rutas y su asociación, grupo de seguridad, clave privada, par de claves, fichero `.pem`, IP elástica, EC2, asociación de la IP y subida de la web.

Paso 8. `terraform apply`

```
terraform apply
```

Vuelve a enseñar el plan y pregunta `Enter a value:`. Escribe **`yes`** y pulsa **Enter**.

El proceso tarda **unos 5 minutos**:

1. En 1-2 minutos crea la red, la IP y el servidor.
2. Luego aparece `terraform_data.subir_web: Still creating...` durante 3-4 minutos. Es normal: Terraform espera a que el servidor instale Nginx, actualice DuckDNS y consiga el certificado.
3. Después sube la web por SSH y termina.

Al final verás:

```
terraform_data.subir_web (remote-exec): ----- Resultado de la instalacion -----
terraform_data.subir_web (remote-exec): ==> LISTO: https://almichain2.duckdns.org

Apply complete! Resources: 13 added, 0 changed, 0 destroyed.

Outputs:
ip_publica          = "x.x.x.x"
ssh                 = "ssh -i almibites-key.pem ubuntu@x.x.x.x"
url                 = "https://almichain2.duckdns.org"
ver_log_instalacion = "sudo tail -n 40 /var/log/almibites-setup.log"
```

Paso 9. Comprobar

Abre **https://almichain2.duckdns.org**. Las pruebas completas, para hacer capturas para la memoria, están en 4. Pruebas (`06-web-aws/docs/04-pruebas.md`).

Resumen de comandos

```
# Cada sesión del lab: copiar credenciales nuevas
nano ~/.aws/credentials

# Primera vez
cp terraform.tfvars.example terraform.tfvars && nano terraform.tfvars
terraform init

# Desplegar o aplicar cambios
terraform plan
terraform apply

# Ver los datos de salida otra vez
terraform output

# Entrar al servidor
ssh -i almibites-key.pem ubuntu@$(terraform output -raw ip_publica)

# Borrar todo
terraform destroy
```

Anexo D. Web: explicación del código

← Volver al README (`06-web-aws/README.md`)

Cómo trabaja Terraform

Terraform lee todos los ficheros `.tf` de la carpeta como si fueran uno solo. En ellos se **describe** la infraestructura que se quiere tener, no los pasos para crearla. Al hacer `terraform apply`, Terraform:

1. Compara lo que dice el código con lo que hay guardado en `terraform.tfstate`.
2. Calcula qué hay que crear, cambiar o borrar (eso es el `plan`).
3. Lo hace en el orden correcto según las dependencias. Por ejemplo, la subred necesita que exista la VPC.
4. Guarda el resultado en `terraform.tfstate`.

Por eso el `terraform.tfstate` es tan importante: es la "memoria" de Terraform.

`versions.tf`

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws   = { source = "hashicorp/aws",   version = "~> 5.0" }
    tls   = { source = "hashicorp/tls",   version = "~> 4.0" }
    local = { source = "hashicorp/local", version = "~> 2.4" }
  }
}

provider "aws" {
  region = var.region
  default_tags { tags = { Proyecto = "Almi-Chain Reto 1", Gestion = "Terraform" } }
}
```

- **Proveedores**: son los "plugins" de Terraform. `aws` habla con Amazon, `tls` genera claves criptográficas y `local` escribe ficheros en el PC.
- `~> 5.0` significa "cualquier versión 5.x".
- El proveedor `aws` no lleva credenciales en el código: las lee de `~/.aws/credentials`. Así no se suben a GitHub.
- `default_tags` pone estas etiquetas a todos los recursos, para reconocerlos en la consola de AWS.

`variables.tf`

Define los datos que se pueden cambiar sin tocar el código. Sus valores se ponen en `terraform.tfvars`.

Variable	Valor por defecto	Para qué sirve
`region`	`us-east-1`	Región de AWS. El Learner Lab solo permite `us-east-1` y `us-west-2`
`proyecto`	`almibites`	Prefijo de los nombres (`almibites-vpc`, `almibites-web`…)
`duckdns_subdominio`	`almichain2`	Subdominio sin `.duckdns.org`. Tiene una **validación**: si alguien escribe `almichain2.duckdns.org`,

Variable	Valor por defecto	Para qué sirve
		Terraform da un error explicándolo
`duckdns_token`	— (obligatoria)	Token de DuckDNS. Marcada `sensitive`: eso **solo** la oculta en la salida de Terraform. El valor viaja en el `user_data` de la EC2 (se puede leer desde la propia máquina y queda en `terraform.tfstate`), así que el `user_data` desactiva el trazado mientras lo usa para que no acabe en los logs. Si alguien entra en el servidor, conviene regenerar el token en duckdns.org
`email`	— (obligatoria)	Correo para Let's Encrypt
`certbot_staging`	`false`	`true` pide un certificado de prueba (ver Mantenimiento (`06-web-aws/docs/05-mantenimiento.md`))
`tipo_instancia`	`t3.micro`	Tamaño de la EC2 (2 vCPU, 1 GB RAM)
`ssh_cidr`	— (obligatoria)	Tu IP pública con `/32`. Tiene una validación que rechaza `0.0.0.0/0`: el SSH no se abre a todo Internet
`vpc_cidr`	`10.0.0.0/16`	Rango de direcciones de la VPC
`subred_publica_cidr`	`10.0.1.0/24`	Rango de la subred pública

`main.tf`

```
locals
locals {
  dominio  = "${var.duckdns_subdominio}.duckdns.org"
  hash_web = sha1(join("", [for f in sort(fileset("${path.module}/web", "**")) : "${f}:$
{filesha1("${path.module}/web/${f}")}"]))
}
```

- `dominio` monta el nombre completo: `almichain2.duckdns.org`.
- `hash_web` es una "huella" de todos los ficheros de `web/`: calcula el SHA-1 de cada fichero y luego el de todos juntos. Si cambia cualquier fichero, cambia la huella, y Terraform sabe que tiene que volver a subir la web.

Red

Recurso	Qué hace
`data.aws_availability_zones.disponibles`	**Consulta** (no crea) qué zonas de disponibilidad hay en la región, para poner la subred en la primera (`us-east-1a`)
`aws_vpc.principal`	Crea la VPC `10.0.0.0/16` con resolución DNS activada
`aws_internet_gateway.igw`	Crea el Internet Gateway y lo conecta a la VPC
`aws_subnet.publica`	Crea la subred `10.0.1.0/24`.

Recurso	Qué hace
	`map_public_ip_on_launch = true` da IP pública a las máquinas al arrancar, para que tengan Internet antes de recibir la IP elástica
`aws_route_table.publica`	Tabla con la ruta `0.0.0.0/0 → Internet Gateway`: todo lo que no es de la VPC sale a Internet
`aws_route_table_association.publica`	Asocia esa tabla a la subred. Esto es lo que la convierte en “pública”

Grupo de seguridad

`aws_security_group.web` es el firewall de la máquina:

Dirección	Puerto	Origen	Motivo
Entrada	80/TCP	Todo Internet	Let’s Encrypt valida el dominio por aquí, y Nginx redirige a HTTPS
Entrada	443/TCP	Todo Internet	La web en HTTPS
Entrada	22/TCP	`var.ssh_cidr`	Terraform entra para subir la web; también sirve para administrar
Salida	Todos	Todo Internet	Instalar paquetes, hablar con DuckDNS y Let’s Encrypt

Clave SSH

```
resource "tls_private_key" "ssh"            # genera un par de claves RSA de 4096 bits
resource "aws_key_pair" "web"               # registra la parte pública en AWS como
"almibites-key"
resource "local_sensitive_file" "clave_privada"  # guarda la privada en almibites-key.pem
(permisos 0400)
```

AWS mete la clave pública en el usuario `ubuntu` de la máquina. Con la privada, Terraform (y tú) podéis entrar por SSH.

IP elástica

`aws_eip.web` reserva una IP pública fija. Lleva `depends_on = [aws_internet_gateway.igw]` porque AWS no deja usar IPs elásticas en una VPC sin gateway.

Servidor EC2

```
data "aws_ami" "ubuntu" { ... }   # busca la imagen más reciente de Ubuntu 24.04 publicada
por Canonical
```

`099720109477` es el ID de la cuenta oficial de Canonical, la empresa de Ubuntu. Así nos aseguramos de que la imagen es la oficial.

```
resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = var.tipo_instancia
  subnet_id              = aws_subnet.publica.id
  vpc_security_group_ids = [aws_security_group.web.id]
  key_name               = aws_key_pair.web.key_name
  root_block_device { volume_size = 8, volume_type = "gp3" }
  user_data = templatefile("user_data.sh.tftpl", { dominio = ..., eip = ..., ... })
  lifecycle { ignore_changes = [ami] }
}
```

- `user_data` es un script que la máquina ejecuta **solo la primera vez que arranca**. `templatefile` coge `user_data.sh.tftpl` y sustituye las variables `${dominio}`, `${eip}`, etc. por los valores reales.
- `ignore_changes = [ami]`: cuando Canonical publica una imagen nueva de Ubuntu, Terraform no recrea la máquina. Si la recreara, se perdería el certificado y habría que pedir otro.

`aws_eip_association.web` une la IP elástica con la máquina.

Subida de la web

```
resource "terraform_data" "subir_web" {
  triggers_replace = [aws_instance.web.id, local.hash_web]
  depends_on       = [aws_eip_association.web]
  connection { type = "ssh", host = aws_eip.web.public_ip, user = "ubuntu", private_key
= ... }

  provisioner "remote-exec" { inline = ["rm -rf /tmp/web"] }
  provisioner "file"        { source = "web", destination = "/tmp" }
  provisioner "remote-exec" { inline = [ "cloud-init status --wait", "sudo cp -r
/tmp/web/. /var/www/almibites/", ... ] }
}
```

- `terraform_data` es un recurso “vacío” que sirve para ejecutar acciones.
- `triggers_replace`: se vuelve a ejecutar si cambia la máquina o la huella de la web. Si no cambia nada, `apply` no vuelve a subirla.
- Los **provisioners** se ejecutan en orden:
  1. Borra `/tmp/web` en el servidor, por si quedaba una copia antigua.
  2. Copia la carpeta `web/` del PC a `/tmp/web` del servidor por SSH (SCP).
  3. Espera a que termine el script de arranque (`cloud-init status --wait`), copia la web a `/var/www/almibites`, le da permisos a `www-data` (el usuario de Nginx) y enseña el resultado de la instalación.

`user_data.sh.tftpl`

Script Bash que ejecuta la EC2 al arrancar. Todo lo que hace queda registrado en `/var/log/almibites-setup.log` (permisos 600, solo root), salvo el token de DuckDNS: el trazado `set -x` se apaga mientras se usa.

Fase	Qué hace
1. Paquetes	`apt-get install nginx certbot python3-certbot-nginx dnsutils curl`
2. Carpeta web	Crea `/var/www/almibites` con una página provisional “Subiendo la web…”
Nginx	Escribe `/etc/nginx/sites-available/almibites` con `server_name almichain2.duckdns.org`, cabeceras de seguridad y caché de 7 días para CSS e imágenes. Lo activa y desactiva el sitio por defecto
3. DuckDNS	Llama a `https://www.duckdns.org/update?domains=almichain2&token=...&ip=IP_ELASTICA` hasta que responde `OK`. Si responde `KO` (token o subdominio incorrectos), lo apunta como `ERROR` en el log y para
Esperas	Espera a que la IP elástica esté asociada (lo consulta en los metadatos de la instancia, IMDSv2) y a que el DNS de DuckDNS devuelva esa IP (`dig @ns1.duckdns.org`). Sin esto, Let’s Encrypt fallaría

Fase	Qué hace
4. Certificado	`certbot --nginx -d almichain2.duckdns.org --redirect`. Certbot pide el certificado, modifica la configuración de Nginx para escuchar en el 443 con el certificado y añade la redirección 80 → 443. Lo reintenta hasta 3 veces

Detalle técnico: dentro de una plantilla de Terraform, `${...}` se sustituye por una variable. Por eso en el script las variables de Bash se escriben como `$VARIABLE`, sin llaves, y Terraform no las toca.

`outputs.tf`

Lo que muestra Terraform al terminar, o con `terraform output`:

Output	Ejemplo
`url`	`https://almichain2.duckdns.org`
`ip_publica`	`3.215.x.x`
`ssh`	`ssh -i almibites-key.pem ubuntu@3.215.x.x`
`ver_log_instalacion`	`sudo tail -n 40 /var/log/almibites-setup.log`

`web/geo.js` (geolocalización)

1. Al pulsar **¿A cuánto estoy?** comprueba que la página va por HTTPS (`window.isSecureContext`) y que el navegador soporta geolocalización.
2. Llama a `navigator.geolocation.getCurrentPosition()`. El navegador pide permiso al usuario.
3. Calcula la distancia en línea recta al restaurante (43.2715, -2.9470) con la **fórmula del haversine**, que tiene en cuenta la curvatura de la Tierra.
4. Si está a 5 km o menos, estima los minutos andando (a 4,5 km/h).
5. Genera un enlace a OpenStreetMap con la ruta desde la ubicación del usuario: a pie si está cerca, en coche si está lejos.
6. Si el usuario deniega el permiso, o hay un error, muestra un mensaje explicándolo.

`.gitignore`

```
.terraform/        # proveedores descargados (se regeneran con terraform init)
*.tfstate          # estado de Terraform (lleva datos sensibles)
*.tfstate.*
terraform.tfvars   # token de DuckDNS y correo
crash.log
*.pem              # clave privada SSH
```

`.terraform.lock.hcl` **sí** se sube: fija las versiones exactas de los proveedores, para que todos usen las mismas.

Anexo E. Herramientas: cómo se hicieron las VM, las pruebas y las capturas

Esta carpeta no es un bloque del reto. Está para que cualquiera pueda **comprobar y repetir** cómo se han hecho las tres máquinas virtuales exportadas (`.ova`), sus pruebas y las capturas de terminal. Sin ella, las OVA serían una caja negra.

Qué hay

Carpeta	Para qué
`herramientas/construir-vm`	Scripts que construyen los discos de ALMI-ODOO, ALMI-BD y ALMI-ARCADE y los empaquetan en `.ova`
`herramientas/probar-vm`	Scripts para arrancar esos discos en QEMU, manejarlos por consola serie, guardar la transcripción de cada prueba y dibujar la captura de terminal a partir de ella
`herramientas/memoria`	Generador de la memoria en Word/PDF a partir de los README

Cómo se construyen las VM (y en qué se diferencia de los README)

Los README de los bloques 02, 05 y 07 explican la instalación **como la haríamos en clase**: VirtualBox, ISO de Ubuntu Server 24.04 y el instalador. Las OVA **no** se hicieron así, porque donde se preparó el repositorio no había VirtualBox (es un servidor Linux sin entorno gráfico). Se hicieron así:

1. `debootstrap` instala Ubuntu 24.04 (noble) en un disco `.raw` montado con un dispositivo *loop*.
2. Dentro (con `chroot`) se instalan los mismos paquetes y se copian **los mismos archivos del repositorio** que usan los README: `docker-compose.yml`, `odoo.conf`, `nginx-odoo.conf`, los scripts de copias, del RAID y de primer arranque, el SQL, la configuración de la recreativa…
3. GRUB para arranque por BIOS, con un `grub.cfg` escrito a mano (por UUID).
4. `herramientas/construir-vm/hacer-ova.py` convierte los discos a VMDK y escribe el `.ovf` y el manifiesto `.mf` con las huellas SHA-256.

Diferencias con una instalación hecha con el instalador: **no hay LVM** (una sola partición ext4), no se instala nada que el instalador ponga y aquí no haga falta (por eso es más pequeña), y la imagen no lleva **claves de servidor ni contraseñas de servicio**: cada VM importada pone el cortafuegos antes de que haya red y genera en su primer arranque sus claves de host SSH, su certificado HTTPS, sus contraseñas de PostgreSQL, la maestra de Odoo, las de los usuarios de Odoo y el secreto de sesiones de Odoo (ODOO), y las de los usuarios de MariaDB (BD). La contraseña de `almiadmin` es una inicial que **obliga a cambiarla** la primera vez que se entra (`chage -d 0`).

Lo que comparten todas las copias de las OVA es la contraseña inicial de `almiadmin` (obligatorio cambiarla) y la pareja de claves de las **copias cruzadas** (para que funcionen nada más importar las dos VM). Esa clave solo puede escribir en la carpeta de copias del otro servidor y solo desde su IP (`rrsync -wo -no-del`, `from=`). Para cambiarla, ver docs/maquinas-virtuales.md (`docs/maquinas-virtuales.md`).

Construirlas

En un Ubuntu 24.04 con `debootstrap`, `qemu-utils`, `grub-pc-bin` y `docker` (para sacar los datos de Odoo), como root:

```
cd herramientas/construir-vm
# Contraseñas iniciales (FUERA del repositorio): ALMIADMIN_PASS=... y ARCADE_PASS=...
export CREDENCIALES=/root/credenciales-vm.txt CLAVES=/root/claves-copias
SALIDA=/var/tmp/almi-vm

./sacar-datos-odoo.sh /var/tmp/datos-odoo /opt/odoo   # de un Odoo ya configurado (configurar_odoo.py)
./construir-odoo.sh /var/tmp/datos-odoo
./construir-bd.sh
./construir-arcade.sh
```

```
./hacer-ova.py ALMI-ODOO 3072 2 "Servidor Odoo 18" $SALIDA/ALMI-ODOO.raw
./hacer-ova.py ALMI-BD 2048 2 "Servidor MariaDB con RAID 1" $SALIDA/ALMI-BD.raw
$SALIDA/ALMI-BD-raid1.raw $SALIDA/ALMI-BD-raid2.raw
./hacer-ova.py --audio ALMI-ARCADE 1024 1 "Recreativa MAME" $SALIDA/ALMI-ARCADE.raw
./partir-ova.sh $SALIDA/ALMI-ODOO.ova      # trozos de 380 MB + huella
```

Si hace falta un proxy con certificado propio para descargar paquetes: `APT_CA=/ruta/ca.crt`.

Cómo se prueban

`herramientas/probar-vm/qemu-vm.sh` arranca un disco en QEMU **a través de una capa qcow2**, así la prueba (primer arranque, RAID, cambio de contraseña…) no toca el disco que se empaqueta. Dos modos:

- `grub`: arranca como VirtualBox (BIOS → GRUB del disco). Sirve para ver que la imagen entregada arranca sola; se mira la pantalla con `qemu-vm.sh pantalla`.
- `serie`: arranca el núcleo de la propia imagen añadiendo `console=ttyS0`, para manejarla por consola serie con `herramientas/probar-vm/consola.py`.

Todas las VM arrancadas así comparten una red (como la Red NAT ALMICHAIN), así que ALMI-ODOO y ALMI-BD pueden hacerse las copias cruzadas.

Los guiones de prueba se ejecutan en este orden (cada uno deja su transcripción en la carpeta `capturas/` del bloque):

```
cd herramientas/probar-vm
export CLAVE_INICIAL=... CLAVE_NUEVA=...          # contraseña inicial de almiadmin y la
nueva
./pruebas-odoo.sh  $SALIDA ../../05-erp-odoo/capturas 2560    # deja ALMI-ODOO encendida
./pruebas-bd.sh    $SALIDA ../../07-bd-raid-copias/capturas   # deja ALMI-BD encendida
./pruebas-copias.sh ../../05-erp-odoo/capturas ../../07-bd-raid-copias/capturas
./pruebas-arcade.sh $SALIDA ../../02-linux-recreativa/capturas
```

Límites de estas pruebas (dichos también en cada README):

- Es **QEMU sin KVM**, no VirtualBox: el procesador está emulado (lento; no sirve para medir rendimiento) y no se prueba el importador de VirtualBox, su controlador gráfico ni su audio.
- No hay Internet dentro de la red de pruebas: el envío de correo de Odoo y las actualizaciones no se prueban.

Cómo se hacen las capturas de terminal

1. `herramientas/probar-vm/consola.py transcribir` ejecuta las órdenes en la VM y guarda **la salida real** en un `.txt` (`$ orden` y debajo lo que sale). Solo se le quitan dos cosas, y siempre a propósito: los mensajes del núcleo que salen mezclados por la consola serie (líneas `[ 123.45…]`) y los *hash* de contraseñas, que se sustituyen por `…(hash recortado)….`
2. `herramientas/probar-vm/pintar-terminal.py` dibuja ese `.txt` como una ventana de terminal (`.png`). Añade solo el *prompt* (`usuario@host:~$`, siempre con `~`) y el título de la ventana; el resto es el texto del `.txt`.

Por eso las capturas de terminal hechas así tienen al lado un `.txt` con el mismo nombre. Las capturas del navegador (Odoo) se hicieron con Chromium sin interfaz y la barra de dirección está dibujada. Las capturas de las carpetas `05-erp-odoo/capturas/01–35` y `07-bd-raid-copias/capturas/01–12` son de un **servidor de pruebas** (ver “Origen de las evidencias” en el README principal (`README.md`)), no de las VM.

Cómo se genera la memoria

`herramientas/memoria/generar.sh` junta los README en `docs/Documentacion-Reto1-Almi-Chain.docx` y `.pdf`. Necesita pandoc 3, `python3-docx`, Pillow, `mmdc` (la CLI de Mermaid) y LibreOffice. Los enlaces

a archivos del repositorio salen como su ruta (el repositorio es privado), las figuras se numeran solas y los diagramas Mermaid se convierten en imágenes.

Las capturas no son correlativas en algunos bloques (en 05 faltan la 20 y la 24; en 07, de la 13 a la 17) porque se quitaron capturas repetidas o de una versión anterior de la VM; se mantiene la numeración para no romper referencias.